```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Farewell 2025 | Cornerstone International School</title>
  <meta name="description" content="Celebrating the Class of 2025 - Share and cherish our memories together" />

  <link rel="preconnect" href="https://fonts.gstatic.com/" />
  <link rel="preconnect" href="https://fonts.gstatic.com/" crossorigin />
  <link rel="https://fonts.googleapis.com/css2?family=Playfair+Display:wghs@400;500;600;700&family=Poppins:wghs@300;400;500;600;700&family=Cancing+Script:wghs@700&display=swap" id="stylesheet" />

  <style>
  :root {
    --primary-gold: #D4AF37;
    --primary-gold-light: #F4E5B2;
    --primary-gold-dark: #B8860B;
    --navy-dark: #1a1a2e;
    --navy-medium: #16213e;
    --navy-light: #0f3460;
    --accent-rose: #e94560;
    --accent-purple: #7B2BBE;
    --text-light: #ffffff;
    --text-muted: rgba(255, 255, 255, 0.7);
    --text-dark: #1a1a2e;
    --glass-bg: rgba(255, 255, 255, 0.1);
    --glass-border: rgba(255, 255, 255, 0.2);
    --success-green: #4CAF50;
    --error-red: #E4E3A;

    --gradient-gold: linear-gradient(135deg, #D4AF37 0%, #F4E5B2 50%, #D4AF37 100%);
    --gradient-navy: linear-gradient(135deg, #1a1a2e 0%, #16213e 50%, #0f3460 100%);
    --gradient-hero: linear-gradient(180deg, rgba(26, 26, 46, 0.9) 0%, rgba(22, 33, 62, 0.8) 100%);

    --font-display: 'Playfair Display', serif;
    --font-body: 'Poppins', sans-serif;
    --font-script: 'Dancing Script', cursive;

    --section-padding: 100px;
    --container-width: 1400px;

    --shadow-soft: 0 4px 20px rgba(0, 0, 0, 0.3);
    --shadow-gold: 0 4px 30px rgba(212, 175, 55, 0.3);
    --transition-smooth: all 0.4s cubic-bezier(0.4, 0, 0.2, 1);
  }

  /* Theme overrides injected from settingsJsonme */
  /* these are updated dynamically via JS by setting style props on documentElement */


  * { margin: 0; padding: 0; box-sizing: border-box; }
  html {
scroll-behavior: auto;
}

  body {
    font-family: var(--font-body);
    background: var(--navy-dark);
    color: var(--text-light);
    line-height: 1.6;
    overflow-x: hidden;
    }

    ::-webkit-scrollbar { width: 10px; }
    ::-webkit-scrollbar-track { background: var(--navy-dark); }
    ::-webkit-scrollbar-thumb { background: var(--primary-gold); border-radius: 5px; }
    ::-webkit-scrollbar-thumb:hover { background: var(--primary-gold-dark); }

    @keyframes float { 0%, 100%{transform:translateY(0) rotate(0deg);} 50%{transform:translateY(-20px) rotate(5deg);} }
    @keyframes pulse { 0%, 100%{opacity:1;} 50%{opacity:0.5;} }
    @keyframes fadeInUp { from{opacity:0; transform:translateY(30px);} to{opacity:1; transform:translateY(0);} }
    @keyframes scaleIn { from{opacity:0; transform:scale(0.9);} to{opacity:1; transform:scale(1);} }
    @keyframes confettiFall { 0%{transform:translateY(-100vh) rotate(0deg); opacity:1;} 100%{transform:translateY(100vh) rotate(720deg); opacity:0;} }
    @keyframes slideInRight { from{opacity:0; transform:translateX(50px);} to{opacity:1; transform:translateX(0);} }
    @keyframes heartBeat { 0%{transform:scale(1);} 14%{transform:scale(1.3);} 28%{transform:scale(1);} 42%{transform:scale(1.3);} 70%{transform:scale(1);} }
    @keyframes spin { to { transform: rotate(360deg); } }
    @keyframes bubble { from { opacity: 0; } to { opacity: 1; } }

    animate-fadeInUp { animation: fadeInUp 0.8s ease forwards; }
    animate-scaleIn { animation: scaleIn 0.6s ease forwards; }

    #particles-container {
      position: fixed;
      top: 0; left: 0;
      width: 100%; height: 100%;
      pointer-events: none;
      z-index: 0;
      overflow: hidden;
      }
      .particle {
      position: absolute;
      width: 4px; height: 4px;
      background: var(--primary-gold);
      border-radius: 50%;
      opacity: 0.3;
      animation: float 8s infinite ease-in-out;
      }

      .navbar {
      position: fixed;
      top: 0; left: 0; right: 0;
      z-index: 1000;
      padding: 20px 20px;
      display: flex;
      justify-content: space-between;
      align-items: center;
      transition: var(--transition-smooth);
      background: transparent;
      }
      .navbar.scrolled {
      background: rgba(26, 26, 46, 0.95);
      backdrop-filter: blur(20px);
      padding: 15px 50px;
      box-shadow: var(--shadow-soft);
      }
      .logo {
      display: flex;
      align-items: center;
      gap: 15px;
      cursor: pointer;
      user-select: none;
      }
      .logo-icon {
      width: 50px; height: 50px;
      background: var(--gradient-gold);
      border-radius: 50%;
      display: flex;
      align-items: center;
      justify-content: center;
      font-size: 24px;
      box-shadow: var(--shadow-gold);
      transition: var(--transition-smooth);
      }
      .logo:hover .logo-icon { transform: rotate(360deg); }
      .logo-text { display: flex; flex-direction: column; }
      .logo-text .school-name {
      font-family: var(--font-display);
      font-size: 1.2rem;
      font-weight: 600;
      color: var(--primary-gold);
      }
      .logo-text .event-name {
      font-size: 0.8rem;
      color: var(--text-muted);
      letter-spacing: 2px;
      text-transform: uppercase;
      }

      .nav-links { display: flex; gap: 40px; list-style: none; }
      .nav-links a {
      color: var(--text-light);
      text-decoration: none;
      font-weight: 500;
      font-size: 0.95rem;
      position: relative;
      padding: 5px 0;
      transition: var(--transition-smooth);
      }
      .nav-links a::after {
      content: '';
      position: absolute;
      bottom: 0; left: 0;
      width: 0; height: 2px;
      background: var(--primary-gold);
      transition: var(--transition-smooth);
      }
      .nav-links a:hover { color: var(--primary-gold); }
      .nav-links a:hover::after { width: 100%; }
      .nav-cta {
      background: var(--gradient-gold);
      color: var(--navy-dark) !important;
      padding: 12px 28px !important;
      border-radius: 25px;
      font-weight: 600 !important;
      box-shadow: var(--shadow-gold);
      transition: var(--transition-smooth) !important;
      }
      .nav-cta:after { display: none !important; }
      .nav-cta:hover { transform: translateY(-3px); box-shadow: 0 6px 30px rgba(212, 175, 55, 0.5); }

      .mobile-menu-btn { display: none; flex-direction: column; gap: 5px; cursor: pointer; padding: 10px; }
      .mobile-menu-btn span { width: 25px; height: 3px; background: var(--primary-gold); border-radius: 2px; transition: var(--transition-smooth); }

      .hero {
      min-height: 100vh;
      display: flex;
      align-items: center;
      justify-content: center;
      text-center: center;
      overflow: hidden;
      background: var(--gradient-navy);
      }
      .hero-bg {
      position: absolute; top:0; left:0; right:0; bottom:0;
      background:
        radial-gradient(ellipse at 20% 80%, rgba(212, 175, 55, 0.15) 0%, transparent 50%),
        radial-gradient(ellipse at 80% 20%, rgba(123, 45, 142, 0.1) 0%, transparent 50%),
        radial-gradient(ellipse at 50% 50%, rgba(15, 52, 96, 0.3) 0%, transparent 70%);
      }
      .hero-content {
      text-align: center;
      z-index: 10;
      padding: 0 20px;
      max-width: 900px;
      }
      .hero-badge {
      display: inline-block;
      background: var(--glass-bg);
      border: 1px solid var(--glass-border);
      padding: 10px 25px;
      border-radius: 30px;
      font-size: 0.9rem;
      color: var(--primary-gold);
      margin-bottom: 30px;
      backdrop-filter: blur(10px);
      animation: fadeInUp 0.8s ease;
      }
      .hero-title {
      font-family: var(--font-display);
      font-size: clamp(4.0rem, 8vw, 6rem);
      font-weight: 700;
      line-height: 1.1;
      margin-bottom: 20px;
      animation: fadeInUp 0.8s ease 0.2s both;
      }
      .hero-title .highlight {
      background: var(--gradient-gold);
      -webkit-background-clip: text;
      -webkit-text-fill-color: transparent;
      background-clip: text;
      }
      .hero-subtitle {
      font-family: var(--font-script);
      font-size: clamp(1.5rem, 4vw, 2.5rem);
      color: var(--primary-gold-light);
      margin-bottom: 20px;
      animation: fadeInUp 0.8s ease 0.4s both;
      }

      .hero-description {
      font-size: 1.2rem;
      color: var(--text-muted);
      max-width: 600px;
      margin: 0 auto 40px;
      animation: fadeInUp 0.8s ease 0.6s both;
      }

      .hero-buttons {
      display: flex;
      gap: 20px;
      justify-content: center;
      flex-wrap: wrap;
      animation: fadeInUp 0.8s ease 0.8s both;
      }


      .btn {
      padding: 16px 36px;
      border-radius: 30px;
      font-size: 1rem;
      font-weight: 600;
      text-decoration: none;
      display: inline-flex;
      align-items: center;
      gap: 10px;
      transition: var(--transition-smooth);
      cursor: pointer;
      border: none;
      font-family: var(--font-body);
      }
      .btn-primary { background: var(--gradient-gold); color: var(--navy-dark); box-shadow: var(--shadow-gold); }
      .btn-primary:hover { transform: translateY(-3px) scale(1.05); box-shadow: 0 6px 40px rgba(212, 175, 55, 0.6); }
      .btn-secondary { background: transparent; color: var(--text-light); border: 2px solid var(--glass-border); backdrop-filter: blur(10px); }
      .btn-secondary:hover { background: var(--glass-bg); border-color: var(--primary-gold); color: var(--primary-gold); }
      .btn-danger { background: rgba(244,67,54,0.2); border: 1px solid rgba(244,67,54,0.6); color: #ffb3ad; }
      .btn-danger:hover { background: rgba(244,67,54,0.35); }


      section { padding: var(--section-padding) 20px; position: relative; }
      .container { max-width: var(--container-width); margin: 0 auto; }

      .section-header { text-align: center; margin-bottom: 60px; }
      .section-badge {
      display: inline-block;
      background: rgba(212, 175, 55, 0.1);
      border: 1px solid rgba(212, 175, 55, 0.3);
      padding: 8px 20px;
      border-radius: 20px;
      font-size: 0.85rem;
      color: var(--primary-gold);
      margin-bottom: 20px;
      text-transform: uppercase;
      letter-spacing: 2px;
      }
      .section-title {
      font-family: var(--font-display);
      font-size: clamp(3.0rem, 5vw, 3.5rem);
      font-weight: 700;
      margin-bottom: 15px;
      }
      .section-title .highlight {
      background: var(--gradient-gold);
      -webkit-background-clip: text;
      -webkit-text-fill-color: transparent;
      background-clip: text;
      }

      .section-description {
      font-size: 1.1rem;
      color: var(--text-muted);
      max-width: 700px;
      margin: 0 auto;
      }

      /* Countdown */
      #countdown { background: linear-gradient(180deg, var(--navy-dark) 0%, var(--navy-medium) 100%); }
      .countdown-container { display: flex; justify-content: center; gap: 30px; flex-wrap: wrap; }
      .countdown-item {
      background: var(--glass-bg);
      border: 1px solid var(--glass-border);
```

```
    border-radius: 20px;
    padding: 30px 40px;
    text-align: center;
    backdrop-filter: blur(10px);
    min-width: 140px;
    transition: var(--transition-smooth);
  }
  .countdown-item:hover { transform: translateY(-10px); border-color: var(--primary-gold); box-shadow: var(--shadow-gold); }
  .countdown-number {
    font-family: var(--font-display);
    color: var(--primary-gold);
    font-weight: 700;
    background: var(--gradient-gold);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
    background-clip: text;
    line-height: 1;
  }
  .countdown-label { font-size: 0.9rem; color: var(--text-muted); text-transform: uppercase; letter-spacing: 2px; margin-top: 10px; }

  .event-date-display {
    text-align: center;
    margin-top: 40px;
    padding: 20px;
    background: rgba(212, 175, 55, 0.1);
    border-radius: 15px;
    border: 1px solid rgba(212, 175, 55, 0.2);
  }
  .event-date-display p { font-size: 1.2rem; color: var(--primary-gold); }
  .event-date-display p span { font-weight: 700; }

  /* Upload */
  #upload { background: var(--navy-medium); }
  .upload-container { max-width: 800px; margin: 0 auto; }
  .upload-card { background: var(--glass-bg); border: 1px solid var(--glass-border); border-radius: 30px; padding: 50px; backdrop-filter: blur(20px); }

  .callout {
    margin: 0 0 20px;
    padding: 14px 16px;
    border-radius: 14px;
    border: 1px solid rgba(255,255,255,0.2);
    background: rgba(255,255,255,0.06);
    color: var(--text-muted);
  }

  .callout strong { color: var(--text-light); }

  .dropzone {
    border: 3px dashed var(--glass-border);
    border-radius: 20px;
    padding: 60px 20px;
    text-align: center;
    transition: var(--transition-smooth);
    cursor: pointer;
    background: rgba(255, 255, 255, 0.02);
  }
  .dropzone:hover, .dropzone.drag-over { border-color: var(--primary-gold); background: rgba(212, 175, 55, 0.05); }
  .dropzone-icon { font-size: 4rem; margin-bottom: 20px; }
  .dropzone-title { font-size: 1.5rem; font-weight: 600; margin-bottom: 10px; }
  .dropzone-subtitle { color: var(--text-muted); margin-bottom: 20px; }
  .dropzone-formats { font-size: 0.85rem; color: var(--text-muted); }
  .dropzone-formats span { background: rgba(212, 175, 55, 0.2); padding: 4px 10px; border-radius: 10px; margin: 0 3px; color: var(--primary-gold); }
  #file-input { display: none; }

  .file-preview-container { margin-top: 30px; display: none; }
  .file-preview-container.has-files { display: block; }
  .file-preview-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(120px, 1fr)); gap: 15px; margin-bottom: 20px; }
  .file-preview-item { position: relative; aspect-ratio: 1; border-radius: 12px; overflow: hidden; background: var(--navy-dark); }
  .file-preview-item img, .file-preview-item video { width: 100%; height: 100%; object-fit: cover; }
  .file-preview-item .video-indicator {
    position: absolute;
    top: 50%; left: 50%;
    transform: translate(-50%, -50%);
    width: 40px; height: 40px;
    background: rgba(0,0,0,0.7);
    border-radius: 50%;
    display: flex; align-items: center; justify-content: center;
    font-size: 1.2rem;
  }
  .file-preview-item .remove-file {
    position: absolute;
    top: 5px; right: 5px;
    width: 24px; height: 25px;
    background: var(--error-red);
    border: none; border-radius: 50%;
    color: white;
    cursor: pointer;
    display: flex; align-items: center; justify-content: center;
    font-size: 0.8rem;
    opacity: 0;
    transition: var(--transition-smooth);
  }
  .file-preview-item:hover .remove-file { opacity: 1; }
  .file-info { font-size: 0.8rem; color: var(--text-muted); margin-bottom: 20px; }

  .upload-form { margin-top: 30px; }
  .form-group { margin-bottom: 25px; }
  .form-group label { display: block; margin-bottom: 10px; font-weight: 500; color: var(--text-light); }
  .form-group label .required { color: var(--accent-rose); }

  .form-input, .form-select, .form-textarea {
    width: 100%;
    padding: 16px 20px;
    background: rgba(255,255,255,0.05);
    border: 2px solid var(--glass-border);
    border-radius: 15px;
    color: var(--text-light);
    font-size: 1rem;
    font-family: var(--font-body);
    transition: var(--transition-smooth);
  }
  .form-input:focus, .form-select:focus, .form-textarea:focus {
    outline: none;
    border-color: var(--primary-gold);
    background: rgba(255,255,255,0.08);
  }
  .form-input::placeholder, .form-textarea::placeholder { color: var(--text-muted); }

  .form-select {
    cursor: pointer;
    appearance: none;
    background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' width='24' height='24' viewBox='0 0 24 24' fill='none' stroke='%23D4AF37' stroke-width='2' stroke-linecap='round' stroke-linejoin='round'%3E%3Cpolyline points='6 9 12 15 18 9'%3E%3C/polyline%3E%3C/svg%3E");
    background-repeat: no-repeat;
    background-position: right 15px center;
    background-size: 20px;
    }
  .form-select option { background: var(--navy-dark); color: var(--text-light); }
  .form-textarea { min-height: 120px; resize: vertical; }

  .char-counter { text-align: right; font-size: 0.85rem; color: var(--text-muted); margin-top: 5px; }
  .char-counter.warning { color: var(--error-red); }

  .upload-progress { display: none; margin-top: 20px; }
  .upload-progress.active { display: block; }
  .progress-bar { height: 10px; background: rgba(255,255,255,0.1); border-radius: 10px; overflow: hidden; margin-bottom: 10px; }
  .progress-fill { height: 100%; background: var(--gradient-gold); border-radius: 10px; width: 0%; transition: width 0.3s ease; }
  .progress-text { text-align: center; color: var(--text-muted); }

  .upload-success { display: none; text-align: center; padding: 40px; }
  .upload-success.active { display: block; animation: fadeIn 0.5s ease; }
  .success-icon { font-size: 5rem; margin-bottom: 20px; }
  .success-title { font-size: 1.8rem; font-weight: 700; margin-bottom: 10px; color: var(--primary-gold); }
  .success-message { color: var(--text-muted); margin-bottom: 20px; }

  @media (max-width: 768px) {
    .nav-bar { padding: 15px 20px; }
    .nav-links {
      position: fixed; top:0 left:0; right:0; bottom:0;
      background: rgba(10,10,20,0.95);
      flex-direction: column;
      justify-content: center;
      align-items: center;
      gap: 30px;
      transform: translateX(-100%);
      transition: var(--transition-smooth);
    }
    .nav-links.active { transform: translateX(0); }
    .mobile-menu-btn { display: flex; z-index: 1001; }
    .mobile-menu-btn.active span:nth-child(1) { transform: rotate(45deg) translate(5px, 5px); }
    .mobile-menu-btn.active span:nth-child(2) { opacity: 0; }
    .mobile-menu-btn.active span:nth-child(3) { transform: rotate(-45deg) translate(7px, -6px); }
    section { padding: 60px 15px; }
    .upload-card { padding: 30px 20px; }
    .dropzone { padding: 40px 20px; }
    .countdown-container { gap: 15px; }
    .countdown-item { padding: 20px 30px; min-width: 100px; }
    .countdown-number { font-size: 2.5rem; }
  }
</style>

<style id="customcss-patch-v6">
  /* Intro Video Overlay */
  .intro-text { position: absolute; top: 50%; left: 50%; transform: translate(-50%, -50%); padding: 15px 30px; font-size: 1.2rem; background: var(--primary-gold); color: var(--navy-dark); font-weight: bold; border: none; border-radius: 30px; cursor: pointer; z-index: 15; box-shadow: 0 4px 20px rgba(0,0,0,0.5); animation: pulseBtn 2s infinite; display: none; }
  @keyframes pulseBtn { 0% { transform: translate(-50%, -50%) scale(1); } 50% { transform: translate(-50%, -50%) scale(1.05); } 100% { transform: translate(-50%, -50%) scale(1); } }

  .intro-video-overlay { position: fixed; top: 0; left: 0; right: 0; bottom: 0; background: #000; z-index: 9999; display: flex; align-items: center; justify-content: center; }
  .intro-video-overlay.hidden { display: none !important; }
  .intro-video { width: 100%; height: 100%; object-fit: cover; }
  .skip-intro-btn { position: absolute; top: 20px; right: 20px; padding: 12px 24px; background: rgba(255,255,255,0.15); border: 1px solid rgba(255,255,255,0.3); color: white; border-radius: 25px; cursor: pointer; font-family: var(--font-body); font-size: 0.95rem; backdrop-filter: blur(10px); transition: var(--transition-smooth); z-index: 10; }
  .skip-intro-btn:hover { background: rgba(255,255,255,0.25); transform: scale(1.02); }
  .toggle-intro-controls { position: absolute; bottom: 20px; right: 20px; padding: 10px 15px; background: rgba(0,0,0,0.5); border: none; color: white; border-radius: 50%; cursor: pointer; font-size: 1.2rem; }

  /* Compilations */
  .compilation-player-overlay { position: fixed; top: 0; left: 0; right: 0; bottom: 0; background: #000; z-index: 8000; display: none; align-items: center; justify-content: center; }
  .compilation-player-overlay.active { display: flex !important; }
  .compilation-slide { position: absolute; top: 0; left: 0; right: 0; bottom: 0; display: flex; align-items: center; justify-content: center; flex-direction: column; opacity: 0; transition: opacity 0.8s ease, transform 0.8s ease; pointer-events: none; }
  .compilation-slide.active { opacity: 1; pointer-events: auto; }
  .compilation-slide img { max-width: 90%; max-height: 80vh; object-fit: contain; border-radius: 12px; box-shadow: 0 20px 60px rgba(0,0,0,0.5); }
  .compilation-caption { position: absolute; bottom: 40px; left: 50%; transform: translateX(-50%); background: rgba(0,0,0,0.75); padding: 10px 15px; border-radius: 20px; color: white; font-size: 1.3vmin; max-width: 85%; text-align: center; backdrop-filter: blur(10px); border: 1px solid rgba(255,255,255,0.2); color: white; width:60px; height: 60px; border-radius: 50%; cursor: pointer; font-size: 1.5rem; transition: var(--transition-smooth); display: flex; align-items: center; justify-content: center; }
  .compilation-nav-btn.hover { background: var(--primary-gold); color: var(--navy-dark); }
  .compilation-prev { left: 30px; }
  .compilation-next { right: 30px; }
  .compilation-close { position: absolute; top: 20px; right: 20px; background: rgba(255,255,255,0.1); border: 1px solid rgba(255,255,255,0.2); color: white; width: 50px; height: 50px; border-radius: 50%; cursor: pointer; font-size: 1.3rem; display: flex; align-items: center; justify-content: center; z-index: 8001;}
  .compilation-close:hover { background: rgba(244,67,54,0.8) }
  .compilation-progress { position: absolute; bottom: 25px; left: 50%; transform: translateX(-50%); display: flex; gap: 10px; }
  .compilation-dot { width: 12px; height: 12px; background: rgba(255,255,255,0.3); border-radius: 50%; cursor: pointer; transition: var(--transition-smooth); }
  .compilation-dot.hover { background: rgba(255,255,255,0.6); }
  .compilation-dot.active { background: var(--primary-gold); transform: scale(1.3); }

  .compilation-slide.trans-slide { transform: translateX(100%); }
  .compilation-slide.trans-slide.active { transform: translateX(0); }
  .compilation-slide.trans-zoom { transform: scale(0.7); }
  .compilation-slide.trans-zoom.active { transform: scale(1); }
  .compilation-slide.trans-flip { transform: rotateY(90deg); }
  .compilation-slide.trans-flip.active { transform: rotateY(0); }
  .compilation-slide.trans-blur { filter: blur(20px); }
  .compilation-slide.trans-blur.active { filter: blur(0); }

  .compilation-modal { position: fixed; top: 0; left: 0; right: 0; bottom: 0; background: rgba(0,0,0,0.9); z-index: 7000; display: none; align-items: flex-start; justify-content: center; padding: 40px 20px; overflow-y: auto; }
  .compilation-modal.active { display: flex !important; }
  .compilation-modal-content { background: var(--navy-medium); border: 1px solid var(--glass-border); border-radius: 20px; padding: 30px; max-width: 1200px; width: 100%; margin-top: 5vh; }
  .compilation-photo-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(120px, 1fr)); gap: 12px; max-height: 400px; overflow-y: auto; padding: 10px; background: rgba(0,0,0,0.2); border-radius: 12px; margin: 15px 0; }
  .compilation-photo-item { position: relative; aspect-ratio: 1; border-radius: 8px; overflow: hidden; cursor: pointer; border: 3px solid transparent; transition: var(--transition-smooth); }
  .compilation-photo-item.hover { border-color: rgba(212,175,55,0.5); }
  .compilation-photo-item.selected { border-color: var(--primary-gold); }
  .compilation-photo-item img { width: 100%; height: 100%; object-fit: cover; }
  .compilation-photo-item .photo-check { position: absolute; top: 5px; right: 5px; width: 24px; height: 24px; background: var(--primary-gold); border-radius: 50%; display: none; align-items: center; justify-content: center; color: var(--navy-dark); font-weight: bold; }
  .compilation-photo-item.selected .photo-check { display: flex; }

  .compilation-slide-preview { display: flex; gap: 12px; overflow-x: auto; padding: 15px 0; min-height: 120px; background: rgba(0,0,0,0.15); border-radius: 12px; margin: 15px 0; }
  .compilation-slide-preview-item { position: relative; flex-shrink: 0; background: rgba(255,255,255,0.05); border: 1px solid var(--glass-border); border-radius: 10px; padding: 8px; cursor: grab; }
  .compilation-slide-preview-img { width: 100%; aspect-ratio: 1; object-fit: cover; border-radius: 5px; }
  .compilation-slide-preview-input { width: 100%; margin-top: 6px; padding: 4px; font-size: 0.75rem; background: rgba(0,0,0,0.3); border: 1px solid var(--glass-border); border-radius: 4px; color: white; }
  .compilation-slide-preview .remove-slide { display: block; width: 100%; margin-top: 4px; padding: 4px; background: rgba(244,67,54,0.3); border: none; border-radius: 4px; color: #ffb3ad; cursor: pointer; font-size: 0.7rem; }

  /* Inline edit mode */
  .inline-editable { cursor: pointer; border: 1px dashed transparent; padding: 2px 4px; border-radius: 4px; transition: var(--transition-smooth); }
  .inline-editable:hover { border-color: var(--primary-gold); background: rgba(212, 175, 55, 0.1); }
  .inline-editable.editing { background: rgba(255,255,255,0.05); 0.1); border-color: var(--primary-gold); }
  .inline-dropzone { background: rgba(255,255,255,0.1); border: 1px solid var(--primary-gold); color: var(--text-light); padding: 4px 8px; border-radius: 5px; font-family: inherit; font-size: inherit; width: 100%; }
  </style>

</head>

<body>

  <!-- Intro Video Overlay -->
  <div id="introVideoOverlay" class="intro-video-overlay">
    <video id="introVideo" class="intro-video" autoplay playsinline></video>

    <div id="nameanswerOverlay" style="position: absolute; top: 0; left: 0; right: 0; bottom: 0; z-index: 15; display: none; align-items: center; justify-content: center; cursor: pointer; background: rgba(0,0,0,0.3)" onclick="SaveUsername()">
      <div style="padding: 15px 30px; font-size: 1.5rem; background: var(--primary-gold); color: var(--navy-dark); font-weight: bold; border-radius: 50px; box-shadow: 0 4px 20px rgba(0,0,0,0.3); pointer-events: none;">    Tap Anywhere for Sound</div>
    </div>
    <button id="skipIntroBtn" class="skip-intro-btn" style="z-index: 20;">Skip ...</button>
    <button id="toggleIntroControls" class="toggle-intro-controls" style="display:none;">    </button>
    </div>

    <div id="particles-container"></div>

    <nav class="navbar" id="navbar">
      <div class="logo" id="logo" title="Click 5 times for Admin login">
      <div class="logo-icon" id="siteLogoIcon">    </div>
      <div class="logo-text">
        <span class="school-name" id="siteSchoolName">Cornerstone</span>
        <span class="event-name" id="siteEventName">Farewell 2025</span>
      </div>
      </div>

      <ul class="nav-links" id="navLinks">
        <li><a href="#home">Home</a></li>
        <li><a href="#countdown">Countdown</a></li>
        <li><a href="#upload">Upload</a></li>
        <li><a href="#memories">Memories</a></li>
        <li><a href="#timeline">Journey</a></li>
        <li><a href="#compilations">Compilations</a></li>
        <li><a id="#upload" class="nav-cta">Upload Memory</a></li>
        </ul>
      <div class="mobile-menu-btn" id="mobileMenuBtn">    <span></span><span></span><span></span></div>
      </nav>

      <section class="hero" id="home">
        <div class="hero-bg"></div>
        <div class="hero-content">
          <div class="hero-badge" id="siteBeyoBadge">Class of 2025</div>
          <h1 class="hero-title" id="siteHeroTitle">Farewell <span class="highlight">2025</span></h1>
          <p class="hero-subtitle" id="siteHeroSubtitle">Where Memories Become Eternal</p>
          <p class="hero-description" id="siteHeroDescription">
            As we close this chapter and begin new journeys, let's celebrate the friendships, lessons, and memories that shaped us.
          </p>
          <div class="hero-buttons">
            <a href="#upload" class="btn btn-primary">Share Your Memory</a>
            <a href="#memories" class="btn btn-secondary">View Memory Wall</a>
          </div>
          </div>

      </section>

      <section id="countdown">
        <div class="container">
          <div class="section-header">
            <span class="section-badge" id="siteCountdownBadge">Mark Your Calendar</span>
            <h2 class="section-title" id="siteCountdownTitle">Countdown to <span class="highlight">Farewell</span></h2>
            <p class="section-description" id="siteCountdownDesc">The moment we've all been waiting for is almost here</p>
            </div>

            <div class="countdown-container">
              <div class="countdown-item">    <div class="countdown-number" id="days">00</div>    <div class="countdown-label">Days</div></div>
              <div class="countdown-item">    <div class="countdown-number" id="hours">00</div>    <div class="countdown-label">Hours</div></div>
              <div class="countdown-item">    <div class="countdown-number" id="minutes">00</div>    <div class="countdown-label">Minutes</div></div>
              <div class="countdown-item">    <div class="countdown-number" id="seconds">00</div>    <div class="countdown-label">Seconds</div></div>
              </div>

            <div class="event-date-display">
              <p id="siteEventDateLine">    <span>March 15, 2026</span>  at  <span>6:00 PM</span>  IST</p>
            </div>
            </div>
      </section>

      <section id="upload">
        <div class="container">
          <div class="section-header">
            <span class="section-badge" id="siteUploadBadge">Share Your Story</span>
            <h2 class="section-title" id="siteUploadTitle">Upload Your <span class="highlight">Memories</span></h2>
            <p class="section-description" id="siteUploadDesc">Share the moments that made your journey special</p>
            </div>
```

```
<div class="upload-container">
  <div class="upload-card">
    <div class="callout" id="uploadSlotsCallout" style="display:none;"></div>

    <div id="uploadForm">
      <div class="dropzone" id="dropzone">
        <div class="dropzone-icon">📤</div>
        <h3 class="dropzone-title">Drag & Drop Your Files</h3>
        <p class="dropzone-subtitle">or click to browse</p>
        <p class="dropzone-formats">Supported: <span>JPG</span> <span>PNG</span> <span>GIF</span> <span>MP4</span> <span>MOV</span></p>
        <p>Single: 5MB/file • Total: 100MB per upload</p>
        <input type="file" id="file-input" multiple accept="image/*,video/*" />
      </div>

      <div class="file-preview-container" id="filePreviewContainer">
        <h4 style="margin-bottom: 15px;">Selected Files:</h4>
        <div class="file-preview-grid" id="filePreviewGrid"></div>
        <p class="file-info" id="filetotals"></p>
      </div>

      <div class="upload-form" id="uploadFormFields" style="display: none;">
        <div class="form-group">
          <label>Your Name <span class="required">*</span></label>
          <input type="text" class="form-input" id="studentName" placeholder="Enter your full name" maxlength="100" />
        </div>

        <div class="form-group">
          <label>Caption / Memory Description <span class="required">*</span></label>
          <textarea class="form-textarea" id="caption" placeholder="Share the story behind this memory..." maxlength="500"></textarea>
          <p class="char-counter">0/<span id="charLimit">500</span> characters</p>
        </div>

        <div class="form-group">
          <label>Memory Category <span class="required">*</span></label>
          <select class="form-select" id="memoryType">
            <option value="">Select a category</option>
            <option value="Friends">Friends</option>
            <option value="Teachers">Teachers</option>
            <option value="Events">Events</option>
            <option value="Fun Moments">Fun Moments</option>
            <option value="Sports">Sports</option>
            <option value="Academics">Academics</option>
            <option value="Other">Other</option>
          </select>
        </div>

        <button type="button" class="btn btn-primary" id="submitUpload" style="width: 100%;">Upload Memory</button>
      </div>

      <div class="upload-progress" id="uploadProgress">
        <div class="progress-bar"><div class="progress-fill" id="progressFill"></div></div>
        <p class="progress-text" id="progressText">Uploading... 0%</p>
      </div>

    </div>

    <div class="upload-success" id="uploadSuccess">
      <div class="success-icon">✓</div>
      <h3 class="success-title">Memory Uploaded!</h3>
      <p class="success-message">Your memory has been submitted.
        Your memory has been submitted and is pending approval. It will appear once approved by the admin.
      </p>
      <button class="btn btn-primary" onclick="resetUploadForm()">Upload Another</button>
    </div>
  </div>
</div>
</main>
</section>
<style>
/* Memories */
#memories { background: linear-gradient(180deg, var(--navy-medium) 0%, var(--navy-dark) 100%); }
.memory-filters { display: flex; justify-content: center; gap: 15px; flex-wrap: wrap; margin-bottom: 25px; }
.filter-btn {
  padding: 10px 25px;
  background: var(--glass-bg);
  backdrop-filter: blur(10px);
  border-radius: 25px;
  color: var(--text-light);
  cursor: pointer;
  transition: var(--transition-smooth);
  font-family: var(--font-body);
  font-size: 0.9rem;
}
.filter-btn:hover, .filter-btn.active { background: var(--primary-gold); color: var(--navy-dark); border-color: var(--primary-gold); }

.memories-toolbar {
  display:flex;
  justify-content:space-between;
  align-items:center;
  gap:15px;
  flex-wrap:wrap;
  margin: 0 0 25px;
}
.memories-toolbar .left, .memories-toolbar .right{
  display:flex; gap:10px; align-items:center; flex-wrap:wrap;
}
.mini-pill {
  padding: 8px 12px;
  border-radius: 999px;
  border: 1px solid var(--glass-border);
  background: rgba(255,255,255,0.05);
  color: var(--text-muted);
  font-size: 0.85rem;
}

.memory-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(300px, 1fr)); gap: 25px; }
.memory-card {
  background: var(--glass-bg);
  border: 1px solid var(--glass-border);
  border-radius: 20px;
  overflow: hidden;
  transition: var(--transition-smooth);
  animation: fadeInUp 0.6s ease;
  position: relative;
}
.memory-card:hover { transform: translateY(-8px); border-color: var(--primary-gold); box-shadow: var(--shadow-gold); }
.memory-media { position: relative; aspect-ratio: 4/3; overflow: hidden; cursor: pointer; }
.memory-media img, .memory-media video { width: 100%; height: 100%; object-fit: cover; transition: var(--transition-smooth); }
.memory-card:hover .memory-media img, .memory-card:hover .memory-media video { transform: scale(1.05); }

.memory-media .play-button {
  position: absolute; top: 50%; left: 50%;
  transform: translate(-50%, -50%);
  width: 60px; height: 60px;
  background: rgba(0,0,0,0.7);
  border-radius: 50%;
  display: flex; align-items: center; justify-content: center;
  font-size: 1.5rem; color: white;
  transition: var(--transition-smooth);
}
.memory-media:hover .play-button { background: var(--primary-gold); transform: translate(-50%, -50%) scale(1.1); }

.memory-type-badge {
  position: absolute; top: 15px; left: 15px;
  background: rgba(0,0,0,0.7);
  padding: 5px 12px;
  border-radius: 15px;
  font-size: 0.75rem;
  color: var(--primary-gold);
  text-transform: uppercase;
  letter-spacing: 1px;
  z-index: 2;
}
.memory-featured-badge{
  position:absolute;
  top: 15px; right: 15px;
  background: rgba(212,175,55,0.15);
  border: 1px solid rgba(212,175,55,0.5);
  color: var(--primary-gold);
  padding: 5px 10px;
  border-radius: 14px;
  font-size: 0.75rem;
  z-index: 2;
  backdrop-filter: blur(8px);
}

.memory-content { padding: 20px; }
.memory-uploer { display: flex; align-items: center; gap: 10px; margin-bottom: 10px; }
.memory-avatar {
  width: 40px; height: 40px;
  background: var(--gradient-gold);
  border-radius: 50%;
  display: flex; align-items: center; justify-content: center;
  font-weight: 700;
  color: var(--navy-dark);
}
.memory-uploer-info { flex: 1; }
.memory-uploer-name { font-weight: 600; font-size: 1rem; }
.memory-time { font-size: 0.8rem; color: var(--text-muted); }

.memory-caption { color: var(--text-muted); font-size: 0.95rem; line-height: 1.5; margin-bottom: 15px; }

.memory-actions { display: flex; align-items: center; justify-content: space-between; gap: 10px; flex-wrap: wrap; }
.action-size { display:flex; align-items:center; gap: 10px; flex-wrap: wrap; }

.like-btn {
  display: flex; align-items: center; gap: 8px;
  background: none; border: none;
  color: var(--text-muted);
  cursor: pointer;
  font-family: inherit;
  padding: 8px 12px;
  border-radius: 20px;
  transition: var(--transition-smooth);
  font-family: var(--font-body);
  border: 1px solid transparent;
}
.like-btn:hover { background: rgba(233, 69, 96, 0.1); color: var(--accent-rose); border-color: rgba(233,69,96,0.2); }
.like-btn.liked { color: var(--accent-rose); }
.like-btn.liked .heart-icon { animation: heartBeat 0.8s ease; }

.react-group{
  display:flex; align-items:center; gap: 8px; flex-wrap:wrap;
}
.react-btn{
  padding: 8px 12px;
  background: rgba(255,255,255,0.04);
  border: 1px solid var(--glass-border);
  background: rgba(255,255,255,0.04);
  color: var(--text-light);
  cursor:pointer;
  transition: var(--transition-smooth);
  font-size: 0.9rem;
}
.react-btn.hover { border-color: var(--primary-gold); transform: translateY(-1px); }
.react-count {
  color: var(--text-muted);
  font-size: 0.9rem;
  margin-left: 6px;
}

.comments-open-btn{
  padding: 8px 12px;
  border-radius: 999px;
  border: 1px solid var(--glass-border);
  background: rgba(255,255,255,0.04);
  color: var(--text-light);
  cursor:pointer;
  transition: var(--transition-smooth);
  font-size: 0.9rem;
}
.comments-open-btn:hover { border-color: var(--primary-gold); transform: translateY(-1px); }

.memory-empty { text-align: center; padding: 80px 20px; color: var(--text-muted); }
.memory-empty-icon { font-size: 4rem; margin-bottom: 20px; opacity: 0.5; }
.memory-loading { text-align: center; padding: 60px; color: var(--text-muted); }
.loading-spinner { width: 50px; height: 50px; border: 4px solid var(--glass-border); border-top-color: var(--primary-gold); border-radius: 50%; animation: spin 1s linear infinite; margin: 0 auto 20px; }

.load-more-wrap{ display:flex; justify-content:center; margin-top: 35px; }
.load-more-btn{ min-width: 220px; justify-content:center; }

/* Lightbox */
.lightbox {
  position: fixed; top:0; left:0; right:0; bottom:0;
  background: rgba(0,0,0,0.95);
  z-index: 2000;
  display: none;
  align-items: center;
  justify-content: center;
  padding: 20px;

.lightbox.active { display: flex; animation: fadeIn 0.3s ease; }

.lightbox-grid{
  width: min(1200px, 96vw);
  display:grid;
  grid-template-columns: 1.2fr 0.8fr;
  gap: 30px;
  align-items: start;
}
.lightbox-main{
  background: rgba(255,255,255,0.06);
  border: 1px solid var(--glass-border);
  border-radius: 18px;
  padding: 16px;
  overflow:hidden;
}
.lightbox-content { width: 100%; position: relative; }
.lightbox-content img, .lightbox-content video { width: 100%; max-height: 75vh; object-fit: contain; border-radius: 12px; background: rgba(0,0,0,0.2); }

.lightbox-side{
  background: rgba(255,255,255,0.06);
  border: 1px solid var(--glass-border);
  border-radius: 18px;
  padding: 16px;
  max-height: 82vh;
  overflow:auto;
}

.lightbox-close { position: fixed; top: 10px; right: 10px; background: rgba(255,255,255,0.08); border: 1px solid var(--glass-border); color: white; font-size: 1.4rem; cursor: pointer; transition: var(--transition-smooth); width: 44px; height: 44px; border-radius: 50%; }
.lightbox-close:hover { color: var(--primary-gold); border-color: var(--primary-gold); transform: scale(1.05); }

.lightbox-size {
  position: fixed; top: 50%;
  transform: translateY(-50%);
  background: rgba(255,255,255,0.1);
  border: 1px solid rgba(255,255,255,0.2);
  color: white;
  cursor: pointer;
  width: 50px; height: 50px;
  border-radius: 50%;
  cursor: pointer;
  transition: var(--transition-smooth);
  display:flex; align-items:center; justify-content:center;

.lightbox-size:hover { background: var(--primary-gold); color: var(--navy-dark); }
.lightbox-prev { left: 10px; }
.lightbox-next { right: 10px; }

.lightbox-info { color: white; }
.lightbox-uploer { font-weight: 700; font-size: 1.1rem; }
.lightbox-caption { color: var(--text-muted); margin-top: 8px; white-space: pre-wrap; }
.lightbox-meta{ margin-top: 10px; font-size: 0.9rem; color: var(--text-muted); display:flex; gap: 10px; flex-wrap: wrap; }
.lightbox-download {
  display:inline-flex;
  align-items: center;
  gap: 8px;
  margin-top: 14px;
  padding: 10px 16px;
  background: var(--gradient-gold);
  color: var(--navy-dark);
  border-radius: 12px;
  text-decoration: none;
  font-weight: 700;
  transition: var(--transition-smooth);
}
.lightbox-download:hover { transform: scale(1.03); }

/* Comments */
.comments-header {
  display:flex;
  justify-content:space-between;
  align-items:center;
  gap: 10px;
  margin-bottom: 12px;
}
.comments-title{
```

```
    font-family: var(--font-display);
    font-size: 1.2rem;
    color: var(--primary-gold);
  }
  .comment-disabled{
    padding: 10px 12px;
    border-radius: 12px;
    border: 1px solid rgba(255,255,255,0.15);
    background: rgba(255,255,255,0.05);
    color: var(--text-muted);
    font-size: 0.95rem;
    margin-bottom: 1.2px;
  }
  .comment-form{
    display:grid;
    gap: 10px;
    margin-bottom: 14px;
  }
  .comment-form-row{
    display:grid;
    grid-template-columns: 1fr;
    gap: 10px;
  }
  .comment-form-actions{
    display:flex; gap: 10px; flex-wrap:wrap; justify-content:flex-end;
  }
  .comment-list{ display:grid; gap: 12px; }
  .comment-item{
    border: 1px solid rgba(255,255,255,0.12);
    background: rgba(0,0,0,0.15);
    border-radius: 14px;
    padding: 12px;
  }
  .comment-top{
    display:flex; justify-content:space-between; align-items:center; gap: 10px;
    margin-bottom: 6px;
  }
  .comment-name{ font-weight: 700; }
  .comment-time{ font-size: 0.8rem; color: var(--text-muted); }
  .comment-text{ color: rgba(255,255,255,0.9); white-space: pre-wrap; }
  .comment-reply-btn{
    margin-top: 8px;
    cursor: pointer;
    color: var(--primary-gold);
    background:none;
    border:none;
    cursor:pointer;
    padding: 0;
  }
  .comment-replies{
    margin-top: 10px;
    padding-left: 14px;
    border-left: 2px solid rgba(212,175,55,0.25);
    display:grid;
    gap: 10px;
  }

  @media (max-width: 380px){
    .lightbox-grid{ grid-template-columns: 1fr; }
    .lightbox-side{ max-height: auto; }
  }
  @media (max-width: 768px){
    .memory-grid { grid-template-columns: 1fr; }
    .lightbox-nav { display:none; }
  }
</style>

<section id="memories">
  <div class="container">
    <div class="section-header">
      <span class="section-badge" id="uiMemoriesBadge">Memory Wall</span>
      <h2 class="section-title" id="uiMemoriesTitle">Our <span class="highlight">Memories</span></h2>
      <p class="section-description" id="uiMemoriesDesc">A collection of moments from our incredible journey together</p>
    </div>

    <div class="memories-toolbar">
      <div class="left">
        <div class="mini-pill" id="memCountPill">Loading...</div>
        <div class="mini-pill" id="liveFill" style="display:none;">Live updates enabled</div>
      </div>
      <div class="right">
        <!-- Infinite Scroll Toggle Removed -->
      </div>
    </div>

    <div class="memory-filters" id="memoryFilters">
      <button class="filter-btn active" data-filter="all">All</button>
      <button class="filter-btn" data-filter="Friends">Friends</button>
      <button class="filter-btn" data-filter="Teachers">Teachers</button>
      <button class="filter-btn" data-filter="Events">Events</button>
      <button class="filter-btn" data-filter="Campus">Campus</button>
      <button class="filter-btn" data-filter="Sports">Sports</button>
      <button class="filter-btn" data-filter="Academics">Academics</button>
      <button class="filter-btn" data-filter="Other">Other</button>
    </div>

    <div class="memory-grid" id="memoryGrid">
      <div class="memory-loading" style="grid-column: 1/-1">
        <div class="loading-spinner"></div>
        <p>Loading memories...</p>
      </div>
    </div>

    <div class="load-more-wrap" id="loadMoreWrap" style="display:none;">
      <button class="btn btn-secondary load-more-btn" id="loadMoreBtn" type="button">Load More</button>
    </div>
  </div>
</section>

<!-- Teachers / Timeline / Quote / Footer kept from previous version (rendered from settings) -->
<style>
  #teachers { background: var(--navy-dark); }
  .teachers-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(280px, 1fr)); gap: 30px; }
  .teacher-card { background: var(--glass-bg); border: 1px solid var(--glass-border); border-radius: 25px; padding: 30px; text-align: center; transition: var(--transition-smooth); }
  .teacher-card:hover { transform: translateY(-10px); border-color: var(--primary-gold); box-shadow: var(--shadow-gold); }
  .teacher-image {
    width: 120px; height: 120px;
    border-radius: 50%;
    margin: 0 auto 20px;
    background: var(--gradient-gold);
    display: flex; align-items: center; justify-content: center;
    font-size: 3rem;
    overflow: hidden;
    color: var(--navy-dark);
    font-weight: 800;
  }
  .teacher-image img { width: 100%; height: 100%; object-fit: cover; }
  .teacher-name { font-family: var(--font-display); font-size: 1.4rem; font-weight: 600; margin-bottom: 5px; }
  .teacher-subject { color: var(--primary-gold); font-size: 0.95rem; margin-bottom: 15px; }
  .teacher-quote { font-style: italic; color: var(--text-muted); font-size: 0.9rem; line-height: 1.6; }

  #timeline { background: var(--navy-medium); }
  .timeline { position: relative; max-width: 800px; margin: 0 auto; }
  .timeline::before { content: ''; position: absolute; left: 50%; transform: translateX(-50%); width: 4px; height: 100%; background: var(--gradient-gold); border-radius: 2px; }
  .timeline-item { display: flex; justify-content: flex-end; padding-right: 50%; position: relative; margin-bottom: 50px; }
  .timeline-item:nth-child(even) { justify-content: flex-start; padding-right: 0; padding-left: 50%; }
  .timeline-content { background: var(--glass-bg); border: 1px solid var(--glass-border); border-radius: 20px; padding: 25px; max-width: 350px; position: relative; margin-right: 30px; }
  .timeline-item:nth-child(even) .timeline-content { margin-right: 0; margin-left: 30px; }
  .timeline-content::before { content: ''; position: absolute; top: 20px; right: -15px; width: 30px; height: 20px; background: var(--primary-gold); border-radius: 50%; border: 4px solid var(--navy-medium); }
  .timeline-item:nth-child(even) .timeline-content::before { right: auto; left: -15px; }
  .timeline-year { font-family: var(--font-display); font-size: 1.5rem; font-weight: 700; color: var(--primary-gold); margin-bottom: 10px; }
  .timeline-title { font-size: 1.1rem; font-weight: 600; margin-bottom: 10px; }
  .timeline-description { color: var(--text-muted); font-size: 0.9rem; }

  #quote { background: var(--navy-dark); text-align: center; padding: 150px 20px; }
  .quote-content { max-width: 900px; margin: 0 auto; }
  .quote-icon { font-size: 4rem; color: var(--primary-gold); margin-bottom: 30px; opacity: 0.5; }
  .quote-text { font-size: clamp(1.2rem, 2vw, 1.5rem); line-height: 1.5; font-style: italic; margin-bottom: 30px; }
  .quote-author { color: var(--primary-gold); font-size: 1.2rem; }

  footer { background: var(--navy-dark); border-top: 1px solid var(--glass-border); padding: 60px 20px 30px; }
  .footer-content {
    max-width: var(--container-width);
    margin: 0 auto;
    display:grid;
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 40px;
    margin-bottom: 40px;
  }
  .footer-section h4 { font-family: var(--font-display); font-size: 1.3rem; margin-bottom: 20px; color: var(--primary-gold); }
  .footer-section p { color: var(--text-muted); line-height: 1.8; }
  .footer-links { list-style: none; }
  .footer-links li { margin-bottom: 10px; }
  .footer-links a { color: var(--text-muted); text-decoration: none; transition: var(--transition-smooth); }
  .footer-links a:hover { color: var(--primary-gold); }
  .footer-bottom { text-align: center; padding-top: 30px; border-top: 1px solid var(--glass-border); color: var(--text-muted); font-size: 0.9rem; }

  @media (max-width: 1024px) {
    .timeline::before { left: 20px; }
    .timeline-item { padding-right: 0; padding-left: 50px; justify-content: flex-start; }
    .timeline-item:nth-child(even) { padding-left: 50px; }
    .timeline-content { margin-right: 0; margin-left: 0; }
    .timeline-item:nth-child(even) .timeline-content { margin-left: 0; }
    .timeline-dot { left: 12px; right: auto;}
    .timeline-item:nth-child(even) .timeline-dot { left: 12px; }
  }
  @media (max-width: 480px) {
    .hero-buttons { flex-direction: column; width: 100%; padding: 0 20px; }
    .btn { width: 100%; justify-content: center; }
    .teachers-grid { grid-template-columns: 1fr; }
    .footer-content { grid-template-columns: 1fr; text-align: center; }
  }
</style>

<section id="compilations" style="background: var(--navy-medium); padding: 80px 0;">
  <div class="container">
    <!-- The Beautiful Header -->
    <div class="section-header" style="text-align: center; margin-bottom: 40px;">
      <span class="section-badge" style="color: var(--primary-gold); font-weight: bold; text-transform: uppercase; letter-spacing: 1px;">Curated Memories</span>
      <h2 class="section-title" style="font-size: 2.5rem; margin-top: 10px;">Memory <span class="highlight" style="color: var(--primary-gold);">Compilations</span></h2>
      <p class="section-description" style="color: #aaa; margin-top: 10px;">Watch special slideshows created from the best moments.</p>
    </div>

    <!-- The Grid where Javascript will automatically load the items -->
    <div class="compilations-grid" id="publicCompilationsGrid" style="display: grid; grid-template-columns: repeat(auto-fill, minmax(300px, 1fr)); gap: 20px; margin-top: 30px;">
      <!-- Compilations will load here automatically -->
    </div>
  </div>
</section>

<section id="teachers">
  <div class="container">
    <div class="section-header">
      <span class="section-badge" id="uiTeachersBadge">Our Guides</span>
      <h2 class="section-title" id="uiTeachersTitle">Beloved <span class="highlight">Teachers</span></h2>
      <p class="section-description" id="uiTeachersDesc">The mentors who shaped our minds and hearts</p>
    </div>
    <div class="teachers-grid" id="teachersGrid"></div>
  </div>
</section>

<section id="timeline">
  <div class="container">
    <div class="section-header">
      <span class="section-badge" id="uiTimelineBadge">Our Journey</span>
      <h2 class="section-title" id="uiTimelineTitle">Through The <span class="highlight">Years</span></h2>
      <p class="section-description" id="uiTimelineDesc">Key milestones of our school journey</p>
    </div>
    <div class="timeline" id="timelineList"></div>
  </div>
</section>

<section id="quote">
  <div class="quote-content">
    <div class="quote-icon" id="uiQuoteIcon">❝</div>
    <p class="quote-text" id="uiQuoteText">"Don't cry because it's over, smile because it happened. The memories we made will last a lifetime."</p>
    <p class="quote-author" id="uiQuoteAuthor">— Class of 2025</p>
  </div>
</section>


<footer>
  <div class="footer-content">
    <div class="footer-section">
      <h4 id="uiFooterSchool">Cornerstone</h4>
      <p id="uiFooterDesc">Cornerstone International School - Where futures are built and memories are made.</p>
    </div>
    <div class="footer-section">
      <h4 id="uiFooterQuickLinksTitle">Quick Links</h4>
      <ul class="footer-links">
        <li><a href="#home">Home</a></li>
        <li><a href="#memories">Memory Wall</a></li>
        <li><a href="#compilations">Memory Compilations</a></li>
        <li><a href="#upload">Upload Memory</a></li>
        <li><a href="#teachers">Teachers</a></li>
      </ul>
    </div>
    <div class="footer-section">
      <h4 id="uiFooterContactTitle">Contact</h4>
      <ul class="footer-links">
        <li id="uiFooterLocation">Cornerstone International School</li>
        <li><a id="uiFooterEmailLink" href="mailto:farewell2025@cornerstone.edu">farewell2025@cornerstone.edu</a></li>
        <li id="uiFooterContactNote">Contact School Office</li>
      </ul>
    </div>
  </div>
  <div class="footer-bottom">
    <p id="uiFooterMadeBy">Made with ♥ by the Class of 2025</p>
    <p style="margin-top: 10px; opacity: 0.7;" id="uiFooterCopyright">© 2025 Cornerstone International School. All rights reserved.</p>
  </div>
</footer>

<!-- Lightbox -->
<div class="lightbox" id="lightbox">
  <button class="lightbox-close" onclick="closeLightbox()">×</button>
  <button class="lightbox-nav lightbox-prev" onclick="navigateLightbox(-1)">‹</button>
  <div class="lightbox-grid">
    <div class="lightbox-main">
      <div class="lightbox-content" id="lightboxContent"></div>
    </div>
    <div class="lightbox-side">
      <div class="lightbox-info" id="lightboxInfo"></div>

      <div style="margin-top: 14px;">
        <div class="comments-header">
          <div class="comments-title">Comments</div>
          <div class="mini-pill" id="commentsCountPill">0</div>
        </div>

        <div class="comments-disabled" id="commentsDisabledBanner" style="display:none;">
          Comments are currently disabled by the admin.
        </div>

        <div class="comment-form" id="commentForm">
          <div class="comment-form-row">
            <input type="text" id="commentName" placeholder="Your name *" maxlength="50" />
            <textarea class="form-textarea" id="commentText" placeholder="Write a comment..." maxlength="1000" style="min-height: 90px;"></textarea>
          </div>
          <div class="comment-form-actions">
            <button class="btn btn-secondary" type="button" id="cancelReplyBtn" style="display:none; padding: 12px 18px;">Cancel Reply</button>
            <button class="btn btn-primary" type="button" id="postCommentBtn" style="padding: 12px 18px;">Post</button>
          </div>
          <div class="mini-pill" id="replyingToPill" style="display:none;">replying...</div>
        </div>

        <div class="comment-list" id="commentList"></div>
      </div>
    </div>
  </div>
  <button class="lightbox-nav lightbox-next" onclick="navigateLightbox(1)">›</button>
</div>

<!-- Admin Login Overlay -->
<style>
  .admin-overlay {
    position: fixed; top:0; left:0; right:0; bottom:0;
    background: rgba(0,0,0,0.9);
    z-index: 3000;
    display: none;
    overflow:auto;
    justify-content: center;
    padding: 20px;
  }
  .admin-overlay.active { display: flex; animation: fadeIn 0.3s ease; }
  .admin-login-card {
    background: var(--navy-medium);
```

```
    border: 1px solid var(--glass-border);
    border-radius: 25px;
    padding: 42px;
    max-width: 440px;
    width: 100%;
    text-align: center;
  }
  .admin-login-card h2 { font-family: var(--font-display); font-size: 2rem; margin-bottom: 10px; color: var(--primary-gold); }
  .admin-login-card p { color: var(--text-muted); margin-bottom: 25px; }
  .admin-login-card .form-group { text-align: left; }
  .admin-login-error {
    background: rgba(244, 67, 54, 0.1);
    border: 1px solid var(--error-red);
    color: var(--error-red);
    padding: 12px;
    border-radius: 10px;
    margin-bottom: 16px;
    display: none;
  }
  .admin-login-error.active { display: block; }
  .admin-close { position: absolute; top: 20px; right: 28px; background: none; border: none; color: white; font-size: 2rem; cursor: pointer; }

  /* Admin dashboard base (full UI + theme editor + users panel in Part 3) */
  .admin-dashboard {
    position: fixed; top:0; left:0; right:0; bottom:0;
    background: var(--navy-dark);
    z-index: 9000;
    display: none;
    overflow-y: auto;
  }
  .admin-dashboard.active { display: block; }

  .admin-header {
    background: var(--navy-medium);
    padding: 20px 40px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    border-bottom: 1px solid var(--glass-border);
    position: sticky;
    top: 0;
    z-index: 100;
    gap: 20px;
    flex-wrap: wrap;
  }
  .admin-title { font-family: var(--font-display); font-size: 1.5rem; color: var(--primary-gold); }
  .admin-actions { display: flex; gap: 12px; flex-wrap: wrap; }
  .admin-btn {
    padding: 10px 16px;
    border-radius: 8px;
    font-size: 0.8rem;
    font-weight: 600;
    cursor: pointer;
    transition: var(--transition-smooth);
    font-family: var(--font-body);
    display: inline-flex;
    align-items: center;
    gap: 8px;
    border: none;
  }
  .admin-btn-primary { background: var(--gradient-gold); color: var(--navy-dark); }
  .admin-btn-secondary { background: var(--glass-bg); color: var(--text-light); border: 1px solid var(--glass-border); }
  .admin-btn-danger { background: rgba(244, 67, 54, 0.2); color: #ffb3ad; border: 1px solid rgba(244,67,54,0.5); }
  .admin-btn:hover { transform: translateY(-2px); }

  .admin-content { padding: 40px; }

  .admin-tabs { display: flex; gap: 10px; flex-wrap: wrap; margin-bottom: 20px; }
  .admin-tab {
    padding: 10px 16px;
    border-radius: 20px;
    cursor: pointer;
    border: 1px solid var(--glass-border);
    background: var(--glass-bg);
    color: var(--text-light);
    transition: var(--transition-smooth);
    user-select: none;
  }
  .admin-tab.active { background: var(--primary-gold); color: var(--navy-dark); border-color: var(--primary-gold); }

  .admin-panel { display: none; }
  .admin-panel.active { display: block; }

  .admin-stats { display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap: 20px; margin-bottom: 30px; }
  .stat-card { background: var(--glass-bg); border: 1px solid var(--glass-border); border-radius: 20px; padding: 25px; text-align: center; }
  .stat-number { font-family: var(--font-display); font-size: 2.5rem; font-weight: 800; color: var(--primary-gold); }
  .stat-label { color: var(--text-muted); font-size: 0.9rem; margin-top: 5px; }

  .admin-filters { display: flex; gap: 15px; margin-bottom: 20px; flex-wrap: wrap; align-items: center; }
  .admin-search { flex: 1; min-width: 250px; }
  .admin-search-input {
    width: 100%;
    padding: 12px 20px;
    background: var(--glass-bg);
    border: 1px solid var(--glass-border);
    border-radius: 25px;
    color: var(--text-light);
    font-size: 0.95rem;
  }
  .admin-search-input:focus { outline: none; border-color: var(--primary-gold); }
  .admin-filter-group { display: flex; gap: 10px; flex-wrap: wrap; }

  .admin-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(300px, 1fr)); gap: 18px; }
  .admin-memory-card { background: var(--glass-bg); border: 1px solid var(--glass-border); border-radius: 15px; overflow: hidden; transition: var(--transition-smooth); }
  .admin-memory-card:hover { border-color: var(--primary-gold); }
  .admin-memory-media { position: relative; aspect-ratio: 16/10; overflow: hidden; background: var(--navy-dark); }
  .admin-memory-media image, .admin-memory-media video { width: 100%; height: 100%; object-fit: cover; }
  .admin-memory-status { position: absolute; top: 10px; right: 10px; padding: 5px 12px; border-radius: 15px; font-size: 0.75rem; font-weight: 800; text-transform: uppercase; }
  .status-approved { background: rgba(76, 175, 80, 0.9); color: white; }
  .status-pending { background: rgba(255, 193, 7, 0.9); color: #333; }
  .status-trash { background: rgba(244,67,54,0.9); color: white; }

  .admin-memory-info { padding: 15px; }
  .admin-memory-status { font-weight: 800; margin-bottom: 5px; }
  .admin-memory-caption { font-size: 0.85rem; color: var(--text-muted); margin-bottom: 10px; display: -webkit-box; line-clamp: 2;
-webkit-box-orient: vertical;
overflow: hidden;
}
  .admin-memory-meta { display: flex; justify-content: space-between; font-size: 0.8rem; color: var(--text-muted); margin-bottom: 10px; gap: 10px; }
  .admin-memory-actions { display: grid; grid-template-columns: 1fr 1fr; gap: 10px; margin-top: 10px; }
  .admin-memory-actions button {
    padding: 8px 12px;
    border-radius: 12px;
    font-size: 0.85rem;
    cursor: pointer;
    transition: var(--transition-smooth);
    border: none;
    font-family: var(--font-body);
    font-weight: 700;
  }
  .approve-btn { background: rgba(76, 175, 80, 0.2); color: #8ff99a; border: 1px solid rgba(76,175,80,0.35); }
  .approve-btn:hover { background: var(--success-green); color: white; }
  .delete-btn { background: rgba(244, 67, 54, 0.2); color: #ffb3ad; border: 1px solid rgba(244,67,54,0.35); }
  .delete-btn:hover { background: var(--error-red); color: white; }
  .download-btn { background: rgba(33, 150, 243, 0.2); color: #9cd3ff; border: 1px solid rgba(33,150,243,0.35); }
  .download-btn:hover { background: #219bf3; color: white; }
  .edit-btn { background: rgba(212,175,55,0.18); color: var(--primary-gold-light); border: 1px solid rgba(212,175,55,0.35); }
  .edit-btn:hover { background: rgba(212,175,55,0.35); }
  .restore-btn { background: rgba(255,255,255,0.08); color: var(--text-light); border: 1px solid rgba(255,255,255,0.2); }
  .restore-btn:hover { border-color: var(--primary-gold); }

  /* notification */
  .notification {
    position: fixed;
    top: 100px;
    right: 20px;
    padding: 16px 22px;
    border-radius: 15px;
    z-index: 4000;
    display: none;
    align-items: center;
    gap: 15px;
    animation: slideInRight 0.5s ease;
    max-width: 420px;
  }
  .notification.active { display: flex; }
  .notification.success { background: rgba(76, 175, 80, 0.95); color: white; }
  .notification.error { background: rgba(244, 67, 54, 0.95); color: white; }
  .notification.info { background: rgba(33, 150, 243, 0.95); color: white; }
  .notification-icon { font-size: 1.5rem; }
  .notification-content { flex: 1; }
  .notification-title { font-weight: 800; margin-bottom: 3px; }
  .notification-message { font-size: 0.9rem; opacity: 0.95; }
  .notification-close { background: none; border: none; color: white; cursor: pointer; font-size: 1.2rem; opacity: 0.7; }
  .notification-close:hover { opacity: 1; }

  .confetti-container { position: fixed; top:0; left:0; width:100%; height:100%; pointer-events:none; z-index:5000; overflow:hidden; }
  .confetti { position: absolute; width: 10px; height: 10px; animation: confettiFall 3s linear forwards; }

  @media (max-width: 768px) {
    .admin-header { padding: 15px 20px; }
    .admin-content { padding: 20px; }
    .admin-filters { flex-direction: column; }
    .admin-search { width: 100%; }
  }
</style>

<div class="admin-overlay" id="adminOverlay">
  <button class="admin-close" onclick="closeAdminLogin()">×</button>
  <div class="admin-login-card">
    <h2>Admin Access</h2>
    <p>Login with your Admin ID + password</p>
    <div class="admin-login-error" id="adminLoginError">Invalid credentials. Please try again.</div>

    <div class="form-group">
      <label>Admin ID</label>
      <input type="text" class="form-input" id="adminUserId" placeholder="super" />
      <p style="color:var(--text-muted); font-size:0.85rem; margin-top:8px;">Tip: Super Admin ID is <strong>super</strong>.</p>
    </div>

    <div class="form-group">
      <label>Password</label>
      <input type="password" class="form-input" id="adminPassword" placeholder="Enter password" />
    </div>

    <button class="btn btn-primary" style="width: 100%;" onclick="adminLogin()">Login</button>
  </div>
</div>

<div class="admin-dashboard" id="adminDashboard">
  <div class="admin-header">
    <h1 class="admin-title">Admin Dashboard</h1>
    <div class="admin-actions">
      <button class="admin-btn admin-btn-primary" id="bmApproveAll" onclick="approveAllPending()">Approve All</button>
      <button class="admin-btn admin-btn-secondary" id="bmDownloadZip" onclick="downloadAllApproved()">Download ZIP</button>
      <button class="admin-btn" onclick="openBatchUpload()" style="background:var(--accent-purple); color:white; border:none; box-shadow:0 4px 15px rgba(123, 45, 142, 0.4);">Batch Upload</button>
      <button class="admin-btn admin-btn-secondary" id="bmExportCsv" onclick="exportCSV()">Export CSV</button>
      <button class="admin-btn admin-btn-danger" onclick="adminLogout()">Logout</button>
    </div>
  </div>

  <div class="admin-content">
    <div class="admin-tabs" id="adminTabs">
      <div class="admin-tab active" id="tabModeration" onclick="switchAdminTab('moderation')">Moderation</div>
      <div class="admin-tab" id="tabSettings" onclick="switchAdminTab('settings')">Site Settings</div>
      <div class="admin-tab" id="tabThemes" onclick="switchAdminTab('themes')">Themes</div>
      <div class="admin-tab" id="tabUsers" onclick="switchAdminTab('users')">Users</div>
      <div class="admin-tab" id="tabCompilations" onclick="switchAdminTab('compilations')">Compilations</div>
      <div class="admin-tab" id="tabSecurity" onclick="switchAdminTab('security')">Security</div>
    </div>

    <!-- Panels: content is completed in Part 3 -->
    <div class="admin-panel active" id="panelModeration">
      <div class="admin-stats" id="adminStats">
        <div class="stat-card">
          <div class="stat-number" id="statTotal">0</div>
          <div class="stat-label">Total Uploads</div>
        </div>
        <div class="stat-card">
          <div class="stat-number" id="statApproved">0</div>
          <div class="stat-label">Approved</div>
        </div>
        <div class="stat-card">
          <div class="stat-number" id="statPending">0</div>
          <div class="stat-label">Pending</div>
        </div>
        <div class="stat-card">
          <div class="stat-number" id="statStorage">0 MB</div>
          <div class="stat-label">Storage</div>
        </div>
      </div>

      <div class="admin-filters">
        <div class="admin-search">
          <input type="text" id="adminSearch" placeholder="Search by name/caption/type..." />
        </div>
        <div class="admin-filter-group">
          <button class="filter-btn active" data-admin-filter="all">All</button>
          <button class="filter-btn" data-admin-filter="approved">Approved</button>
          <button class="filter-btn" data-admin-filter="pending">Pending</button>
          <button class="filter-btn" data-admin-filter="featured">Featured</button>
          <button class="filter-btn" data-admin-filter="trash">Trash</button>
        </div>
        <select class="form-select" id="adminSort" style="width: auto; padding: 10px 40px 10px 15px;">
          <option value="newest">Newest First</option>
          <option value="oldest">Oldest First</option>
          <option value="likes">Most Liked</option>
        </select>
        <select class="form-select" id="adminPageSize" style="width:auto; padding: 10px 40px 10px 15px;">
          <option value="30">30/page</option>
          <option value="60">60/page</option>
        </select>
        </div>

        <div style="display:flex; gap: 10px; flex-wrap:wrap; margin-bottom: 14px;">
          <button class="admin-btn admin-btn-secondary" id="bulkSelectToggle" onclick="toggleBulkSelect()">Bulk Select: OFF</button>
          <button class="admin-btn admin-btn-secondary" onclick="bulkActionPrompt('approve')">Bulk Approve</button>
          <button class="admin-btn admin-btn-secondary" onclick="bulkActionPrompt('trash')">Bulk Trash</button>
          <button class="admin-btn admin-btn-secondary" onclick="bulkActionPrompt('restore')">Bulk Restore</button>
          <button class="admin-btn admin-btn-secondary" onclick="bulkActionPrompt('setType')">Bulk Change Category</button>
          <button class="admin-btn admin-btn-secondary" onclick="bulkActionPrompt('feature')">Bulk Feature</button>
          <button class="admin-btn admin-btn-secondary" onclick="bulkActionPrompt('unfeature')">Bulk Unfeature</button>
        </div>

        <div class="admin-grid" id="adminGrid">></div>

        <div class="load-more-wrap" style="margin-top: 25px;">
          <button class="btn btn-secondary load-more-btn" id="adminLoadMoreBtn" type="button" onclick="adminLoadMore()">Load More</button>
        </div>
        </div>

        <div class="admin-panel" id="panelSettings">></div>
        <div class="admin-panel" id="panelThemes">></div>
        <div class="admin-panel" id="panelUsers">></div>
        <div class="admin-panel" id="panelCompilations">></div>
        <div class="admin-panel" id="panelSecurity">></div>
      </div>
    </div>

    <div class="notification" id="notification">
      <div class="notification-icon" id="notificationIcon">></span>
      <div class="notification-content">
        <div class="notification-title" id="notificationTitle">Success</div>
        <div class="notification-message" id="notificationMessage">Operation completed successfully.</div>
      </div>
      <button class="notification-close" onclick="hideNotification()">×</button>
    </div>

    <div class="confetti-container" id="confettiContainer">></div>


  <!-- Compilation Creator Modal -->
  <div class="compilation-modal" id="compilationCreatorModal">
    <div class="compilation-modal-content">
      <div style="display:flex; justify-content:space-between; align-items:center; margin-bottom:20px;">
      <h2 style="color:var(--primary-gold); font-family:var(--font-display);">Compilation Creator</h2>
      <button class="admin-btn admin-btn-secondary" onclick="closeCompilationCreator()">Close</button>
      </div>
      <div style="display:grid; grid-template-columns: repeat(auto-fit, minmax(150px, 1fr)); gap: 15px; margin-bottom:20px;">
        <div class="form-group">
        <label>Name</label>
        <input type="text" class="form-input" id="compName" placeholder="Farewell Highlights" />
        </div>
        <div class="form-group">
        <label>Display Mode</label>
        <select class="form-select" id="compDisplayMode">
          <option value="auto">Auto (Slideshow)</option>
          <option value="manual">Manual (Arrows)</option>
        </select>
        </div>
        <div class="form-group">
        <label>Transition</label>
        <select class="form-select" id="compTransition">
          <option value="fade">Fade</option>
          <option value="slide">Slide</option>
          <option value="zoom">Zoom</option>
          <option value="flip">Flip</option>
          <option value="blur">Blur</option>
        </select>
```

```
            </div>
            <div class="form-group">
              <label>Default Duration (sec)</label>
              <input type="number" class="form-input" id="compDefaultDuration" value="5" min="1" max="60" />
            </div>
          </div>
          <div style="display:grid; grid-template-columns: 1fr; gap:20px;">
            <div>
              <h3 style="color:var(--primary-gold); margin-bottom: 10px;">Select Photos</h3>
              <div class="compilation-photo-grid" id="compPhotoGrid"></div>
            </div>
            <div>
              <h3 style="color:var(--primary-gold); margin-bottom: 10px;">Slides Sequence</h3>
              <div class="compilation-slide-preview" id="compSlidesPreview" style="flex-wrap:wrap; display:flex;"></div>
              <button class="admin-btn admin-btn-primary" style="width:100%; justify-content:center; margin-top:20px;" onclick="saveCompilation()">Save Compilation</button>
            </div>
          </div>
        </div>
      </div>

<!-- Compilation Player -->
<div class="compilation-player-overlay" id="compilationPlayer">
  <button class="compilation-close" onclick="closeCompilationPlayer()">✕</button>
  <button class="compilation-nav-btn compilation-prev" onclick="prevCompilationSlide()">❮</button>
  <button class="compilation-nav-btn compilation-next" onclick="nextCompilationSlide()">❯</button>
  <div id="compilationSlides" style="width: 100%; height: 100%; position: relative;"></div>
  <div class="compilation-progress" id="compilationProgress"></div>
</div>

<!-- Batch Upload Modal -->
<div class="admin-overlay" id="batchUploadModal">
  <button class="admin-close" onclick="closeBatchUpload()">✕</button>
  <div class="admin-login-card" style="max-width: 550px; padding: 30px; text-align: left;">
    <h2 style="text-center; text-center; text-display:); color: var(--primary-gold); margin-bottom: 20px; text-align: center;">Batch Upload</h2>
    <div class="form-group">
      <label>Batch Name (Uploaded)</label>
      <input type="text" class="form-input" id="batchName" placeholder="Admin Team" />
    </div>
    <div class="form-group">
      <label>Default Caption</label>
      <input type="text" class="form-input" id="batchCaption" placeholder="Memory from event..." />
    </div>
    <div class="form-group">
      <label>Category</label>
      <select class="form-select" id="batchType">
        <option value="Friends">Friends</option>
        <option value="Teachers">Teachers</option>
        <option value="Events">Events</option>
        <option value="Fun Moments">Fun Moments</option>
        <option value="Sports">Sports</option>
        <option value="Academics">Academics</option>
        <option value="Other">Other</option>
      </select>
    </div>
    <div class="form-group" style="display: flex; align-items: center; gap: 10px;">
      <input type="checkbox" id="batchAutoApprove" checked style="transform: scale(1.2);" />
      <label style="margin-bottom: 0;">Auto-Approve immediately</label>
    </div>
    <div class="form-group" style="margin-top: 15px;">
      <label class="btn-secondary" style="width: 100%; justify-content: center; cursor: pointer; border-style: dashed;">
        <span id="batchFileLabel">Click to select files</span>
        <input type="file" id="batchFile" multiple accept="image/*,video/*" style="display: none;" onchange="updateBatchFileCount(this)" />
      </label>
    </div>
    <div class="upload-progress" id="batchProgress" style="display:none; margin-top: 15px;">
      <div class="progress-bar"><div class="progress-fill" id="batchProgressFill"></div></div>
      <p class="progress-text" id="batchProgressText" style="text-align: center;">Uploading batch...</p>
    </div>
    <div style="display: flex; gap: 10px; margin-top: 25px;">
      <button class="btn btn-primary" style="flex: 1; justify-content: center;" id="btnBatchBatch" onclick="submitBatchUpload()">Start Upload</button>
    </div>
  </div>
</div>



<script>
  // ==========================
  // UNIFIED CONFIG - SINGLE POINT OF API CONFIGURATION
  // Change this ONE line to point to your server
  // ==========================
  const CONFIG = {
    API_BASE: "https://unsincerary-appointed-directors-dewd.trycloudflare.com",
    REFRESH_INTERVAL: 60000,
    MAX_FILE_SIZE: 100 * 1024 * 1024,
    MAX_TOTAL_SIZE: 200 * 1024 * 1024,
    MAX_FILES: 20,
    ALLOWED_TYPES: ['image/jpeg','image/png','image/gif','image/webp','video/mp4','video/quicktime','video/webm'],
    WS_AUTO: true
  };

  function getUrl(path) {
    const base = CONFIG.API_BASE.replace(/\/$/, '');
    const cleanPath = path.startsWith('/') ? path : '/' + path;
    return base + cleanPath;
  }

  function mediaUrl(url) {
    if (!url) return null;
    if (url.startsWith('http://') || url.startsWith('https://')) return url;
    if (url.startsWith('/')) return `${CONFIG.API_BASE}${url}`;
    return `${CONFIG.API_BASE}/${url}`;
  }

  const DEFAULT_SETTINGS = {
    schoolName: "Cornerstone International School",
    schoolTagline: "Farewell 2025",
    logoUrl: '',
    contactEmail: "farewell2025@cornerstone.edu",
    locationLine: "Cornerstone International School",
    farewellISO: "2026-03-15T18:00",
    heroBadge: "Class of 2025",
    heroTitle: "Farewell {year}",
    heroSubtitle: "Where Memories Become Eternal",
    heroDescription: "As we close this chapter and begin new journeys, let's celebrate the friendships, lessons, and memories that shaped us.",
    quotesIntro: "Here's what our classmates miss, smile because it happened:",
    quotesAuthor: "— Class of 2025",
    footerDesc: "Where futures are built and memories are made.",
    footerContactNote: "Contact School Office",
    footerMadeBy: "Made with ♥ by the Class of 2025",
    footerCopyright: "© 2026 Cornerstone International School. All rights reserved.",
    uploadsEnabled: true,
    requireApproval: true,
    profanityFilterEnabled: false,
    uploadWindowEnabled: false,
    uploadWindowStartISO: "",
    uploadWindowEndISO: "",
    autoApproveEnabled: false,
    autoApproveStartISO: "",
    autoApproveEndISO: "",
    teachers: [],
    events: [],
    theme: {
      primaryGold: "#D4AF37",
      primaryGoldLight: "#F4EC82",
      primaryGoldDark: "#B88608",
      navyDark: "#1a1a2e",
      navyMedium: "#16213e",
      navyLight: "#0f3460",
      accentBlue: "#64b5f6",
      accentPurple: "#7E3D8E",
      textLight: "#FFF8F",
      textMuted: "rgba(255, 255, 255, 0.75)",
      glassBg: "rgba(255, 255, 255, 0.1)",
      glassBorder: "rgba(255, 255, 255, 0.2)",
      successGreen: "#4CAF50",
      errorRed: "#f44336"
    }
  };

  function safeParse(s) { try { if (s === 'undefined') return null; return JSON.parse(s); } catch { return null; } }
  function formatFileSize(bytes) { if (bytes === 0) return '0 Bytes'; const k = 1024; const sizes = ['Bytes','KB','MB','GB']; const i = Math.floor(Math.log(bytes) / Math.log(k)); return parseFloat(bytes / Math.pow(k, i).toFixed(2)) + ' ' + sizes[i]; }
  function timeAgo(dateStr) { const date = new Date(parseInt(dateStr)); const now = new Date(); const seconds = Math.floor((now - date) / 1000); const intervals = [{ label: 'year', seconds: 31536000 }, { label: 'month', seconds: 2592000 }, { label: 'week', seconds: 604800 }, { label: 'day', seconds: 86400 }, { label: 'hour', seconds: 3600 }, { label: 'minute', seconds: 60 }]; for (const interval of intervals) { const count = Math.floor(seconds / interval.seconds); if (count >= 1) return `${count} ${interval.label}${count > 1 ? 's' : ''} ago`; } return 'Just now'; }
  function escapeHtml(text) { const div = document.createElement('div'); divtextContent = text ?? ''; return div.innerHTML; }
  function escapeAttr(text) { return (escapeHtml(text) || '').replaceAll('"', '&quot;').replaceAll('\'', '&#39;'); }
  function debounce(func, wait) { let timeout; return function executedFunction(...args) { const later = () => { clearTimeout(timeout); func(...args); }; clearTimeout(timeout); timeout = setTimeout(later, wait); }; }
  function getInitials(name) { const parts = String(name || '').trim().split(/\s+/).filter(Boolean); if (parts.length === 0) return '?'; if (parts.length === 1) return parts[0].slice(0, 2).toUpperCase(); return (parts[0][0] + parts[parts.length - 1][0]).toUpperCase(); }
  function setVal(id, val) { const el = document.getElementById(id); if (el && value != null) el.value = val ?? ''; }
  function getVal(id) { const el = document.getElementById(id); return el ? el.value.trim() : ''; }
  function capitalize(s) { return s.charAt(0).toUpperCase() + s.slice(1); }
  function mergeDeep(target, source) { if (typeof target !== 'object' || target === null) return source; if (typeof source !== 'object' || source === null) return target; for (const key of Object.keys(source)) { if (Array.isArray(source[key])) { target[key] = source[key]; } else if (typeof source[key] === 'object' && source[key] !== null) { target[key] = mergeDeep(target[key] ?? {}, source[key]); } else { target[key] = source[key]; } } return target; }

  let state = {
    teachers: [],
    events: [],
    memories: [],
    memoPage: 1,
    memoLimit: 2000,
    memoHasMore: true,
    memoLoading: false,
    memoCursor: null,
    currentFilter: 'all',
    initialsScroll: true,
    lightboxIdx: 0,
    replyingTo: null,
    adminToken: localStorage.getItem('adminToken') || null,
    adminUser: safeParse(localStorage.getItem('adminUser')),
    adminFilter: 'all',
    adminSort: 'newest',
    adminSearch: '',
    adminPage: 1,
    adminPageSize: 1,
    adminHideMotion: true,
    adminBulkMode: false,
    adminSelectedIds: new Set(),
    settings: structuredClone(DEFAULT_SETTINGS),
    ws: null,
    wsConnected: false
    };

    let activeMemoryId = null;

    function teacherRowHtml(t = {}, idx = 0) {
  return `<div class="admin-memory-card" style="padding:12px;">
    <div style="display:flex; justify-content:space-between; align-items:center; gap:10px;">
      <div style="fontweight:700;">Teacher #${idx + 1}</div>
      <button class="admin-btn admin-btn-danger" type="button" onclick="removeTeacherRow(${idx})">Remove</button>
    </div>
    <div style="margin-top:10px; display:grid; grid-template-columns: 1fr 1fr; gap:12px;">
      <div class="form-group">
        <label>Name</label>
        <input class="form-input" data-teacher-field="name" data-teacher-idx="${idx}" value="${escapeAttr(t.name || '')}" />
      </div>
      <div class="form-group">
        <label>Subject</label>
        <input class="form-input" data-teacher-field="subject" data-teacher-idx="${idx}" value="${escapeAttr(t.subject || '')}" />
      </div>
    </div>
    <div>
      <div class="form-group" style="margin-top:10px;">
        <label>Quote</label>
        <textarea class="form-textarea" data-teacher-field="quote" data-teacher-idx="${idx}" maxlength="250">${escapeHtml(t.quote || '')}</textarea>
      </div>
      <div class="form-group" style="margin-top:10px;">
        <label>Image URL (optional)</label>
        <input class="form-input" data-teacher-field="imageUrl" data-teacher-idx="${idx}" value="${escapeAttr(t.imageUrl || '')}" placeholder="https://..." />
      </div>
    </div>
  </div>`;
}

  function timelineRowHtml(e = {}, idx = 0) {
  return `<div class="admin-memory-card" style="padding:12px;">
    <div style="display:flex; justify-content:space-between; align-items:center; gap:10px;">
      <div style="fontweight:700;">Event #${idx + 1}</div>
      <button class="admin-btn admin-btn-danger" type="button" onclick="removeTimelineRow(${idx})">Remove</button>
    </div>
    <div style="margin-top:10px; display:grid; grid-template-columns: 1fr 1fr; gap:12px;">
      <div class="form-group">
        <label>Year</label>
        <input class="form-input" data-timeline-field="year" data-timeline-idx="${idx}" value="${escapeAttr(e.year || '')}" placeholder="2022" />
      </div>
      <div class="form-group">
        <label>Title</label>
        <input class="form-input" data-timeline-field="title" data-timeline-idx="${idx}" value="${escapeAttr(e.title || '')}" />
      </div>
    </div>
    <div>
      <div class="form-group" style="margin-top:10px;">
        <label>Description</label>
        <textarea class="form-textarea" data-timeline-field="description" data-timeline-idx="${idx}" maxlength="350">${escapeHtml(e.description || '')}</textarea>
      </div>
    </div>
  `;
}

  function renderTeachersEditor() { const wrap = document.getElementById('teachersEditor'); if (!wrap) return; const arr = Array.isArray(state.settings.teachers) ? state.settings.teachers : []; wrap.innerHTML = arr.map((t, i) => teacherRowHtml(t, i)).join('') || `<div class="muted-pill">No teachers yet.</div>`; }
  function renderTimelineEditor() { const wrap = document.getElementById('timelineEditor'); if (!wrap) return; const arr = Array.isArray(state.settings.timeline) ? state.settings.timeline : []; wrap.innerHTML = arr.map((e, i) => timelineRowHtml(e, i)).join('') || `<div class="muted-pill">No timeline items yet.</div>`; }
  function addTeacherRow() { if (!Array.isArray(state.settings.teachers)) state.settings.teachers = []; state.settings.teachers.push({ name: '', subject: '', quote: '', imageUrl: '' }); renderTeachersEditor(); }
  function removeTeacherRow(idx) { if (Array.isArray(state.settings.teachers)) return; state.settings.teachers.splice(idx, 1); renderTeachersEditor(); }
  function addTimelineRow() { if (!Array.isArray(state.settings.timeline)) state.settings.timeline = []; state.settings.timeline.push({ year: '', title: '', description: '' }); renderTimelineEditor(); }
  function removeTimelineRow(idx) { if (!Array.isArray(state.settings.timeline)) state.settings.timeline = []; state.settings.timeline.splice(idx, 1); renderTimelineEditor(); }
  function readTeachersEditor() { const nodes = Array.from(document.querySelectorAll('[data-teacher-idx]')); const arr = []; for (const el of nodes) { const idx = Number(el.dataset.teacherIdx); const field = el.dataset.teacherField; arr[idx] = arr[idx] || { name: '', subject: '', quote: '', imageUrl: '' }; arr[idx][field] = el.value; } return arr.filter(t => !!(t.name || t.subject || t.quote || t.imageUrl)); }
  function readTimelineEditor() { const nodes = Array.from(document.querySelectorAll('[data-timeline-idx]')); const arr = []; for (const el of nodes) { const idx = Number(el.dataset.timelineIdx); const field = el.dataset.timelineField; arr[idx] = arr[idx] || { year: '', title: '', description: '' }; arr[idx][field] = el.value; } return arr.filter(e => !!(e.year || e.title || e.description)); }

  document.addEventListener('DOMContentLoaded', async () => {
    initParticles();initNavbar();initUpload();initMemoryWall();initLightboxComments();initAntiGrigger();
    await loadSettings();
    applySettingsToUI();
    initCountdown();
    initCountdown();
    updateFarewell();
    renderTeachers();
    renderTimeline();
    await loadMemories(true);
    await autoFillMemoriesOnLoad();
    await loadPublicCompilations();
    setTimeout(() => { initItemsVideo(); }, 50);
    ADMIN_WS_AUTO && connectWS();
    setInterval(() => { if (document.getElementById('adminDashboardFly').classList.contains('active')) loadMemories(true); }, CONFIG.REFRESH_INTERVAL);
    });

    function initParticles() { const container = document.getElementById('particles-container'); if (!container) return; for (let i = 0; i < 50; i++) { const particle = document.createElement('div'); particle.className = 'particle'; particle.style.left = Math.random() * 100 + '%'; particle.style.top = Math.random() * 100 + '%'; particle.style.animationDuration = (8 + Math.random() * 4) + 's'; container.appendChild(particle); } }
    function initNavbar() { const navbar = document.getElementById('navbar'); const mobileMenuBtn = document.getElementById('mobileMenuBtn'); const navLinks = document.getElementById('navLinks'); if (navbar && navLinks) { window.addEventListener('scroll', () => { if (window.scrollY > 50) { if (!navbarScrolled) { navbarScrolled = true; navbar.classList.add('scrolled'); } } else { if (navbarScrolled) { navbarScrolled = false; navbar.classList.remove('scrolled'); } } }, { passive: true }); } if (mobileMenuBtn && navLinks) { mobileMenuBtn.addEventListener('click', () => { mobileMenuBtn.classList.toggle('active'); navLinks.classList.toggle('active'); }); navLinks.querySelectorAll('a').forEach(link => { link.addEventListener('click', () => { mobileMenuBtn.classList.remove('active'); navLinks.classList.remove('active'); }); }); } }

    async function loadSettings() { try { const res = await fetchWithTimeout(getUrl('/api/settings')); const data = await res.json(); if (data?.success) { state.settings = mergeDeep(structuredClone(DEFAULT_SETTINGS), data.settings || {}); } else { state.settings = structuredClone(DEFAULT_SETTINGS); } } catch { state.settings = structuredClone(DEFAULT_SETTINGS); } updateUploadStatusCallback(); }
    function applySettingsToUI() { const s = state.settings; const setText = (id, val) => { const el = document.getElementById(id); if (el) el.textContent = val ?? ''; }; setText('ssLogoLoaded', s.logoUrl ? '✓' : ''); setText('ssSchoolName', s.schoolName); setText('ssFooterSchool', s.schoolName); setText('ssFooterDesc', s.footerDesc); setText('ssFooterLocation', s.locationLine); const emailLink = document.getElementById('ssFooterEmailLink'); if (emailLink) { emailLink.textContent = s.contactEmail ? `✉ ${s.contactEmail}` : ''; emailLink.href = mailto:${s.contactEmail ?? ''}; } setText('ssFooterLocationLine', s.locationLine); const quotesLink = document.getElementById('ssFooterQuotesLink'); if (quotesLink) { quotesLink.textContent = s.quotesIntro ? `${s.quotesIntro} ${s.quotesAuthor}` : ''; } setText('ssFooterMadeBy', s.footerMadeBy); setText('ssFooterCopyright', s.footerCopyright); const eventDateLine = document.getElementById('ssEventDateLine'); if (eventDateLine) { eventDateLine.textContent = `${s.eventName ?? 'Farewell'} ${s.schoolName ?? 'School'}`; } }
    function applyThemeToUI(theme) { const root = document.documentElement; const mapping = { primaryGold: '--primary-gold', primaryGoldLight: '--primary-gold-light', primaryGoldDark: '--primary-gold-dark', navyDark: '--navy-dark', navyMedium: '--navy-medium', navyLight: '--navy-light', accentBlue: '--accent-blue', accentPurple: '--accent-purple', textLight: '--text-light', textMuted: '--text-muted', successGreen: '--success-green', glassBg: '--glass-bg', glassBorder: '--glass-border', successGreen: '--success-green', successGreen: '--success-green', errorRed: '--error-red', errorRed: '--error-red' }; if (!theme) return; for (const [k, v] of Object.entries(mapping)) { if (theme[k] && v) root.style.setProperty('--gold-primary', 'linear-gradient(135deg, ${primaryGold} 0%, ${primaryGoldLight} 50%, ${primaryGold} 100%)'); } if (theme.dark && navyDark && navyMedium && navyLight) root.style.setProperty('--gradient-navy', 'linear-gradient(135deg, ${navyDark} 0%, ${navyMedium} 50%, ${navyLight} 100%)'); }
    function updateUploadStatusCallback() { const el = document.getElementById('uploadStatusCallback'); if (!el) return; const s = state.settings; let lines = []; if (!s.uploadsEnabled) lines.push('Uploads are currently disabled.'); if (s.uploadWindowEnabled) { const fmt = (iso) => { try { return new Date(iso).toLocaleSlocaleString(); } catch { return iso; } }; if (s.uploadWindowStartISO && s.uploadWindowEndISO) lines.push(`Uploads are allowed between ${fmt(s.uploadWindowStartISO)} and ${fmt(s.uploadWindowEndISO)}.`); else if (s.uploadWindowStartISO && !s.uploadWindowEndISO) lines.push(`Uploads are allowed after ${fmt(s.uploadWindowStartISO)}.`); else if (!s.uploadWindowStartISO && s.uploadWindowEndISO) lines.push(`Uploads are allowed until ${fmt(s.uploadWindowEndISO)}.`); } if (s.requireApproval) lines.push('Uploads require approval before they appear.'); if (s.profanityFilterEnabled) lines.push('Profanity filter: <strong>ON</strong>'); el.innerHTML = lines.joinlines.length ? el.style.display = 'block' : el.style.display = 'block'; }

    function initCountdown() { updateCountdown(); setInterval(updateCountdown, 1000); }
    function updateCountdown() { const target = parseISTLocalToDate(state.settings.farewellISO); if (!target) return; const now = new Date(); const diff = target - now; const days$1 = document.getElementById('days'); const hours$1 = document.getElementById('hours'); const minutes$1 = document.getElementById('minutes'); const seconds$1 = document.getElementById('seconds'); if (!days$1 || !hours$1 || !minutes$1 || !seconds$1) return; if (diff <= 0) { days$1.textContent = '0'; hours$1.textContent = '0'; minutes$1.textContent = '0'; seconds$1.textContent = '0'; return; } const days = Math.floor(diff / (1000 * 60 * 60 * 24)); const hours = Math.floor(diff % (1000 * 60 * 60 * 24) / (1000 * 60 * 60)); const minutes = Math.floor(diff % (1000 * 60) / (1000 * 60)); const seconds = Math.floor(diff % (1000 * 60) / 1000); days$1.textContent = String(days).padStart(2, '0'); hours$1.textContent = String(hours).padStart(2, '0'); minutes$1.textContent = String(minutes).padStart(2, '0'); seconds$1.textContent = String(seconds).padStart(2, '0'); }
    function parseISTLocalToDate(iso) { if (!iso) return null; const m = iso.match(/^(\d{4})-(\d{2})-(\d{2})T(\d{2}):(\d{2})$/); if (!m) return null; const [, y, mo, d, h, mi] = m.map(Number); return new Date(Date.UTC(y, mo - 1, D, h - 5, m - 30, 0)); }
    function formatFarewellLinesISTLocal() { const d = parseISTLocalToDate(state.settings.farewellISO); if (!d) return; const fmt = new Intl.DateTimeFormat('en-IN', { timeZone: 'Asia/Kolkata', year: 'numeric', month: 'long', day: 'numeric' }).format(d); const timeFmt = new Intl.DateTimeFormat('en-IN', { timeZone: 'Asia/Kolkata', hour: 'numeric', minute: '2-digit' }).format(d); return `${fmt} at ${timeFmt} IST`; }
    function initYearFromISTLocal() { const d = parseISTLocalToDate(state.settings.farewellISO); if (!d) return; const parts = new Intl.DateTimeFormat('en-IN', { timeZone: 'Asia/Kolkata', year: 'numeric' }).formatToParts(d); const y = parts.find(p => p.type === 'year')?.value; return y ? Number(y) : null; }

    function initUpload() { const dropzone = document.getElementById('dropzone'); const fileInput = document.getElementById('fileInput'); const caption = document.getElementById('caption'); const charCount = document.getElementById('charCount'); if (dropzone && fileInput) { dropzone.addEventListener('click', () => fileInput.click()); dropzone.addEventListener('dragover', (e) => { e.preventDefault(); dropzone.classList.add('drag-over'); }); dropzone.addEventListener('dragleave', () => dropzone.classList.remove('drag-over')); dropzone.addEventListener('drop', (e) => { e.preventDefault(); dropzone.classList.remove('drag-over'); handleFiles(e.dataTransfer.files); }); fileInput.addEventListener('change', (e) => { handleFiles(e.target.files); }); } if (caption && charCount) { caption.addEventListener('input', () => { charCount.textContent = caption.value.length; charCount.parentElement.classList.toggle('warning', caption.value.length > 450); }); } const submitBtn = document.getElementById('uploadSubmitBtn'); if (submitBtn) submitBtn.addEventListener('click', submitUpload); }
    function handleFiles(files) { const fileArray = Array.from(files); let totalSize = 0; const validFiles = []; if (fileArray.length > CONFIG.MAX_FILES) { showNotification('error', 'Too many files', `Maximum ${CONFIG.MAX_FILES} files.`); return; } for (const file of fileArray) { if (!CONFIG.ALLOWED_TYPES.includes(file.type)) { showNotification('error', 'Invalid Type', `${file.name} not supported.`); continue; } if (file.size > CONFIG.MAX_FILE_SIZE) { showNotification('error', 'Too Large', `${file.name} exceeds 100MB.`); continue; } totalSize += file.size; validFiles.push(file); } if (totalSize > CONFIG.MAX_TOTAL_SIZE) { showNotification('error', 'Total Size Exceeded', 'Max 200MB total.'); return; } state.selectedFiles = validFiles; updateFilePreview(); }
    function updateFilePreview() { const container = document.getElementById('filePreviewContainer'); const grid = document.getElementById('filePreviewGrid'); const info = document.getElementById('fileInfo'); const formFields = document.getElementById('uploadFormFields'); if (!container || !grid) return; grid.innerHTML = ''; if (state.selectedFiles.length === 0) { container.classList.remove('has-files'); if (formFields) formFields.style.display = 'none'; return; } container.classList.add('has-files'); if (formFields) formFields.style.display = 'block'; let totalSize = 0; state.selectedFiles.forEach(file, index) => { totalSize += file.size; const item = document.createElement('div'); item.className = 'file-preview-item'; if (file.type.startsWith('image/')) { const img = document.createElement('img'); img.src = URL.createObjectURL(file); item.appendChild(img); } else { const video = document.createElement('video'); video.src = URL.createObjectURL(file); item.appendChild(video); } const indicator = document.createElement('div'); indicator.className = 'video-indicator'; indicator.textContent = '▶'; item.appendChild(indicator); } const removeBtn = document.createElement('button'); removeBtn.className = 'remove-file'; removeBtn.textContent = '✕'; removeBtn.onclick = (e) => { e.stopPropagation(); removeFile(index); }; item.appendChild(removeBtn); grid.appendChild(item); }); if (info) { info.textContent = `${state.selectedFiles.length} file(s) · ${formatFileSize(totalSize)}`; } }
    function removeFile(index) { state.selectedFiles.splice(index, 1); updateFilePreview(); }
    function showUploadProgress() { const el = document.getElementById('uploadProgress'); if (el) el.style.display = 'block'; }
    function updateProgress(percent, text) { const el = document.getElementById('uploadProgressFill'); if (el) el.style.width = percent + '%'; const textEl = document.getElementById('uploadProgressText'); if (textEl && text) textEl.textContent = text; }
    function hideUploadProgress() { setTimeout(() => { const el = document.getElementById('uploadProgress'); if (el) el.style.display = 'none'; }, 2000); }
    function resetUploadForm() { state.selectedFiles = []; const uploadForm = document.getElementById('uploadForm'); const uploadFormFields = document.getElementById('uploadFormFields'); if (uploadForm) uploadForm.reset(); if (uploadFormFields) uploadFormFields.style.display = 'block'; if (uploadFormFields) uploadFormFields.classList.remove('active'); }
    function submitUpload() { const fileInput = document.getElementById('fileInput'); const uploadFormFields = document.getElementById('uploadFormFields'); if (!fileInput || state.selectedFiles.length === 0) { showNotification('error', 'No Files', 'Select files.'); return; } const formDataObj = new FormData(); const uploaderName = document.getElementById('uploaderName'); const caption = document.getElementById('caption'); const isAnonymous = document.getElementById('isAnonymous'); const isPublic = document.getElementById('isPublic'); const isFavorite = document.getElementById('isFavorite'); if (uploaderName && uploaderName.value) formDataObj.append('uploader', uploaderName.value.trim()); if (caption && caption.value) formDataObj.append('caption', caption.value.trim()); formDataObj.append('isAnonymous', isAnonymous && isAnonymous.checked ? '1' : '0'); formDataObj.append('isPublic', isPublic && isPublic.checked ? '1' : '0'); formDataObj.append('isFavorite', isFavorite && isFavorite.checked ? '1' : '0'); const progressFill = document.getElementById('uploadProgressFill'); const progressText = document.getElementById('uploadProgressText'); const submitBtn = document.getElementById('uploadSubmitBtn'); if (progressFill) progressFill.classList.add('active'); if (submitBtn) submitBtn.disabled = true; try { const xhr = new XMLHttpRequest(); xhr.upload.addEventListener('progress', (e) => { if (e.lengthComputable && progressFill && progressText) { const percent = Math.round((e.loaded / e.total) * 100); progressFill.style.width = percent + '%'; progressText.textContent = `Uploading... ${percent}% ${e.loaded / 1 * 100}`; } }); xhr.addEventListener('load', () => { let response = {}; try { response = JSON.parse(xhr.responseText); } catch { } if (xhr.status >= 200 && response.success) { const uploadItems = document.getElementById('uploadItems'); const uploadSuccess = document.getElementById('uploadSuccess'); const uploadMore = document.getElementById('uploadMore'); if (uploadItems) uploadItems.style.display = 'none'; if (uploadSuccess) uploadSuccess.classList.add('active'); if (uploadMore && uploadSuccess && uploadSuccess.length > 1) { showNotification('success', 'Uploaded!', `${uploadSuccess.length} uploaded. Pending approval.`); } else { showNotification('success', 'Uploaded!', 'Pending approval.'); } resetUploadForm(); updateFilePreview(); } else { showNotification('error', 'Upload Failed', response.error || 'Upload failed.'); } hideUploadProgress(); } else { showNotification('error', 'Upload Failed', `Server error: ${xhr.status}`); hideUploadProgress(); } if (submitBtn) submitBtn.disabled = false; } ); xhr.addEventListener('error', () => { showNotification('error', 'Upload Failed', 'Network error.'); if (progressFill) progressFill.classList.remove('active'); if (submitBtn) submitBtn.disabled = false; } ); xhr.open('POST', apiUrl('/api/uploads')); xhr.send(formDataObj); } catch (e) { showNotification('error', 'Upload Failed', e.message); if (progressFill) progressFill.classList.remove('active'); if (submitBtn) submitBtn.disabled = false; } }
    function resetUploadForm() { const uploaderName = document.getElementById('uploaderName'); const caption = document.getElementById('caption'); const isAnonymous = document.getElementById('isAnonymous'); const isPublic = document.getElementById('isPublic'); const isFavorite = document.getElementById('isFavorite'); if (uploaderName) uploaderName.value = ''; if (caption && caption.value) { caption.value = ''; if (charCount && charCount.textContent != '0') charCount.textContent = '0'; } if (isAnonymous) isAnonymous.checked = false; if (isPublic && isPublic.checked) isPublic.checked = false; if (isFavorite && isFavorite.checked) isFavorite.checked = false; }

    fileInput.value = '';
    }

    function initMemoryWall() {
    document.querySelectorAll('.memory-filter, .filter-btn').forEach(btn => {
    btn.addEventListener('click', asyncc () => {
      document.querySelectorAll('.memory-filter, .filter-btn').forEach(b => b.classList.remove('active'));
      btn.classList.add('active');
```

```
    state.currentFilter = bm.dataset.filter;
    await loadMemories(true);
  });
});

const loadMoreBtn = document.getElementById('loadMoreBtn');
if (loadMoreBtn) {
  loadMoreBtn.addEventListener('click', async () => {
    await loadMemories(false);
  });
}

async function loadMemories(reset) {
  if (state.memoryLoading) return;
  state.memoryLoading = true;

  if (reset) {
    state.memoryPage = 1;
    state.memoryCursor = null;
    state.memoryMoreHere = true;
    state.memoryItem = [];
    renderMemoryGrid(true);
  }

  try {
    const params = new URLSearchParams();
    params.set('page', Stringtate.memoryPage);
    params.set('limit', Stringtate.memoryLimit);
    if (state.currentFilter != 'all') params.set('type', state.currentFilter);

    const res = await fetchApiUrl('/api/memories?${params}');
    const data = await res.json();

    if (data.success) {
      const newItems = (data.memories || []).map(m => ({
        ...m,
        file_url: mediaUrl(m.file_id)
      ));

      if (reset) {
        state.memories = newItems;
      } else {
        const existingIds = new Set(state.memories.map(m => m.id));
        const deduped = newItems.filter(m => !existingIds.has(m.id));
        state.memories = state.memories.concat(deduped);
      }

      if (typeof data.hasMore === 'boolean') {
        state.memoryMoreHere = data.hasMore;
      } else {
        state.memoryMoreHere = newItems.length === state.memoryLimit;
      }

      state.memoryPage += 1;
      renderMemories();
    }
    } catch (e) {
      console.error('Failed to load memories:', e);
    } finally {
      state.memoryLoading = false;
    }
}

function renderMemoryLoading() {
  const grid = document.getElementById('memoryGrid');
  if (!grid) return;
  grid.innerHTML = '<div class="memory-loading" style="grid-column: 1/-1;"><div class="loading-spinner"></div><p>Loading memories...</p></div>';
}

function renderMemories() {
  const grid = document.getElementById('memoryGrid');
  if (!grid) return;

  const countEl = document.getElementById('memoryCountPill');
  if (countEl) {
    countEl.textContent = '${state.memories.length} loaded${state.memoryMoreHere ? ' + more available' : ''}';
  }

  if (state.memories.length === 0) {
    grid.innerHTML = '<div class="memory-empty" style="grid-column: 1/-1;"><div class="memory-empty-icon">  </div><h3>No Memories Yet</h3><p>Be the first to share!</p></div>';
    const loadMoreWrap = document.getElementById('loadMoreWrap');
    if (loadMoreWrap) loadMoreWrap.style.display = 'none';
    return;
  }

  grid.innerHTML = state.memories.map((memory, index) => {
    <div class="memory-card" data-index="${index}">
      <div class="memory-media" onclick="openLightbox(${index})">
        ${memory.file_type === 'video'
          ? '<video preload="metadata" muted playsinline loop autoplay="false"></video><div class="play-button">  </div>'
          : '<img src="${memory.file_url}" alt="${escapeAttr(memory.caption||'')}" loading="lazy" />'}
        <span class="memory-type-badge">${escapeHtml(memory.memory_type)}</span>
        ${memory.featured ? '<span class="memory-featured-badge">Featured</span>' : ''}
      </div>
      <div class="memory-content">
        <div class="memory-author">
          <div class="memory-avatar">${escapeHtml(memory.student_name || '').charAt(0).toUpperCase()}</div>
          <div class="memory-author-info">
            <div class="memory-author-name">${escapeHtml(memory.student_name || '')}</div>
            <div class="memory-time">${timeAgo(memory.created_at)}</div>
          </div>
        </div>
        <p class="memory-caption">${escapeHtml(memory.caption || '')}</p>
        <div class="memory-actions">
          <div class="action-row">
            <button class="like-btn${liked.has(memory.id) ? 'liked' : ''}" onclick="likeMemory(${memory.id}, this)">
              <span class="like-icon">${liked.has(memory.id) ?  '  '}</span>
              <span class="like-count">${memory.likes || 0}</span>
            </button>
            <button class="comments-open-btn" onclick="openLightbox(${index}); setTimeout(() => scrollCommentsTop(), 50);">  Comments</button>
          </div>
          <div class="mod-group">${renderReactionsBtn(memory)}</div>
        </div>
      </div>
    </div>
  ).join('');
}

const loadMoreWrap = document.getElementById('loadMoreWrap');
if (loadMoreWrap) {
  loadMoreWrap.style.display = (totalsInfoSeen && state.memoryMoreHere) ? 'flex' : 'none';
}

function reactionEmojiFromKey(key) { const m = memory.reactions || {}; return [{ t: 'like', e: '  ' }, { t: 'love', e: '  ' }, { t: 'laugh', e: '  ' }, { t: 'wow', e: '  ' }, { t: 'sad', e: '  ' }].map(x => ({ ...x, n: (m[x.t] || [].length ?? 0) }).filter(x => x.n > 0).map(x => x.e).join('  '); }
async function toggleReaction(memoryId, type) { try { const res = await fetchApiUrl('/api/reactions/${memoryId}'), { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ type: t }); const data = await res.json(); if (data.success) { const m = state.memories.find(x => x.id === memoryId); if (m) m.reactions = data.reactions; renderMemories(); } } catch (e) { console.error('Failed to react:', e); } }
function openReactionPicker(memoryId, anchorEl) { const existing = document.getElementById('reactionPicker'); if (existing) existing.remove(); const pickerHtml = '<div id="reactionPicker" class="reaction-picker">' + ['like', 'love', 'laugh', 'wow', 'sad'].map(t => '<button type="button" data-t="${t}">${reactionEmojiFromKey(t)}</button>').join('') + '</div>'; const wrapper = document.createElement('div'); wrapper.innerHTML = pickerHtml; const picker = wrapper.firstChild; document.body.appendChild(picker); const rect = anchorEl.getBoundingClientRect(); picker.style.position = 'fixed'; picker.style.bottom = (window.innerHeight - rect.top + 8) + 'px'; picker.style.left = Math.max(8, rect.left - 120) + 'px'; picker.querySelectorAll('button').forEach(b => { b.onclick = (e) => { e.stopPropagation(); toggleReaction(memoryId, b.dataset.t); picker.remove(); } ); setTimeout(() => { const closeOnOutside = (ev) => { if (!picker.contains(ev.target)) { picker.remove(); document.removeEventListener('click', closeOnOutside, true); } } ); document.addEventListener('click', closeOnOutside, true); } ); }
function renderReactionsBtn(m) { const total = (m.reactions || [].length ?? 0); const emojis = reactionEmojiFromKey(m); return '<button type="button" onclick="openReactionPicker(${m.id}, this)" title="${total ? 'Reactions: ${emojis}' : 'Add reaction'}">  ${emojis ? emojis + ' ' : ''}${total ? total : ''}</button>'; }
function openLightbox(index) { const lightboxIndex = index; state.replyingTo = null; activeMemoryId = state.memories[index].id; updateLightbox(); const lightbox = document.getElementById('lightbox'); if (!lightbox) lightbox.classList.add('active'); document.body.style.overflow = 'hidden'; }
function closeLightbox() { const lightbox = document.getElementById('lightbox'); if (lightbox) lightbox.classList.remove('active'); activeMemoryId = null; document.body.style.overflow = ''; }
function updateLightbox() { const m = state.memories[lightboxIndex]; if (!m) return; const media = document.getElementById('lightboxMedia'); if (m.file_type == 'video') { media.innerHTML = '<video controls autoplay playsinline loop muted_id="${m.id}" ${m.file_id ? 'data-file_id="${m.file_id}"' : ''}
```

```
  if (video) {
    video.muted = false;
    video.currentTime = 0;
  video.play().catch(e => console.log('Play failed', e));
  }
}

if (overlay) {
  overlay.style.display = 'none';
}
};

async function initIntroVideo() {
  const overlay = document.getElementById('introVideoOverlay');
  const video = document.getElementById('introVideo');
  const skipBtn = document.getElementById('skipIntroBtn');
  const unmuteOverlay = document.getElementById('unmuteOverlay');

  try {
    localStorage.removeItem('introSeen');
    sessionStorage.removeItem('introSeen');
  } catch {e} {}

  if (!overlay || !video) return;

  const introPath = state.settings?.introVideoPath;
  if (!introPath) {
    overlay.style.display = 'none';
    overlay.classList.add('hidden');
    return;
  }

  overlay.style.display = 'flex';
  overlay.classList.remove('hidden');
  overlay.style.opacity = '1';

  if (unmuteOverlay) {
    unmuteOverlay.style.display = 'none';
  }

  const src = mediaUrl('/uploads/' + introPath.replace(/^/+/g, ''));

  video.pause();
  video.removeAttribute('src');
  video.load();
  video.src = src;
  video.currentTime = 0;
  video.load();

  video.oneended = (){ => skipIntro();
  video.onerror = (){ => skipIntro();

  if (skipBtn) {
    skipBtn.onclick = (e) => {
      e.stopPropagation();
      skipIntro();
    };
    skipBtn.style.display = state.settings?.introHideSkip ? 'none' : '';
  }

  try {
    video.muted = false;
    await video.play();
  if (unmuteOverlay) unmuteOverlay.style.display = 'none';
  } catch (e) {
    try {
      video.muted = true;
      await video.play();
      if (unmuteOverlay) unmuteOverlay.style.display = 'flex';
    } catch (e2) {
      if (unmuteOverlay) unmuteOverlay.style.display = 'flex';
    }
  }
}

function skipIntro() {
  const overlay = document.getElementById('introVideoOverlay');
  const video = document.getElementById('introVideo');

  if (overlay) {
    overlay.style.opacity = '0';
    overlay.style.transition = 'opacity 0.5s ease';
    setTimeout(() => {
      overlay.style.display = 'none';
      overlay.classList.add('hidden');
    }, 500);
  }

  if (video) {
    video.pause();
    video.currentTime = 0;
  }
}

// ═══════════════════════════════════════════════════════════════════════════════
// MISSING ADMIN FUNCTIONS (Video & Hashes)
// ═══════════════════════════════════════════════════════════════════════════════
async function uploadIntroVideoFile(file) {
  if (!file) return;
  const status = document.getElementById('currentIntroVideoStatus');
  if (status) status.textContent = 'Uploading...';
  const fd = new FormData();
  fd.append('file', file);
  try {
    const res = await fetch(apiUrl('/api/admin/settings/intro-video'), {
      method: 'POST',
      headers: { 'Authorization': 'Bearer ' + state.adminToken },
      body: fd
    });
    const data = await res.json();
    if (data.success) {
      showNotification('success', 'Uploaded', 'Intro video saved.');
      if (status) status.textContent = 'Video uploaded successfully';
      state.settings.introVideoPath = data.path;
    } else {
      showNotification('error', 'Failed', data.error || 'Upload failed');
      if (status) status.textContent = 'Upload failed';
    }
  } catch (e) {
    showNotification('error', 'Error', e.message);
    if (status) status.textContent = 'Upload error';
  }
}

async function removeIntroVideo() {
  if (!confirm('Remove intro video?')) return;
  try {
    const res = await fetch(apiUrl('/api/admin/settings/intro-video'), {
      method: 'DELETE',
      headers: { 'Authorization': 'Bearer ' + state.adminToken }
    });
    const data = await res.json();
    if (data.success) {
      showNotification('success', 'Removed', 'Intro video deleted.');
      const status = document.getElementById('currentIntroVideoStatus');
      if (status) status.textContent = 'No video';
      state.settings.introVideoPath = null;
    } else showNotification('error', 'Failed', data.error);
  } catch (e) { showNotification('error', 'Error', e.message); }
}

async function resetUploadHashes() {
  if (!confirm('Are you sure you want to reset all duplicate detection hashes?')) return;
  try {
    const res = await fetch(apiUrl('/api/admin/reset-hashes'), {
      method: 'POST',
      headers: { 'Authorization': 'Bearer ' + state.adminToken }
    });
    const data = await res.json();
    if (data.success) showNotification('success', 'Hashes Reset', 'Duplicate detection cleared.');
    else showNotification('error', 'Failed', data.error);
  } catch (e) { showNotification('error', 'Error', e.message); }
}
// ═══════════════════════════════════════════════════════════════════════════════
// COMPILATION PLAYER
// ═══════════════════════════════════════════════════════════════════════════════
let currentCompilation = null; let currentSlideIndex = 0; let compilationTimer = null;

async function playCompilation(compilationId) {
  try {
    const res = await fetch(apiUrl('/api/compilations/' + compilationId));
    const data = await res.json();
    if (!data.success || !data.compilation) return showNotification('error', 'Error', 'Could not load compilation');
    currentCompilation = data.compilation; currentSlideIndex = 0;
    const modal = document.getElementById('compilationPlayer');
    const player = document.getElementById('compilationPlayer');
    if (player) player.classList.add('active');
    document.body.style.overflow = 'hidden';
    showCompilationSlide(0);
  } catch (e) { showNotification('error', 'Error', e.message); }
}

function renderCompilationSlides() {
  const container = document.getElementById('compilationSlides');
  const progress = document.getElementById('compilationProgress');
  if (!container || !currentCompilation) return;
  const transClass = 'trans-' + (currentCompilation.transitionType || 'fade');

  container.innerHTML = currentCompilation.slides.map((slide, i) => {
    // Fallback robustly: try API provided file_url first, then local cached memory arrays
    const memory = compAvailableMemories.find(m => m.id === slide.memoryId);
    const mediaUrl = slide.file_url || (memory ? memory.file_url : '');
    const imgUrl = mediaUrl ? mediaUrl(mediaUrl) : '';

    return `<div class="compilation-slide ${isCurrentClass}" data-index="${i}">
      ${imgUrl ? '<img src="${imgUrl}" alt="Memory" onerror="this.style.display=\'none\'" />
      ${slide.caption ? '<div class="compilation-caption">${escapeHtml(slide.caption)}</div>' : ''}
    </div>`;
  }).join('');

  progress.innerHTML = currentCompilation.slides.map(_, i) => `<div class="compilation-dot" onclick="goToCompilationSlide(${i})"></div>`).join('');
}
function showCompilationSlide(index) {
  if (!currentCompilation) return;
  currentSlideIndex = index;
  document.querySelectorAll('.compilation-slide').forEach(s, i) => s.classList.toggle('active', i === index);
  document.querySelectorAll('.compilation-dot').forEach(d, i) => d.classList.toggle('active', i === index);
  if (compilationTimer) clearTimeout(compilationTimer);
  if (currentCompilation.displayMode === 'auto') {
    const duration = (currentCompilation.slides[index].duration || 5) * 1000;
    compilationTimer = setTimeout(() => {
      if (currentSlideIndex < currentCompilation.slides.length - 1) {
        showCompilationSlide(currentSlideIndex + 1);
      } else {
        showCompilationSlide(0);
      }
    }, duration);
  }
}

function nextCompilationSlide() { if (currentCompilation) showCompilationSlide((currentSlideIndex + 1) % currentCompilation.slides.length); }
function prevCompilationSlide() { if (currentCompilation) showCompilationSlide((currentSlideIndex - 1 + currentCompilation.slides.length) % currentCompilation.slides.length); }
function goToCompilationSlide(index) { showCompilationSlide(index); }

function closeCompilationPlayer() {
  if (compilationTimer) clearTimeout(compilationTimer);
  const player = document.getElementById('compilationPlayer');
  if (player) player.classList.remove('active');
  document.body.style.overflow = '';
  currentCompilation = null;
}

// ═══════════════════════════════════════════════════════════════════════════════
// INLINE EDITING
// ═══════════════════════════════════════════════════════════════════════════════
function makeInlineEditable(element, memoryId, field) {
  if (!element || !canEditMemory()) return;
  element.classList.add('inline-editable');
  element.setAttribute('data-memory-id', memoryId);
  element.setAttribute('data-field', field);
  element.addEventListener('dblclick', function(e) { e.stopPropagation(); startInlineEdit(this); });
}

function startInlineEdit(element) {
  if (element.classList.contains('editing')) return;
  const originalText = element.textContent;
  element.classList.add('editing');
  const input = document.createElement('input');
  input.type = 'text'; input.className = 'inline-edit-input'; input.value = originalText;
  element.textContent = ''; element.appendChild(input); input.focus(); input.select();

  const save = async () => {
    const newValue = input.value.trim();
    element.classList.remove('editing');
    element.textContent = newValue || originalText;
    if (newValue && newValue !== originalText) {
      try {
        const res = await fetch(apiUrl('/api/admin/memories/' + element.getAttribute('data-memory-id')), {
          method: 'PATCH', headers: { 'Content-Type': 'application/json', 'Authorization': 'Bearer ' + state.adminToken },
          body: JSON.stringify({ [element.getAttribute('data-field')]: newValue })
        });
        const data = await res.json();
        if (data.success) showNotification('success', 'Saved', 'Updated ' + element.getAttribute('data-field'));
        else { element.textContent = originalText; showNotification('error', 'Failed', data.error); }
      } catch (e) { element.textContent = originalText; showNotification('error', 'Error', e.message); }
    }
  };
  input.addEventListener('blur', save);
  input.addEventListener('keydown', (e) => { if (e.key === 'Enter') save(); if (e.key === 'Escape') { element.classList.remove('editing'); element.textContent = originalText; } });
}

// ═══════════════════════════════════════════════════════════════════════════════
// COMPILATION CREATOR
// ═══════════════════════════════════════════════════════════════════════════════
let compSelectedSlides = []; let compEditingId = null; let compAvailableMemories = [];

function openCompilationCreator(editId = null) {
  compEditingId = editId; compSelectedSlides = [];
  document.getElementById('compName').value = ''; document.getElementById('compDisplayMode').value = 'auto';
  document.getElementById('compTransition').value = 'fade'; document.getElementById('compDefaultDuration').value = '5';
  loadCompPhotosGrid(); renderCompSlidesPreview();
  document.getElementById('compilationCreatorModal').classList.add('active');
}

function closeCompilationCreator() { document.getElementById('compilationCreatorModal').classList.remove('active'); compSelectedSlides = []; compEditingId = null; }

async function loadCompPhotosGrid() {
  const grid = document.getElementById('compPhotosGrid'); if (!grid) return;
  try {
    const res = await fetch(apiUrl('/api/admin/memories?limit=5000&filter=approved'), { headers: { 'Authorization': 'Bearer ' + state.adminToken } });
    const data = await res.json();
    if (data.success) return;
    const memories = (data.memories || []).filter(m => m.file_type === 'image');
    compAvailableMemories = memories;
    grid.innerHTML = memories.map(m => `<div class="compilation-photo-item" data-id="${m.id}" onclick="toggleCompPhoto(${m.id})"><img src="${mediaUrl(m.file_url)}" alt="" />${m.file_type && m.file_type !== 'image' ? '<div class="photo-check">✓</div>' : ''}</div>`).join('') || '<div class="mini-pdl">No photos available</div>';
  } catch (e) { grid.innerHTML = '<div class="mini-pdl">Error loading photos</div>'; }
}

function toggleCompPhoto(memoryId) {
  const idx = compSelectedSlides.findIndex(s => s.memoryId === memoryId);
  if (idx >= 0) compSelectedSlides.splice(idx, 1);
  else compSelectedSlides.push({ memoryId, caption: '', duration: parseInt(document.getElementById('compDefaultDuration').value) || 5 });
  updateCompPhotosSelection(); renderCompSlidesPreview();
}

function updateCompPhotosSelection() { document.querySelectorAll('.compilation-photo-item').forEach(el => { el.classList.toggle('selected', compSelectedSlides.some(s => s.memoryId === parseInt(el.dataset.id))); } }

function renderCompSlidesPreview() {
  const preview = document.getElementById('compSlidesPreview'); if (!preview) return;
  if (compSelectedSlides.length === 0) { preview.innerHTML = '<div class="mini-pdl" style="margin:auto;">No slides selected yet</div>'; return; }
  preview.innerHTML = compSelectedSlides.map((slide, i) => {
    const mem = compAvailableMemories.find(m => m.id === slide.memoryId || state.memories.find(m => m.id === slide.memoryId);
    const imgUrl = mem ? mediaUrl(mem.file_url) : '';
    return `<div class="compilation-slide-preview" data-index="${i}">${imgUrl ? '<img src="${imgUrl}" alt="" />' : ''}<input type="text" placeholder="Caption..." value="${escapeAttr(slide.caption)}" onchange="updateSlideCaption(${i}, this.value)" />
<input type="number" min="1" max="60" value="${slide.duration}" onchange="updateSlideDuration(${i}, this.value)" style="margin-top:4px;" />
<button class="remove-slide" onclick="removeCompSlide(${i})">×</button></div>`;
  }).join('');
}

function updateSlideCaption(index, caption) { if (compSelectedSlides[index]) compSelectedSlides[index].caption = caption; }
function updateSlideDuration(index, duration) { if (compSelectedSlides[index]) compSelectedSlides[index].duration = Math.max(1, Math.min(60, parseInt(duration) || 5)); }
function removeCompSlide(index) { compSelectedSlides.splice(index, 1); updateCompPhotosSelection(); renderCompSlidesPreview(); }

async function saveCompilation() {
  const name = document.getElementById('compName').value.trim();
  if (!name) return showNotification('error', 'Name required', 'Enter a compilation name');
  if (compSelectedSlides.length < 2) return showNotification('error', 'Need slides', 'Select at least 2 photos');
  const payload = { name, slides: compSelectedSlides, displayMode: document.getElementById('compDisplayMode').value, transitionType: document.getElementById('compTransition').value };
  try {
    const url = compEditingId ? apiUrl('/api/admin/compilations/' + compEditingId) : apiUrl('/api/admin/compilations');
    const res = await fetch(url, { method: 'POST', headers: { 'Content-Type': 'application/json', 'Authorization': 'Bearer ' + state.adminToken }, body: JSON.stringify(payload) });
  }
```

```
    const data = await res.json();
    if (data.success) { showNotification('success', 'Saved', compEditingId ? 'Compilation updated' : 'Compilation created'); closeCompilationCreator(); loadCompilationsAdmin(); loadPublicCompilations(); }
    else showNotification('error', 'Failed', data.error);
  } catch (e) { showNotification('error', 'Error', e.message); }
}

function renderCompilationsPanelRemix() {
  return `<div class="star-card" style="text-align:left;"><div style="display:flex; justify-content:space-between; align-items:center; gap:10px; flex-wrap:wrap;"><h3 style="font-family:var(--font-display); color:var(--primary-gold); font-size:1.3rem;">Memory Compilations</h3><div style="display:flex; gap:10px;"><button class="admin-btn admin-btn-secondary" onclick="loadCompilationsAdmin()">Refresh</button><button class="admin-btn admin-btn-primary" onclick="openCompilationCreator()">+ New Compilation</button></div></div><div class="mink-pill" style="margin-top:12px;">Create photo slideshows with transitions and captions</div><div id="compilationsListAdmin" style="margin-top:16px;">${loading}</div>`;
}

async function loadCompilationsAdmin() {
  try {
    const res = await fetch(apiUrl('/api/compilations'));
    const data = await res.json();
    if (!data.success) return;
    const list = document.getElementById('compilationsListAdmin'); if (!list) return;
    if (!data.compilations || data.compilations.length === 0) { list.innerHTML = '<div class="mink-pill">No compilations yet. Create your first one!</div>'; return; }
    list.innerHTML = data.compilations.map(c => `<div class="admin-memory-card" style="padding:10px; margin-bottom:12px;"><div style="display:flex; justify-content:space-between; align-items:center; flex-wrap:wrap; gap:10px;"><div><div style="font-weight:800; font-size:1.1rem;">${escapeHtml(c.name)}</div><div style="color:var(--text-muted); font-size:0.85rem; margin-top:4px;">${c.slides.length} slides • ${c.displayMode} mode • ${c.transitionType} transition</div></div><div style="display:flex; gap:8px; flex-wrap:wrap;"><button class="admin-btn admin-btn-primary" onclick="playCompilation(${c.id})">  Play</button><button class="admin-btn admin-btn-secondary" onclick="editCompilationAdmin(${c.id})">Edit</button><button class="admin-btn admin-btn-danger" onclick="deleteCompilationAdmin(${c.id})">Delete</button></div></div><div style="display:flex; gap:8px; margin-top:12px; overflow-x:auto; padding:8px 0;">${c.slides.slice(0, 8).map(s => { const mem = compAvailableMemories.find(m => m.id === s.memoryId); return mem ? `<div style="${mem.media_type==='video'?'width:60px;':''} height:60px; object-fit:cover; border-radius:8px;" >${mem.media_type==='video'?'<video muted />':''}</div>` : ''; }).join('')}${c.slides.length > 8 ? '<span class="mink-pill">+${c.slides.length - 8} more</span>' : ''}</div></div>`).join('');
  } catch (e) { console.error('Load compilations error', e); }
}

async function editCompilationAdmin(id) {
  try {
    const res = await fetch(apiUrl('/api/compilations' + id));
    const data = await res.json();
    if (!data.success || !data.compilation) return;
    compEditingId = id; compSelectedSlides = data.compilation.slides.map(s => ({ memoryId: s.memoryId, caption: s.caption || '', duration: s.duration || 5 ?:));
    document.getElementById('compName').value = data.compilation.name; document.getElementById('compDisplayMode').value = data.compilation.displayMode || 'auto'; document.getElementById('compTransition').value = data.compilation.transitionType || 'fade';
    loadCompPhotosGrid(); setTimeout(() => { updateCompPhotoSelection(); renderCompSlidesPreview(); }, 500);
    document.getElementById('compilationCreatorModal').classList.add('active');
  } catch (e) { showNotification('error', 'Error', e.message); }
}

async function deleteCompilationAdmin(id) {
  if (!confirm('Delete this compilation?')) return;
  try {
    const res = await fetch(apiUrl('/api/admin/compilations' + id), { method: 'DELETE', headers: { 'Authorization': 'Bearer ' + state.adminToken } });
    const data = await res.json();
    if (data.success) { showNotification('success', 'Deleted', 'Compilation removed'); loadCompilationsAdmin(); loadPublicCompilations(); }
    else showNotification('error', 'Failed', data.error);
  } catch (e) { showNotification('error', 'Error', e.message); }
}


// ═══════════════════════════════════════════════════════════════════════════════
// PUBLIC COMPILATIONS
// ═══════════════════════════════════════════════════════════════════════════════
async function loadPublicCompilations() {
  const section = document.getElementById('compilations');
  const list = document.getElementById('publicCompilationsGrid');
  if (!section || !list) return;

  try {
    const res = await fetch(apiUrl('/api/compilations'));
    const data = await res.json();

    if (data.success || !data.compilations || !data.compilations.length) {
      section.style.display = 'none';
      return;
    }

    section.style.display = 'block';
    list.innerHTML = data.compilations.map(c => `
      <div class="memory-card">
        <div class="memory-content">
          <div class="memory-author">
            <div class="memory-avatar">${c.name[0]}</div>
            <div class="memory-author-info">
              <div class="memory-author-name">${escapeHtml(c.name)} (${c.slides.length} slides)</div>
              <div class="memory-time">${c.slides.length} ${c.displayMode} mode</div>
            </div>
          </div>
          ${c.transitionType ? '<p class="memory-caption">Transition: ${escapeHtml(c.transitionType)}</p>' : ''}
          <div class="memory-actions">
            <div class="action-row">
              <button class="btn btn-primary" type="button" onclick="playCompilation(${c.id})">  Play Compilation</button>
            </div>
          </div>
        </div>
      </div>
    ).join('');
  } catch (e) {
    console.error('Failed to load public compilations', e);
    section.style.display = 'none';
  }
}


// ═══════════════════════════════════════════════════════════════════════════════
// BATCH UPLOAD (ADMIN)
// ═══════════════════════════════════════════════════════════════════════════════
function openBatchUpload() {
  document.getElementById('batchUploadModal').classList.add('active');
  document.getElementById('batchName').value = ''; document.getElementById('batchCaption').value = ''; document.getElementById('batchType').value = 'Friends';
  document.getElementById('batchFiles').value = ''; document.getElementById('batchFileLabel').textContent = 'Click to select files';
  document.getElementById('batchProgress').style.display = 'none'; document.getElementById('startBatchUpl').disabled = false;
}

function closeBatchUpload() { document.getElementById('batchUploadModal').classList.remove('active'); }
function updateBatchFileCount(input) { document.getElementById('batchFileLabel').textContent = input.files && input.files.length > 0 ? `${input.files.length} file(s) selected` : 'Click to select files'; }

async function submitBatchUpload() {
  const name = document.getElementById('batchName').value.trim(); const caption = document.getElementById('batchCaption').value.trim();
  const type = document.getElementById('batchType').value; const fileInput = document.getElementById('batchFiles');
  const autoApprove = document.getElementById('batchAutoApprove').checked;
  if (!name || !caption || !fileInput.files.length) return showNotification('error', 'Missing fields', 'Fill all fields and select files.');

  document.getElementById('startBatchUpl').disabled = true;
  const progressDiv = document.getElementById('batchProgress'); const fill = document.getElementById('batchProgressFill'); const text = document.getElementById('batchProgressText');
  progressDiv.style.display = 'block';

  const files = Array.from(fileInput.files); const total = files.length; let successCount = 0; const CHUNK_SIZE = 5;
  for (let i = 0; i < total; i += CHUNK_SIZE) {
    const chunk = files.slice(i, i + CHUNK_SIZE);
    const fd = new FormData(); fd.append('name', name); fd.append('caption', caption); fd.append('type', type); fd.append('autoApprove', autoApprove ? 'true' : 'false');
    chunk.forEach(f => fd.append('files', f));
    try {
      text.textContent = `Uploading batch ${Math.ceil(i/${CHUNK_SIZE})} of ${Math.ceil(total/${CHUNK_SIZE})}...`;
      const res = await fetch(apiUrl('/api/admin/upload-batch'), { method: 'POST', headers: { 'Authorization': 'Bearer ${state.adminToken}' }, body: fd });
      const data = await res.json();
      if (data.success) successCount += (data.count || 0); else showNotification('error', 'Error in chunk', data.error);
      fill.style.width = Math.min(100, Math.round((i + chunk.length) / total) * 100) + '%';
    } catch (e) { console.error(e); }
  }
  showNotification('success', 'Batch Complete', `${successCount} uploaded successfully.`);
  closeBatchUpload(); loadAdminMemories(true);
}

// ═══════════════════════════════════════════════════════════════════════════════
// FAVICON HANDLING
// ═══════════════════════════════════════════════════════════════════════════════
function updateFavicon() {
  const t = state.settings; let faviconUrl = 'favicon.ico';
  if (t.faviconUploaded) {
    faviconUrl = 'favicon.ico?v=' + Date.now();
  } else if (t.logoDesc) {
    const canvas = document.createElement('canvas'); canvas.width = 32; canvas.height = 32;
    const ctx = canvas.getContext('2d'); ctx.font = '28px sans'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle';
    ctx.fillStyle = t.logoDesc; ctx.fillRect(0, 0, 32, 32); ctx.fillText(t.logoDesc, 16, 18); faviconUrl = canvas.toDataURL('image/png');
  }
  let link = document.querySelector("link[rel*='icon']");
  if (!link) { link = document.createElement('link'); link.rel = 'icon'; document.head.appendChild(link); }
  link.href = faviconUrl;
}

// This forces the compilations to load the second the website opens!
document.addEventListener('DOMContentLoaded', () => {
  if (typeof loadPublicCompilations === 'function') {
    loadPublicCompilations();
  }
});
</script>


<style>
/* === SNIPER_PATCH_V2_START === */
:root{
  --sniper-overlay-z: 12000;
}
body.sniper-ui-hidden * :not(#toastRackOverlay):not(#sniperSfxHost):not(#sniperGlobalFx):not(script):not(style){
  display:none !important;
}
#sniperGlobalFx{position:fixed; inset:0; pointer-events:none; z-index:var(--sniper-overlay-z);}
.sniper-disposable-flash{
  position:fixed; inset:0; background:#ffF; opacity:0; pointer-events:none;
  z-index:calc(var(--sniper-overlay-z) + 50);
}
.sniper-disposable-flash.active{animation:sniperFlash 1.8s ease forwards;}
@keyframes sniperFlash{
  0%{opacity:0}
  10%{opacity:1}
  30%{opacity:0}
  100%{opacity:0}
}
.sniper-face-bell{
  position:fixed; inset:0; z-index:calc(var(--sniper-overlay-z) + 20);
  display:none; align-items:center; justify-content:center; flex-direction:column;
  background:rgba(0,0,0,0.92); text-align:center; padding:30px;
}
.sniper-face-bell.active{display:flex}
.sniper-face-bell h1{
  font-family:var(--font-display); font-size:clamp(2.2rem,8vw,5rem); color:var(--primary-gold);
}
.sniper-face-bell p{color:#ffF; opacity:.85; margin-top:16px; font-size:1.1rem;}
.sniper-final-video{
  width:min(1100px,90vw); max-height:70vh; border-radius:10px; overflow:hidden;
  box-shadow:0 20px 80px rgba(0,0,0,.6); margin-top:24px; background:#000;
}
.sniper-final-video video{width:100%; height:100%; display:block; object-fit:contain;}
.sniper-teardown-fire{
  margin-left:12px;
}
#toastRackOverlay{
  position:fixed; inset:0; background:#000; z-index:calc(var(--sniper-overlay-z) + 30);
  display:none; overflow:hidden;
}
#toastRackOverlay.active{display:block}
#toastRackStage{
  position:absolute; inset:0; display:flex; align-items:center; justify-content:center;
}
#toastRackStage img{
  max-width:80vw; max-height:80vh; object-fit:contain; border-radius:16px;
  box-shadow:0 30px 100px rgba(0,0,0,.7); will-change:transform, opacity;
}
#toastRackControls{
  position:absolute; left:28px; right:28px; bottom:28px; display:flex; justify-content:space-between; align-items:center;
  z-index:2;
}
.sniper-pill{
  background:rgba(255,255,255,.12); border:1px solid rgba(255,255,255,.18); color:#ffF;
  padding:10px 14px; border-radius:999px; backdrop-filter:blur(12px);
}
#mosqPhysicsLayer{
  position:fixed; inset:0; pointer-events:none; z-index:calc(var(--sniper-overlay-z) + 10); overflow:hidden;
}
.mosq-particle{
  position:fixed; font-size:28px; will-change:transform; user-select:none; pointer-events:none;
  text-shadow:0 4px 18px rgba(0,0,0,.35);
}
.teacher-card{position:relative}
.teacher-audio-badge{
  position:absolute; top:12px; right:12px; z-index:3; padding:6px 10px; border-radius:999px;
  background:rgba(0,0,0,.52); color:var(--primary-gold); font-size .75rem; border:1px solid rgba(255,255,255,.12);
}
#boombotDock{
  position:fixed; left:18px; right:18px; bottom:14px; z-index:2500;
  display:flex; gap:16px; align-items:flex-end; justify-content:center; pointer-events:none;
}
#boombotDock *{pointer-events:auto}
#boombotUnit{
  width:auto;Cl0px,50vw}; background:linear-gradient(180deg,#6c4c5d,#2a2a31);
  border:1px solid rgba(255,255,255,.18); border-radius:24px; box-shadow:0 20px 60px rgba(0,0,0,.45);
  padding:16px;
}
.boombotu-top{display:flex; gap:12px; align-items:center; justify-content:space-between; margin-bottom:12px;}
.boombotu-brand{font-weight:700; color:var(--primary-gold);}
.boombotu-dose{
  height:110px; border-radius:16px; border:2px solid rgba(255,255,255,.12); background:#111;
  display:flex; align-items:center; justify-content:center; color:#888; pointerrelative; overflow:hidden;
}
.boombotu-doochae-tape{background:linear-gradient(180deg,#1b1b1f,#0a0a0c);}
.boombotu-speakers{display:grid; grid-template-columns:18 1fr; gap:12px; margin-top:12px;}
.boombotu-speaker{
  height:60px; border-radius:50%; background:radial-gradient(circle,#0d0d0f 0 20%,#2b2b33 21% 45%,#6c6c6f 46% 60%,#444 61% 100%);
  border:2px solid rgba(255,255,255,.08);
}
#cameraRack{
  display:flex; gap:10px; flex-wrap:wrap; justify-content:center; margin-bottom:6px;
}
.camera{
  min-width:140px; padding:12px 10px; border-radius:14px; cursor:grab;
  background:linear-gradient(180deg,#0b3b8,#0b1f38); color:#111; font-weight:700; border:1px solid rgba(0,0,0,.12);
  box-shadow:0 10px 24px rgba(0,0,0,.28);
}
.camera.dragging{opacity:.5}
#ghostCursorBuggies{
  position:fixed; right:18px; bottom:18px; z-index:2600;
}
.ghost-cursor{
  position:fixed; transform:translate(-2px,-2px); pointer-events:none; z-index:2601;
}
.ghost-cursor .tip{
  width:14px; height:14px; background:#ffF; clip-path:polygon(0 0,100% 50%,50% 55%,68% 100%,48% 100%,35% 64%,0 100%);
  box-shadow:0 2px 8px rgba(0,0,0,.3);
}
.ghost-cursor .label{
  margin-left:8px; margin-top:-8px; display:inline-block; padding:4px 8px; border-radius:999px; font-size:11px; color:#ffF;
  background:rgba(0,0,0,.6); border:1px solid rgba(255,255,255,.16);
}
#ghostCursorPopup, #paperTutorial, #ghostOffModal{
  position:fixed; inset:0; z-index:calc(var(--sniper-overlay-z) + 40); display:none; align-items:center; justify-content:center;
  background:rgba(0,0,0,.7); padding:20px;
}
#ghostCursorPopup.active, #paperTutorial.active, #ghostOffModal.active{display:flex}
.sniper-modal{
  max-width:640px; width:100%; border-radius:24px; background:var(--navy-medium);
  border:1px solid var(--glass-border); padding:24px; box-shadow:0 20px 80px rgba(0,0,0,.5);
}
.sniper-modal h3{font-family:var(--font-display); color:var(--primary-gold); margin-bottom:10px;}
.sniper-modal p{color:var(--text-muted); margin-bottom:16px;}
.sniper-actions{display:flex; gap:12px; flex-wrap:wrap;}
#paperAirplaneLayer{
  position:fixed; inset:0; pointer-events:none; z-index:2602; overflow:hidden;
}
.paper-plane{
  position:fixed; width:34px; height:34px; transform:translate(-18px,-
  filter:drop-shadow(0 8px 15px rgba(0,0,0,.3));
}
.paper-plane::before{
  content:""; position:absolute; inset:0; background:linear-gradient(135deg,#ffF,#e8e8e8);
  clip-path:polygon(0 48%,100% 0,62% 100%,48% 63%);
}
#mailNeedles{
  position:fixed; left:18px; bottom:118px; z-index:2600;
}
#lockerTransition{
  position:fixed; inset:0; z-index:calc(var(--sniper-overlay-z) + 25); pointer-events:none; display:none;
}
#lockerTransition.active{display:block}
.locker-door{
  position:absolute; top:0; bottom:0; width:50%; background:
  linear-gradient(180deg,#5a5d5a,#3d3139),
  repeating-linear-gradient(90deg,rgba(255,255,255,.04),rgba(255,255,255,.04) 2px,transparent 2px,transparent 40px);
  border:2px solid rgba(0,0,0,.3);
  box-shadow:inset 0 0 0px rgba(255,255,255,.05);
}
.locker-left{left:-52%; transform:translate(-2px,-2px); clip-path:polygon(0 0,100% 0,100% 100%,0 100%);}
.locker-right{right:-52%; transform:translate(2px,-2px); clip-path:polygon(0 0,100% 0,100% 100%,0 100%);}
#lockerTransition.active .locker-left{animation:lockerLeft 1.6s ease forwards;}
#lockerTransition.active .locker-right{animation:lockerRight 1.6s ease forwards;}
@keyframes lockerLeft{
  0%{left:-52%} 28%{left:0} 72%{left:0} 100%{left:-52%}
}
@keyframes lockerRight{
```

```
  0%{right:-52%} 28%{right:0} 72%{right:0} 100%{right:-52%}
}
.rip-ball{
  position:fixed; background:rgba(255,255,255,.86); backdrop-filter:blur(4px); z-index:calc(var(--sniper-overlay-z) + 45);
  border:1px solid rgba(255,255,255,.14);
}
#distanceMapSection{background:linear-gradient(180deg,#090b13,#111822c)}
#distanceMapWrap{
  min-height:500px; border-radius:24px; position:relative; overflow:hidden; background:radial-gradient(circle at center,#182440 0,#090b13 70%);
  border:1px solid rgba(255,255,255,.08);
}
#distanceGlobe{
  position:absolute; width:min(520px,70vw); aspect-ratio:1; border-radius:50%;
  left:50%; top:50%; transform:translate(-50%,-50%);
  background:
    radial-gradient(circle at 30% 30%,rgba(255,255,255,.18),transparent 18%),
    radial-gradient(circle at 35% 40%,rgba(212,175,55,.14),transparent 25%),
    radial-gradient(circle at 50% 50%,#1f2f5e 0,#0a1d39 58%,#050710c 100%);
  box-shadow:0 0 0 1px rgba(255,255,255,.08), inset -40px -30px 80px rgba(0,0,0,.38), 0 20px 80px rgba(0,0,0,.45);
}
#distanceGlobe::after{
  content:""; position:absolute; inset:10%; border-radius:50%;
  border:1px dashed rgba(255,255,255,.12);
}
.distance-school-dot{
  position:absolute; width:12px; height:12px; border-radius:50%; background:gold; box-shadow:0 0 14px gold, 0 0 32px gold;
  left:50%; top:50%; transform:translate(-50%,-50%);
}
.distance-arc{
  position:absolute; left:50%; top:50%; width:2px; transform-origin:bottom center;
  background:linear-gradient(180deg,rgba(212,175,55,.95),rgba(212,175,55,0));
  opacity:.92; filter:drop-shadow(0 0 8px rgba(212,175,55,.45));
}
#distanceControls{
  position:relative; z-index:2; max-width:980px; margin:0 auto; padding:30px;
  display:flex; gap:14px; flex-wrap:wrap; justify-content:center;
}
.secret-memory{
  position:relative; overflow:hidden;
}
.secret-memory::after{
  content:""; position:absolute; inset:0; background:
  radial-gradient(circle at 50% 50%, rgba(212,175,55,.12), rgba(255,255,255,.03) 35%, rgba(0,0,0,.58) 78%);
  filter:blur(18px);
  backdrop-filter:blur(10px) saturate(.8);
}
.secret-memory .scratch-canvas{
  position:absolute; inset:0; z-index:4; cursor:crosshair;
}
.secret-memory .secret-tip{
  position:absolute; left:50%; top:50%; transform:translate(-50%,-50%);
  z-index:5; padding:10px 14px; border-radius:999px; background:rgba(0,0,0,.58); color:#ff7; font-size:.8rem; pointer-events:none;
  border:1px solid rgba(255,255,255,.15);
}
</style>

<div id="sniperGlobalFx">
  <div id="sniperDisposableFlash" class="sniper-disposable-flash"></div>
  <div id="snipPhysicsLayer"></div>
  <div id="paperAirplaneLayer"></div>
  <div id="lockerTransition">
    <div class="locker-door locker-left"></div>
    <div class="locker-door locker-right"></div>
  </div>
  <div id="sniperLastHell" class="sniper-last-hell">
    <h1>Last Hell</h1>
    <p>One final ring. One final goodbye.</p>
    <div class="sniper-final-video">
      <video id="sniperFinalGoodbyeVideo" playsinline></video>
    </div>
  </div>
</div>

<div id="lessRackOverlay">
  <div id="lessRackStage"></div>
  <div id="lessRackControls">
    <div class="sniper-pill" id="lessRackStatus">Lean Rack Mode</div>
    <button class="lbtn lbtn-danger" type="button" onclick="stopLessRackMode()">Exit</button>
  </div>
</div>

<div id="boombox-Dock">
  <div id="cassetteRack"></div>
  <div id="boomboxUnit">
    <div class="boombox-brand">Cornerstone StereoDeck</div>
    <button class="lbtn lbtn-secondary" type="button" id="ejectTapeBtn">Eject</button>
  </div>
  <div class="boombox-door" id="boomboxDoor">Drag a tape here</div>
  <div class="boombox-speakers">
    <div class="boombox-speaker"></div>
    <div class="boombox-speaker"></div>
  </div>
</div>

<button class="lbtn lbtn-secondary" id="ghostCursorTroggle" type="button">Ghost Cursors: ON</button>
<button class="lbtn lbtn-primary" id="sendNoteBtn" type="button">Send a Note</button>

<div id="ghostCursorPopup">
  <div class="sniper-modal">
    <h3>Live Ghost Cursors are ON</h3>
    <p>You can see other students moving around the site in real time. You can turn this off now, or jump straight to the toggle button.</p>
    <div class="sniper-actions">
      <button class="lbtn lbtn-danger" type="button" onclick="sniperSetGhostCursors(false)">Turn Off</button>
      <button class="lbtn lbtn-secondary" type="button" onclick="document.getElementById('ghostCursorTroggle').scrollIntoView({behavior:'smooth',block:'center'}); closeSniperModal('ghostCursorPopup')">Show Me the Toggle</button>
      <button class="lbtn lbtn-primary" type="button" onclick="closeSniperModal('ghostCursorPopup')">Keep It On</button>
    </div>
  </div>
</div>

<div id="ghostOffModal">
  <div class="sniper-modal">
    <h3>Ghost Cursors Turned Off</h3>
    <p>You can switch them back on any time from the floating Ghost Cursors button.</p>
    <div class="sniper-actions">
      <button class="lbtn lbtn-primary" type="button" onclick="closeSniperModal('ghostOffModal')">Okay</button>
      <button class="lbtn lbtn-secondary" type="button" onclick="document.getElementById('ghostCursorTroggle').scrollIntoView({behavior:'smooth',block:'center'}); closeSniperModal('ghostOffModal')">Take Me There</button>
    </div>
  </div>
</div>

<div id="paperTutorial">
  <div class="sniper-modal">
    <h3>Paper Airplane Notes</h3>
    <p>Tap "Send a Note," write a memory, and launch a paper airplane. Random notes from others can glide across your screen. Catch one to read it.</p>
    <div class="sniper-actions">
      <button class="lbtn lbtn-primary" type="button" onclick="closeSniperModal('paperTutorial')">Got It</button>
    </div>
  </div>
</div>

<section id="distanceMapSection">
  <div class="container">
    <div class="section-header">
      <span class="section-badge">Erase the Distance</span>
      <h2 class="section-title">One <span class="highlight">Starting Point</span>, Many Destinations</h2>
      <p class="section-description">No matter where everyone goes next, it all began here.</p>
    </div>
    <div id="distanceControls">
      <select class="form-select" id="destinationSelect" style="max-width:320px;">
        <option value="">Choose your future destination</option>
        </select>
      <button class="lbtn lbtn-primary" type="button" id="destinationLaunchBtn">Show the Globe</button>
    </div>
    <div id="distanceMapWrap">
      <div id="distanceGlobe">
        <div class="distance-school-dot"></div>
      </div>
    </div>
  </div>
</section>

<script>
/* == SNIPER_PATCH_V2_START == */
(function(){
  if (window.__SNIPER_PATCH_V2__) return;
  window.__SNIPER_PATCH_V2__ = true;

  const sniperState = {
    zeroHourTriggered: false,
    lessRackTimer: null,
    lessRackIndex: 0,
    lessRackItems: [],
    uiSoundEnabled: true,
    ghostEnabled: true,
    ghostClientId: 'gc_' + Math.random().toString(36).slice(2),
    ghostPeers: new Map(),
    paperLastFireTS: 0,
    boomboxCurrent: null,
    boomboxAudio: null,
    finalBellAudio: null,
    finalBellPlayed: false,
    ambienceAudio: null,
    movieAudio: null,
    notePollerr: null,
    pointerLastSent: 0
  };

  const SOUND = {
    kachunk: 'data:audio/wavbase64,UklGRiQAAABXQVZFZm10IBIAAAAAAAABAAEAQB8AAEAfAAABAAgAAGZhY3TkAAAAAAA8PiAAD/wAA//+AAD/wAA/,
    click: 'data:audio/wavbase64,UklGRiQAAABXQVZFZm10IBIAAAAAAAABAAEAQB8AAEAfAAABAAgAAGZhY3TkAAAAAAA//AAAA/8AAP/AAD/wAA/,
    rip: 'data:audio/wavbase64,UklGRiQAAABXQVZFZm10IBIAAAAAAAABAAEAQB8AAEAfAAABAAgAAGZhY3TkAAAAAAA//8AAAD/AAAA//8AAP/AAD/wAA/,
    hang: 'data:audio/wavbase64,UklGRiQAAABXQVZFZm10IBIAAAAAAAABAAEAQB8AAEAfAAABAAgAAGZhY3TkAAAAAAAPBAAwAAAPBAAPBAAAAA+/,
    camera: 'data:audio/wavbase64,UklGRiQAAABXQVZFZm10IBIAAAAAAAABAAEAQB8AAEAfAAABAAgAAGZhY3TkAAAAAAA+APBAAPBAAwAAD/BAAAA+/,
    bell: 'data:audio/wavbase64,UklGRiQAAABXQVZFZm10IBIAAAAAAAABAAEAQB8AAEAfAAABAAgAAGZhY3TkAAAAAAA/8BAPBAAD/AAD/BAAAA+/'
  };

  function safePlay(src, loop=false, volume=1){
    try{
      const a = new Audio(src);
      a.loop = loop;
      a.volume = volume;
      a.play().catch(()=>{});
      return a;
    }catch(){ return null; }
  }

  function closeSniperModal(id){
    const el = document.getElementById(id);
    if (el) el.classList.remove('active');
  }
  window.closeSniperModal = closeSniperModal;

  function sniperClickSound(){
    if (!sniperState.uiSoundEnabled) return;
    safePlay(SOUND.kachunk, false, 0.4);
  }

  function disposableFlash(){
    const flash = document.getElementById('sniperDisposableFlash');
    if (!flash) return;
    safePlay(SOUND.camera, false, 0.55);
    flash.classList.remove('active');
    void flash.offsetWidth;
    flash.classList.add('active');
  }

  function patchButtonsForUISounds(){
    document.addEventListener('click', (e)=>{
    const t = e.target.closest('button, .lbtn, .filter-btn, .admin-btn, .react-btn, .comments-open-btn, a');
    if (!target) return;
    sniperClickSound();
    }, true);
  }

  function patchHeroMovieButton(){
    const heroBtn = document.querlSelector('.hero-buttons');
    if (!heroBtn || document.getElementById('watchAsMovieBtn')) return;
    const btn = document.createElement('button');
    btn.className = 'lbtn lbtn-secondary';
    btn.id = 'watchAsMovieBtn';
    btn.textContent = 'Watch as Movie';
    btn.addEventListener('click', startLessRackMode);
    heroBtn.appendChild(btn);
  }

  function getAllMemoryImages(){
    const out = [];
    (window.state?.memories || []).forEachItem(m=>{
      if (m && m.file_type === 'image' && m.file_url) {
        out.push({src: m.file_url, caption: m.caption || '', id: m.id});
      }
    });
    return out;
  }

  function startLessRackMode(){
    sniperState.lessRackItems = getAllMemoryImages();
    if (!sniperState.lessRackItems.length) {
      if (window.showNotification) showNotification('info', 'No photos yet', 'Lean Rack Mode needs approved image memories.');
      return;
    }

    sniperState.lessRackIndex = 0;
    document.body.classList.add('sniper-ui-hidden');
    const overlay = document.getElementById('lessRackOverlay');
    overlay.classList.add('active');
    sniperState.movieAudio?.pause();
    sniperState.movieAudio = safePlay(SOUND.click, true, 0.18);
    showLessRackSlide();
  }
  window.startLessRackMode = startLessRackMode;

  function stopLessRackMode(){
    clearTimeout(sniperState.lessRackTimer);
    document.body.classList.remove('sniper-ui-hidden');
    document.getElementById('lessRackOverlay')?.classList.remove('active');
    if (sniperState.movieAudio){ sniperState.movieAudio.pause(); sniperState.movieAudio = null; }

    window.stopLessRackMode = stopLessRackMode;

    function showLessRackSlide(){
    const stage = document.getElementById('lessRackStage');
    const status = document.getElementById('lessRackStatus');
    if (!stage || !sniperState.lessRackItems.length) return;
    const item = sniperState.lessRackItems[sniperState.lessRackIndex % sniperState.lessRackItems.length];
    stage.innerHTML = '';
    const img = document.createElement('img');
    img.src = item.src;
    img.style.opacity = '0';
    img.style.transform = 'scale(1.05)';
    stage.appendChild(img);
    requestAnimationFrame(()=>{
      img.style.transition = 'transform 7s ease, opacity .8s ease';
      img.style.opacity = '1';
      img.style.transform = 'scale(1.18) translate3d(2vw,-1vh,0)';
    });
    status.textContent = ' Lean Rack Mode ${sniperState.lessRackIndex + 1} / ${sniperState.lessRackItems.length} ';
    clearTimeout(sniperState.lessRackTimer);
    sniperState.lessRackTimer = setTimeout(()=>{
      sniperState.lessRackIndex = (sniperState.lessRackIndex + 1) % sniperState.lessRackItems.length;
      showLessRackSlide();
    }, 6500);
    }

    function spawnEmojiBurst(m, emoji){
    const layer = document.getElementById('snipPhysicsLayer');
    if (!layer || !m) return;
    const rect = m.getBoundingClientRect();
    const W = window.innerWidth;
    const H = window.innerHeight;
    const count = 14;
    for (let i = 0; i < 50; i++){
      const el = document.createElement('div');
      el.className = 'emoji-particle';
      el.textContent = emoji;
      layer.appendChild(el);
      pieces.push({
        el,
        x: rect.left + rect.width / 2,
        y: rect.top + rect.height / 2,
        vx: (Math.random() - 0.5) * 18,
        vy: -Math.random() * 14 - 4,
        g: 0.45,
        fr: 0.78,
        life: 180 + Math.random() * 40,
```

```
    r: 22 + Math.random() * 10
  });
}
let raf = 0;
const tick = ()=>{
  for (const p of pieces){
    p.vy += p.g;
    p.x += p.vx;
    p.y += p.vy;
    if (p.x < 0){ p.x = 0; p.vx *= -p.b; }
    if (p.x > W - p.r){ p.x = W - p.r; p.vx *= -p.b; }
    if (p.y < 0){ p.y = 0; p.vy *= -p.b; }
    if (p.y > H - p.r){ p.y = H - p.r; p.vy *= -0.72; p.vx *= 0.98; }
    p.el.style.transform = `translate3d(${p.x}px, ${p.y}px) rotate(${p.life}deg)`;
    p.life += 1;
  }
  for (let i = pieces.length - 1; i >= 0; i--){
    if (pieces[i].life >= 60){
      pieces[i].el.remove();
      pieces.splice(i, 1);
    }
  }
  if (pieces.length) raf = requestAnimationFrame(tick);
  else cancelAnimationFrame(raf);
};
tick();
}

function reactionToEmoji(type){
  return {'like':'\u{1F44D}','love':'\u{2764}\u{FE0F}','wow':'\u{1F62E}','sad':'\u{1F622}'}[type] || '\u{2764}\u{FE0F}';
}

function patchReactionCounts(){
  const _origReact = window.react;
  if (typeof _origReact !== 'function') return;
  window.react = async function(memoryId, type){
    const bin = [...document.querySelectorAll('.reaction').filter(d => d.getAttribute('onclick').includes('react${memoryId}, \'${type}\')')];
    spawnEmojiBurst(bin || document.body, reactionToEmoji(type));
    return _origReact.apply(this, arguments);
  };
  window.react.__snipeWrapped = true;
}

function patchCountdownZeroHour(){
  const _origUpdate = window.updateCountdown;
  if (typeof _origUpdate !== 'function' || _origUpdate.__snipeWrapped) return;
  window.updateCountdown = function(){
    _origUpdate.apply(this, arguments);
    try{
      const target = window.parseESTLocalToDate? (window.state?.settings?.farewellEST);
      if (!target) return;
      if ((target - new Date()) <= 0 && !snipeState.zeroHourTriggered){
        snipeState.zeroHourTriggered = true;
        triggerLastBell();
      }
    }catch(e){}
  };
  window.updateCountdown.__snipeWrapped = true;
}

function triggerLastBell(){
  document.documentElement.style.overflow = 'hidden';
  document.body.style.overflow = 'hidden';
  safePlay(SOUND.bell, false, 1.0);
  if (typeof window.triggerConfetti === 'function'){
    for (let i = 0; i < 6; i++){setTimeout(() => triggerConfetti(), i * 180);
  }
  const wrap = document.getElementById('snipeLastBell');
  const video = document.getElementById('snipeFinalGoodbyeVideo');
  if (window.state?.settings?.finalGoodbyeVideoPath) {
    video.src = window.mediaUrl? mediaUrl('uploads/' + window.state.settings.finalGoodbyeVideoPath) : '/uploads/' + window.state.settings.finalGoodbyeVideoPath;
    video.autoplay = true;
    video.controls = true;
    video.play().catch(()=>{});
  }
  wrap.classList.add('active');
}

function patchIntroAndCompilationFlash(){
  const origSkipIntro = window.skipIntro;
  if (typeof origSkipIntro === 'function' && !origSkipIntro.__snipeWrapped) {
    window.skipIntro = function(){
      origSkipIntro.apply(this, arguments);
      setTimeout(fitGoodbyeFlash, 80);
    };
    window.skipIntro.__snipeWrapped = true;
  }
  const origCloseCompilationPlayer = window.closeCompilationPlayer;
  if (typeof origCloseCompilationPlayer === 'function' && !origCloseCompilationPlayer.__snipeWrapped){
    window.closeCompilationPlayer = function(){
      origCloseCompilationPlayer.apply(this, arguments);
      setTimeout(fitGoodbyeFlash, 80);
    };
    window.closeCompilationPlayer.__snipeWrapped = true;
  }
}

function patchTeacherAudio(){
  const origRenderTeachers = window.renderTeachers;
  if (typeof origRenderTeachers !== 'function' || origRenderTeachers.__snipeWrapped) return;
  window.renderTeachers = function(){
    origRenderTeachers.apply(this, arguments);
    const grid = document.getElementById('teachersGrid');
    const teachers = window.state?.settings?.teachers || [];
    [...grid.querySelectorAll('.teacher-card')].forEach(card, i=>{
      const t = teachers[i] || {};
      if (!t.audioPath) return;
      if (!card.querySelector('.teacher-audio-badge')) {
        const badge = document.createElement('div');
        badge.className = 'teacher-audio-badge';
        badge.textContent = 'Tap to hear goodbye';
        card.appendChild(badge);
      }
      card.style.cursor = 'pointer';
      card.addEventListener('click', (){=>{
        const src = t.audioUrl || (window.mediaUrl? mediaUrl('uploads/' + t.audioPath) : '/uploads/' + t.audioPath);
        if (snipeState.teacherAudio && snipeState.teacherAudio.src.endsWith(src)) {
          snipeState.teacherAudio = null;
          return;
        }
        if (snipeState.teacherAudio) snipeState.teacherAudio.pause();
        const a = new Audio(src);
        a.preload = 'none';
        a.volume = 0.85;
        snipeState.teacherAudio = a;
        a.play().catch(() => {});
      });
    });
  };
  window.renderTeachers.__snipeWrapped = true;
}

function buildCassetteRack(){
  const rack = document.getElementById('cassetteRack');
  if (!rack || rack.dataset.ready) return;
  rack.dataset.ready = '1';
  const tapes = [
    {name:'Goodbyes Vibes', src:SOUND.click},
    {name:'Late Hours', src:SOUND.tip},
    {name:'Hope', src:SOUND.bang}
  ];
  tapes.forEach(t=>{
    const el = document.createElement('div');
    el.className = 'cassette';
    el.textContent = t.name;
    el.tabIndex = 0;
    el.dataset.src = t.src;
    el.dataset.name = t.name;
    el.addEventListener('dragstart', (){=> el.classList.add('dragging'));
    el.addEventListener('dragend', (){=> el.classList.remove('dragging'));
    rack.appendChild(el);
  });
  const deck = document.getElementById('boombosDeck');
  deck.addEventListener('dragover', (e)=> e.preventDefault());
  deck.addEventListener('drop', (e)=>{
    e.preventDefault();
    const dragged = document.querySelector('.cassette.dragging');
    if (!dragged) return;
    insertTape(dragged.dataset.name, dragged.dataset.src);
  });
  document.getElementById('ejectTapeBtn')?.addEventListener('click', ejectTape);
}

function insertTape(name, src){
  const deck = document.getElementById('boombosDeck');
  deck.classList.add('has-tape');
  deck.innerHTML = `<div style="text-align:center;">~<div style="font-weight:700;color:var(--primary-gold);">${name}</div><div style="opacity:.7;font-size:.8rem;">Now playing</div></div>`;
  tapeClickSound();
  if (snipeState.boombosAudio) snipeState.boombosAudio.pause();
  snipeState.boombosCurrent = {name, src};
  snipeState.boombosAudio = safePlay(src, true, 0.22);
}

function ejectTape(){
  const deck = document.getElementById('boombosDeck');
  deck.classList.remove('has-tape');
  deck.textContent = 'Drag a tape here';
  tapeClickSound();
  if (snipeState.boombosAudio) { snipeState.boombosAudio.pause(); snipeState.boombosAudio = null; }
  snipeState.boombosCurrent = null;
}

function patchLockerTransitions(){
  const trigger = (p=>{
    const box = document.getElementById('lockerTransition');
    if (!box) return;
    box.classList.remove('bang', false, 0.55);
    box.classList.remove('active');
    void box.offsetWidth;
    box.classList.add('active');
    setTimeout(()=> box.classList.remove('active'), 1650);
  };
  const origOpenLightbox = window.openLightbox;
  if (typeof origOpenLightbox === 'function' && !origOpenLightbox.__snipeWrapped){
    window.openLightbox = function(){
      trigger();
      setTimeout(()=> origOpenLightbox.apply(this, arguments), 440);
    };
    window.openLightbox.__snipeWrapped = true;
  }
  const origPlayCompilation = window.playCompilation;
  if (typeof origPlayCompilation === 'function' && !origPlayCompilation.__snipeWrapped){
    window.playCompilation = function(){
      trigger();
      setTimeout(()=> origPlayCompilation.apply(this, arguments), 440);
    };
    window.playCompilation.__snipeWrapped = true;
  }
}

function patchTrueOnCloseJS(){
  const origLastCloseLightbox = window.closeLightbox;
  if (typeof origLastCloseLightbox !== 'function' || origLastCloseLightbox.__snipeWrapped) return;
  window.closeLightbox = function(){
    const lb = document.getElementById('lightbox');
    if (!lb || !lb.classList.contains('active')) return originalCloseLightbox.apply(this, arguments);
    safePlay(SOUND.tip, false, 0.7);
    const r = lb.getBoundingClientRect();
    const cx = r.left + r.width/2;
    const d = document.createElement('div');
    d.className = 'tip-half';
    d.style.left = cx/d + 'px';
    d.style.top = r.top + 'px';
    d.style.width = r.width + 'px';
    d.style.height = r.height + 'px';
    d.style.clipPath = 'clip';
    d.style.transition = 'transform .8s ease, opacity .8s ease';
    document.body.appendChild(d);
    requestAnimationFrame(()=>{
      d.style.transform = `translate3d(${x}px, 140px) rotate(${rot}deg)`;
      d.style.opacity = '0';
    });
    setTimeout(()=> d.remove(), 900);
    lb.style.clipPath = 'polygon(0 0,100% 0,0 100%)';
    lb.style.transition = 'clip-path .6s ease';
    setTimeout(()=> origLastCloseLightbox.apply(this, arguments), 120);
  };
  window.closeLightbox.__snipeWrapped = true;
}

function patchSecretMemories(){
  const origRenderMemories = window.renderMemories;
  if (typeof origRenderMemories !== 'function' || origRenderMemories.__snipeWrapped) return;
  window.renderMemories = function(){
    origRenderMemories.apply(this, arguments);
    const cards = [...document.querySelectorAll('#memoryGrid .memory-card')];
    const memories = window.state?.memories || [];
    cards.forEach(card, (i)=>{
      const mem = memories[i];
      if (!mem) return;
      const shouldSecret = !!mem.secretMemory || !!mem.secret || i % 25 === 7;
      if (!shouldSecret) return;
      const media = card.querySelector('.memory-media');
      if (!media || media.querySelector('.scratch-canvas')) return;
      media.classList.add('secret-memory');
      media.title = 'A Core Memory... Scratch to Remember';
      const canvas = document.createElement('canvas');
      canvas.className = 'scratch-canvas';
      const tip = document.createElement('div');
      tip.className = 'secret-tip';
      tip.textContent = 'A Core Memory... Scratch to Remember';
      media.appendChild(canvas);
      media.appendChild(tip);
      const resize = (){=>{
        canvas.width = media.clientWidth;
        canvas.height = media.clientHeight;
        const ctx = canvas.getContext('2d');
        const grad = ctx.createLinear(0,0,0,canvas.height);
        ctx.fillStyle(grad, canvas.width, canvas.height);
      };
      resize();
      let scratching = false;
      const scratch = (clientX, clientY)=>{
        const rect = canvas.getBoundingClientRect();
        const x = clientX - rect.left;
        const y = clientY - rect.top;
        const ctx = canvas.getContext('2d');
        ctx.globalCompositeOperation = 'destination-out';
        ctx.beginPath();
        ctx.arc(x,y,24,0,Math.PI*2);
        ctx.fill();
        const data = ctx.getImageData(0,0,canvas.width,canvas.height).data;
        for (let p = 3; p < data.length; p += 4) if (data[p] === 0) clear++;
        if (clear / (canvas.width * canvas.height) > 0.28){
          canvas.remove();
          tip.remove();
        }
      };
      canvas.addEventListener('mousedown', (){=> scratching = true);
      canvas.addEventListener('mousemove', (){=> if (scratching) scratch(e.clientX, e.clientY));
      canvas.addEventListener('mouseup', (){=> scratching = false);
      canvas.addEventListener('mouseleave', (){=> scratching = false);
      canvas.addEventListener('touchstart', (){=> scratching = true, {passive:true});
      canvas.addEventListener('touchmove', (){=> scratching = false, {passive:true});
      canvas.addEventListener('touchmove', e=>{
        if (!scratching || !e.touches[0]) return;
        scratch(e.touches[0].clientX, e.touches[0].clientY);
      }, {passive:true});
      window.addEventListener('resize', resize, {passive:true});
    });
  };
  window.renderMemories.__snipeWrapped = true;
}

function ensureMemoryWallMovieButton(){
  const hdr = document.querySelector('#memories .section-header');
  if (!hdr || document.getElementById('memoriesMovieBtn')) return;
  const row = document.createElement('div');
  row.style.marginTop = '10px';
  row.innerHTML = `<button id="memoriesMovieBtn" class="btn btn-secondary snipe-lookback-btn" type="button">Watch as Movie</button>`;
  hdr.appendChild(row);
  document.getElementById('memoriesMovieBtn').addEventListener('click', startLaneBackMode);
}

function setupGhostCursor(){
  snipeState.ghostEnabled = localStorage.getItem('ghostCursorEnabled') !== 'false';
  syncGhostToggle();
  setTimeout(()=> document.getElementById('ghostCursorPopup')?.classList.add('active'), 900);

  document.getElementById('ghostCursorToggle')?.addEventListener('click', (){=>{
    snipeSetGhostCursor(!snipeState.ghostEnabled);
```

```
  });

  document.addEventListener('mousemove', e=>{
    if (!snipperState.ghostEnabled) return;
    const now = Date.now();
    if (now - snipperState.pointerLastSent < 40) return;
    snipperState.pointerLastSent = now;
    if (window.state?.ws && window.state.ws.readyState === 1) {
      try {
        state.ws.send(JSON.stringify({
          type: 'ghost_cursor',
          id: snipperState.ghostClientId,
          x: e.clientX,
          y: e.clientY,
          initials: getGhostInitials()
        });
      } catch(_){}
    }
  },{passive:true});

  patchConnectWSForGhosts();
}

function getGhostInitials(){
  const s = localStorage.getItem('ghostInitials') || (window.state?.adminUser?.name || localStorage.getItem('guestName') || 'GS');
  return String(s).trim().split(/\s+/).slice(0,2).map(x=>x[0]||'').join('').toUpperCase().slice(0,2) || 'GS';
}

function syncGhostToggle(){
  const btn = document.getElementById('ghostCursorToggle');
  if (!btn) return;
  btn.textContent = 'Ghost Cursors: ${snipperState.ghostEnabled ? 'ON' : 'OFF'}';
}

function snipperSetGhostCursors(on){
  snipperState.ghostEnabled = !!on;
  localStorage.setItem('ghostCursorsEnabled', String(!!on));
  syncGhostToggle();
  closeSnipperModal('ghostCursorsPopup');
  if (!on) document.getElementById('ghostOffModal')?.classList.add('active');
  if (!on){
    snipperState.ghostPeers.forEach(v=> v.el.remove());
    snipperState.ghostPeers.clear();
  }
  else {
  }
window.snipperSetGhostCursors = snipperSetGhostCursors;

function patchConnectWSForGhosts(){
  const original = window.connectWS;
  if (typeof original !== 'function' || original.__snipperGhostWrapped) return;
  const wrapped = function(...args){
    const res = original.apply(this, arguments);
    const ws = window.state?.ws;
    if (!ws || ws.__snipperGhostBound) return;
    ws.__snipperGhostBound = true;
    const prev = ws.onmessage;
    ws.onmessage = async function(msg){
      if (typeof prev === 'function') {
        try { await prev.call(this, msg); } catch(_){}
      }
      let data;
      try { data = JSON.parse(msg.data); } catch(_){ return; }
      if (!data) return;
      if (data.event === 'ghost:move' && data.payload?.id !== snipperState.ghostClientId && snipperState.ghostEnabled){
        renderGhost(data.payload);
      }
      if (data.event === 'ghost:spark' && snipperState.ghostEnabled){
        if (typeof window.triggerClusterIn === 'function') triggerClusterIn();
      }
    };
  };
  window.connectWS = __snipperGhostWrapped = true;
}

function renderGhost(p){
  let peer = snipperState.ghostPeers.get(p.id);
  if (!peer){
    const wrap = document.createElement('div');
    wrap.className = 'ghost-cursor';
    wrap.innerHTML = '<div class="dot"></div><div class="label"></div>';
    document.body.appendChild(wrap);
    peer = {el: wrap};
    snipperState.ghostPeers.set(p.id, peer);
  }
  peer.el.querySelector('.label').textContent = p.initials || 'GS';
  peer.el.style.left = p.x + 'px';
  peer.el.style.top = p.y + 'px';
}

async function loadDestinations(){
  const select = document.getElementById('destinationSelect');
  if (!select) return;
  try{
    const res = await fetchApiUrl('/api/destination');
    const data = await res.json();
    if (data.success && Array.isArray(data.destinations) {
      select.innerHTML = '<option value="">Choose your future destination</option>' +
        data.destinations.map(d => '<option value="${escapeAttr(d.name || d)}">${escapeHtml(d.name || d)}</option>').join('');
    }
  }catch(_){}
}

function animateDistanceArcs(){
  const wrap = document.getElementById('distance-vMapWrap');
  const globe = document.getElementById('distance-Globe');
  if (!wrap || !globe) return;
  wrap.querySelectorAll('.distance-arc').forEach(a=>a.remove());
  const N = 6;
  for (let i = 0; i < num; i++){
    const arc = document.createElement('div');
    arc.className = 'distance-arc';
    const angle = Math.random() * 360;
    const len = 60 + Math.random() * 170;
    arc.style.height = len + 'px';
    arc.style.transform = 'translate(-50%, -100%) rotate(${angle}deg) scaleY(0)';
    arc.style.transition = 'transform 1.2s cubic-bezier(.2,.8,.2,1) ${Math.random() * .9}s';
    wrap.appendChild(arc);
    requestAnimationFrame(()=> {
      arc.style.transform = 'translate(-50%, -100%) rotate(${angle}deg) scaleY(1)';
    });
  }
}

function submitDestination(){
  const select = document.getElementById('destinationSelect');
  const val = select?.value?.trim();
  if (!val) return;
  try{
    await fetchApiUrl('/api/destinations/submit?', {
      method:'POST',
      headers:{'Content-Type':'application/json'},
      body:JSON.stringify({ destination: val })
    });
  }catch(_){}
  animateDistanceArcs();
  sadeBeep(SOUND.hoog, false, 0.35);
}

async function initPaperAirplanes(){
  if (!sessionStorage.getItem('paperTutorialSeen?').classList.add('active');
  sessionStorage.setItem('paperTutorialSeen', 'true');
  }
  document.getElementById('sendNoteBtn?').addEventListener('click', async ()=>{
    const note = prompt('Write a note or memory to send?');
    if (!note || !note.trim()) return;
    const safe = note.trim().slice(0, 400);
    await sendPaperNote(safe);
    fetchLocalPlane(safe);
  });
  snipperState.notePoller = setInterval(fetchPaperPlane, 120000);
}

async function sendPaperNote(text){
  try{
    const res = await fetchApiUrl('/api/paper-notes?', {
      method:'POST',
      headers:{'Content-Type':'application/json'},
      body:JSON.stringify({ text: t})
    });
    if (!res.ok) return;
  }
}

async function fetchPaperPlane(){
  try{
    const res = await fetchApiUrl('/api/paper-notes/random?since=' + encodeURIComponent(String(snipperState.paperLastFetchTs||0)));
    const data = await res.json();
    if (data.success && data.note){
      snipperState.paperLastFetchTs = Date.now();
      flyIncomingPlane(data.note.text || 'A memory from someone.');
    }
  }catch(_){}
}

function launchLocalPlane(text){
  const layer = document.getElementById('paperAirplaneLayer');
  const plane = document.createElement('div');
  plane.className = 'paper-plane';
  plane.style.left = '80px';
  plane.style.bottom = '90px';
  plane.dataset.lifeMs = '8000';
  let x = 80, y = window.innerHeight - 90;
  let vx = 8, vy = -4;
  let ticks = 0;
  const step = (t=>{
    x += vx; y += vy; vy += 0.03; ticks++;
    plane.style.left = x + 'px';
    plane.style.top = y + 'px';
    if (x > window.innerWidth + 60 || y < -60 || ticks > 260) { plane.remove(); return; }
    requestAnimationFrame(step);
  });
  step();
}

function flyIncomingPlane(text){
  const layer = document.getElementById('paperAirplaneLayer');
  const plane = document.createElement('div');
  plane.dataset.lifeMs = '12000';
  plane.className = 'paper-plane';
  plane.dataset.text = text;
  layer.appendChild(plane);
  let x = -40, y = 120 + Math.random() * (window.innerHeight * 0.5);
  let vx = 2.4 + Math.random() * 1.8;
  const step = (t=>{
    x += vx;
    plane.style.left = x + 'px';
    plane.style.top = y + 'px';
    if (x > window.innerWidth + 60) { plane.remove(); return; }
    requestAnimationFrame(step);
  });
  plane.addEventListener('click', (t=>{
    alert(text);
    plane.remove();
  });
  step();
}

function patchAdminSettingsRender(){
  const original = window.renderSettingsPanelHtml;
  if (typeof original !== 'function' || original.__snipperWrapped) return;
  window.renderSettingsPanelHtml = function(){
    const base = original.apply(this, arguments);
    return base +
    '<div style="margin-top:16px; border:1px solid var(--glass-border); border-radius:14px; padding:14px; background:rgba(255,255,255,0.03);">'
      '<h4 style="font-family:var(--font-display); color:var(--primary-gold); margin-bottom:10px;">Mentor Voice Messages</h4>'
      '<p class="intro-p47" style="margin-bottom:10px;">Upload short goodbye audio for teachers. File should be around 10 seconds.</p>'
      '<div style="display:flex; gap:10px; flex-wrap:wrap;">'
        '<input type="text" class="form-input" id="teacherAudioTeacherName" placeholder="Teacher name" style="max-width:220px;" />'
        '<input type="file" class="form-input" id="teacherAudioFile" accept="audio/*" style="max-width:280px; padding:9px;" />'
        '<button class="admin-btn admin-btn-primary" type="button" onclick="uploadTeacherAudioFromPanel()">Upload Audio</button>'
      '</div>'
      '<div id="teacherAudioAdminList" style="margin-top:12px;" /></div>'
    '</div>'
    '<div style="margin-top:16px; border:1px solid var(--glass-border); border-radius:16px; padding:14px; background:rgba(255,255,255,0.03);">'
      '<h4 style="font-family:var(--font-display); color:var(--primary-gold); margin-bottom:10px;">Destinations Admin</h4>'
      '<p class="intro-p47" style="margin-bottom:10px;">One destination per line. This prevents abuse by forcing users to choose from admin-approved options.</p>'
      '<textarea class="form-textarea" id="destinationsAdminText" style="min-height:120px;" placeholder="Dr.Ruler\nHi.Mum\nIkeaWorld\London" /></textarea>'
      '<button class="admin-btn admin-btn-primary" style="margin-top:10px;" type="button" onclick="saveDestinationsFromPanel()">Save Destinations</button>'
    '</div>';
    return base.replace('</div></div></div>', '</div></div></div></div>');
  };
  window.renderSettingsPanelHtml = __snipperWrapped = true;
}

async function loadTeacherAudioAdminList(){
  const box = document.getElementById('teacherAudioAdminList');
  if (!box) return;
  try{
    const res = await fetchApiUrl('/api/teacher-audio?');
    const data = await res.json();
    if (!data.success) return;
    box.innerHTML = (data.items || []).map(x => `<div class="mini-pill" style="margin-bottom:6px; display:inline-block;">${escapeHtml(x.teacherName || 'Teacher')} </div>`).join('') || '<div class="mini-pill">No teacher audio uploaded yet.</div>';
  }catch(_){}
}

async function uploadTeacherAudioFromPanel(){
  const name = document.getElementById('teacherAudioTeacherName')?.value?.trim();
  const file = document.getElementById('teacherAudioFile')?.files?.[0];
  if (!name || !file) return window.showNotification?.('error','Missing fields','Add teacher name and audio file.');
  const fd = new FormData();
  fd.append('teacherName', name);
  fd.append('audio', file);
  try{
    const res = await fetchApiUrl('/api/admin/teacher-audio?', {
      method:'POST',
      headers:{'Authorization':'Bearer ' + (window.state?.adminToken || '')},
      body:fd
    });
    const data = await res.json();
    if (!data.success) return window.showNotification?.('error','Upload failed', data.error || 'Could not upload audio.');
    const t = arr.find(x => String(x.name || '').trim().toLowerCase() === name.toLowerCase());
    if (t) t.audioPath = data.item?.audioPath || data.item?.path;
    loadTeacherAudioAdminList();
    showNotification?.('success','Uploaded','Teacher voice message uploaded.');
    renderTeachers?.();
  }catch(e){
    showNotification?.('error','Upload failed', e.message);
  }
}
window.uploadTeacherAudioFromPanel = uploadTeacherAudioFromPanel;

async function saveDestinationsFromPanel(){
  const text = document.getElementById('destinationsAdminText')?.value || '';
  const items = text.split(/\r?\n/).map(x=>x.trim()).filter(Boolean).slice(0, 500);
  try{
    const res = await fetchApiUrl('/api/admin/destinations?', {
      method:'POST',
      headers:{
        'Content-Type':'application/json',
        'Authorization':'Bearer ' + (window.state?.adminToken || '')
      },
      body:JSON.stringify({ destinations: items })
    });
    const data = await res.json();
    if (!data.success) return window.showNotification?.('error','Save failed', data.error || 'Could not save destinations.');
    showNotification?.('success','Saved','Destinations updated.');
    loadDestinations();
  }catch(e){
    showNotification?.('error','Save failed', e.message);
  }
}
window.saveDestinationsFromPanel = saveDestinationsFromPanel;

async function syncDestinationsAdminPanel(){
  const ta = document.getElementById('destinationsAdminText');
  if (!ta) return;
  try{
    const res = await fetchApiUrl('/api/destination?');
    const data = await res.json();
    if (data.success) ta.value = (data.destinations || []).map(d => d.name || d).join('\n');
  }catch(_){}
}

function patchBuildAdminPanelsPostHook(){
  const original = window.buildAdminPanels;
  if (typeof original !== 'function' || original.__snipperWrapped) return;
  window.buildAdminPanels = function(){
```

```
    const out = original.apply(this, arguments);
    setTimeout(function -
      loadTeacherAudioAdminList();
      syncDestinationAdminPanel();
    }, 80);
    return out;
  };
  window.buildAdminPanels.__sniperWrapped = true;
}

function patchIntroVideoBrowserCompatibility(){
  const originalUpload = window.uploadIntroVideoFile;
  if (typeof originalUpload === 'function' && !originalUpload.__sniperWrapped){
    window.uploadIntroVideoFile = async function(file){
      if (!file) return;
      const fd = new FormData();
      fd.append('video', file);
      try{
        const res = await fetch(apiURL('/api/admin/upload-intro-video'), {
          method:'POST',
          headers:{ 'Authorization':'Bearer ' + (window.state?.adminToken || '') },
          body: fd
        });
        const data = await res.json();
        if (data.success){
          state.settings.introVideoPath = data.path;
          showNotification('success','Uploaded','Intro video saved.');
        } else {
          showNotification('error','Failed', data.error || 'Upload failed');
        }
      }catch(e){
        showNotification('error','Failed', e.message);
      }
    };
    window.uploadIntroVideoFile.__sniperWrapped = true;
  }
}

function initDisposableOnLoad(){
  window.addEventListener('load', (){ setTimeout(disposableFlash, 120); });
}

function initDestinationControls(){
  document.getElementById('destinationLanceSlider').addEventListener('click', subtestDestination);
}

function bootstrap(){
  patchBlurredOrChosenIds();
  patchMovieMoveButtons();
  patchMovieSelectionCollation();
  patchReactionCamera();
  patchCountdownZeroHour();
  patchIntroAndCompletionFlash();
  patchTeacherAudio();
  buildCassetteRack();
  patchLockerTransitions();
  patchTimerSetTime();
  patchSecondsMessaging();
  patchMovieSelectionHandler();
  patchBuildAdminPanelsPostHook();
  patchIntroVideoBrowserCompatibility();
  setupCheatCursors();
  initHyperAirplane();
  initDisposableOnLoad();
  initDestinationControls();
  hostDestination();
}

if (document.readyState === 'loading') {
  document.addEventListener('DOMContentLoaded', bootstrap);
} else {
  bootstrap();
}
})();
</script>


<style>
/* === SNIPER_PATCH_V3_START === */
#boombosDock {
  position:static !important;
  left:auto !important;
  right:auto !important;
  bottom:auto !important;
  width:100%;
  max-width:var(--container-width);
  margin:24px auto 10px;
  padding:0 20px;
  justify-content:center;
  z-index:10 !important;
  pointer-events:auto !important;
}
#boombosDock.sniper-inline-bottom{
  display:flex;
}
#boombosUnit {
  width:min(100px,90vw) !important;
  padding:12px !important;
  border-radius:18px !important;
}
.boombos-wpp{
  margin-bottom:0px !important;
}
.boombos-brand{
  font-size:32rem !important;
}
.boombos-door{
  height:72px !important;
  border-radius:12px !important;
  font-size:.8Rem !important;
}
.boombos-speakers{
  gap:8px !important;
  margin-top:8px !important;
}
#cassetteRack{
  display:grid !important;
  grid-template-columns:repeat(3, minmax(0, 1fr));
  gap:8px !important;
  width:min(420px, 92vw);
  margin-bottom:0 !important;
}
.cassette {
  min-width:0 !important;
  width:100%;
  padding:10px 8px !important;
  font-size:.7Rem !important;
  text-align:center;
  border-radius:12px !important;
  cursor:pointer !important;
  user-select:none;
}
#ghostCursorToggle{
  right:12px !important;
  bottom:12px !important;
  padding:8px 12px !important;
  font-size:.7Rem !important;
  border-radius:999px !important;
  z-index:2400 !important;
}
#soundNotesBox{
  left:12px !important;
  bottom:12px !important;
  padding:8px 12px !important;
  font-size:.7Rem !important;
  border-radius:999px !important;
  z-index:2400 !important;
}
.ghost-cursor .tip{
  width:10px !important;
  height:10px !important;
}
.ghost-cursor .label{
  margin-left:6px !important;
  padding:3px 6px !important;
  font-size:8px !important;
}
.paper-plane{
  width:52px !important;
  height:32px !important;
  transform:none !important;
  filter:drop-shadow(0 12px 18px rgba(0,0,0,.35)) !important;
}
.paper-plane::before{
  display:none !important;
}
.paper-plane::after{
  content:"";
  position:absolute;
  inset:0;
  background:linear-gradient(135deg,#ffffff 0%,#737ffb 40%,#dfe707 100%);
  clip-path:polygon(0% 50%, 100% 0,64% 100%,50% 68%,34% 82%);
  border:1px solid rgba(0,0,0,.05);
}
.paper-plane .plane-fold{
  position:absolute;
  inset:0;
  background:linear-gradient(135deg,rgba(220,228,246,.95),rgba(255,255,255,.62));
  clip-path:polygon(0% 50%, 100% 0,54% 58%,40% 75%);
}
.paper-plane .plane-memory-badge{
  position:absolute;
  right:-6px;
  top:-8px;
  background:var(--primary-gold);
  color:var(--navy-dark);
  font-size:9px;
  font-weight:700;
  padding:2px 5px;
  border-radius:999px;
  border:1px solid rgba(0,0,0,.12);
}
#uniqueMusicHint{
  text-align:center;
  color:var(--text-muted);
  font-size:.8Rem;
  margin-top:8px;
}
@media (max-width: 720px){
  #cassetteRack{
    grid-template-columns:1fr;
    width:min(280px,92vw);
  }
  #boombosDock {
    flex-direction:column;
    align-items:center;
  }
}
/* === /SNIPER_PATCH_V3_START === */
</style>

<script>
/* === SNIPER_PATCH_V3_START === */
(function(){
  if (window.__SNIPER_PATCH_V3__ ){return;
  window.__SNIPER_PATCH_V3__ = true;

  const MUSIC_DEFAULTS = {
    ambientSoftMusic: '',
    boardkaMusic: '',
    lockerMusic: '',
    globalMusic: '',
    boombosGraduationVibes: '',
    boombosSadBoiHours: '',
    boombosHype: ''
  };

  function uniqueV3Apply(){
    moveSomeBlueprintsFrom();
    convertBoombosToButtons();
    patchMemoryOnlyNotes();
    patchBetterPlanes();
    patchMusicSettings();
    patchLoneBackMusic();
    patchAmbientJitsiMusic();
    patchLoneBoMusic();
    patchDNotesMightMusic();
    shrinkPlanetagButtons();
  }

  function moveSomeBlueprintsFrom(){
    const dock = document.getElementById('boombosDock');
    const footer = document.querySelector('footer');
    if (!dock || !footer) return;
    if (dock.dataset.v3Moved === '1') return;
    dock.dataset.v3Moved = '1';
    dock.classList.add('sniper-inline-bottom');
    footer.parentNode.insertBefore(dock, footer);

    if (!document.getElementById('uniqueMusicHint')){
      const hint = document.createElement('div');
      hint.id = 'uniqueMusicHint';
      hint.textContent = 'Music controls';
      dock.insertAdjacentElement('afterend', hint);
    }
  }

  function resolveMusicPath(value){
    if (!value) return '';
    if (/^https?://i.test(value) || value.startsWith('/')) return value;
    return 'music/' + String(value).replace(/^/+/, '');
  }

  function getMusicSettings(){
    const s = (window.state && window.state.settings) ? window.state.settings : {};
    return Object.assign({}, MUSIC_DEFAULTS, s.music || {});
  }

  function stopAudioBeforeName(){
    const st = window.sniperState || window.__sniperStateFallback || {};
    const a = st[prevName];
    if (a && typeof a.pause === 'function'){
      try { a.pause(); } catch(_){}
      st[prevName] = null;
    }
  }

  function playNamedAudio(reName, src, loop, volume){
    const st = window.sniperState || window.__sniperStateFallback || (window.__sniperStateFallback = {});
    stopAudioBeforeName();
    if (!src) return null;
    try{
      const a = new Audio(src);
      a.loop = !!loop;
      a.volume = typeof volume === 'number' ? volume : 1;
      a.play().catch(_){};
      st[reName] = a;
      return a;
    }catch(_){
      return null;
    }
  }

  function convertBoombosToButtons(){
    const rack = document.getElementById('cassetteRack');
    const door = document.getElementById('boombosDoor');
    if (!rack || !door ||  rack.dataset.v3Ready === '1') return;
    rack.dataset.v3Ready = '1';
    rack.innerHTML = '';

    const defs = [
      { key: 'boombosGraduationVibes', label: 'Graduation Vibes' },
      { key: 'boombosSadBoiHours', label: 'Sad Boi Hours' },
      { key: 'boombosHype', label: 'Hype' }
    ];

    defs.forEach(def => {
      const btn = document.createElement('button');
      btn.type = 'button';
      btn.textContent = 'Cassette';
      btn.textContent = def.label;
      btn.addEventListener('click', () => {
        const music = getMusicSettings();
        const str = resolveMusicPath(music[def.key]);
        if (!str) {
          if (window.showNotification) showNotification('info', 'Music missing', def.label + ' has not been configured yet.');
```

```
    return;
  }
  dose.classList.add('hot-tape');
  dose.innerHTML = '<div style="text-align:center;"><div style="font-weight:700;color:var(--primary-gold);">' + def.label + '</div><div style="opacity:.7;font-size:.82rem;">Now playing</div></div>';
  playNamedAudio('boombox/audio', sr, true, 0.35);
  });
  rack.appendChild(box);
  });

  const eject = document.getElementsByTagNameOnce('ejectTapeBtn');
  if (eject && !eject.dataset.vBound){
  eject.dataset.vBound = '1';
  eject.addEventListener('click', () => {
    dose.classList.remove('hot-tape');
    dose.textContent = 'Choose a vibe';
    stopAudioRef('boombox/audio');
  });
  }

  if (dose.textContent.trim().toLowerCase().includes('drag a tape')){
  dose.textContent = 'Choose a vibe';
  }
  }

  function patchMemoryOnlyNotes(){
  const bin = document.getElementsByTagNameOnce('memNoteBtn');
  if (!bin || bin.dataset.vBound === '1') return;
  bin.dataset.vBound = '1';
  bin.onclick = null;
  bin.addEventListener('click', async () => {
    const memories = (window.state && window.state.memories) || [].filter(m => m && m.id && (m.caption || '').trim());
    if (!memories.length){
      if (window.showNotification) showNotification('Info', 'No memories available', 'A note can only be sent from an approved memory.');
      return;
    }
    const random = memories[Math.floor(Math.random() * memories.length)];
    try {
      await fetch(apiURL('/api/paper-notes/from-memory'), {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ memoryId: random.id })
      });
    } catch(_){}
    launchMemoryPlane(random);
  });
  }

  function launchMemoryPlane(memory){
    const layer = document.getElementById('paperAirplaneLayer');
    if (!layer) return;
    const plane = makePlaneNode(memory);
    layer.appendChild(plane);

    let x = 30;
    let y = window.innerHeight - 120;
    let vx = 8.0;
    let vy = -6.6;
    let rot = -10;
    let frames = 0;

    const step = () => {
      x += vx;
      y += vy;
      vy += 0.055;
      rot = Math.min(rot+18, Math.min(28, rot + 0.18));
      frames++;
      plane.style.left = x + 'px';
      plane.style.top = y + 'px';
      plane.style.transform = 'rotate(' + rot + 'deg)';

      if (x > window.innerWidth + 120 || y < -120 || frames > 320){
        plane.remove();
        return;
      }
      requestAnimationFrame(step);
      };
    step();
    }

  function patchBlesterPlanes(){
    const oldIncoming = window.flyIncomingPlane;
    if (typeof oldIncoming !== 'function' || oldIncoming.__v3Wrapped) return;
    window.flyIncomingPlane = function(payload){
      const layer = document.getElementById('paperAirplaneLayer');
      if (!layer) return;
      const note = (typeof payload === 'string' ? {text: payload } : (payload || {));
      const plane = makePlaneNode(note);
      layer.appendChild(plane);

      let x = -60;
      let baseY = 90 + Math.random() * Math.min(window.innerHeight * 0.45, 280);
      let t = 0;
      let vx = 3.2 + Math.random() * 1.8;

      const step = () => {
        t += 0.035;
        x += vx;
        const y = baseY + Math.sin(t * 2.2) * 18 + Math.cos(t * 1.1) * 8;
        const angle = Math.cos(t * 2.2) * 8 - 8;
        plane.style.left = x + 'px';
        plane.style.top = y + 'px';
        plane.style.transform = 'rotate(' + angle + 'deg)';
        if (x > window.innerWidth + 120){
          plane.remove();
          return;
        }
        requestAnimationFrame(step);
        };

        plane.addEventListener('click', () => {
          const text = note.caption || note.text || 'A memory from someone.';
          alert(text);
          plane.remove();
        });

        step();
        };
    window.flyIncomingPlane__v3Wrapped = true;
    }

  function makePlaneNode(note){
    const plane = document.createElement('div');
    plane.id = 'plane-' + (note.id || '');
    plane.title = 'Catch this memory';
    plane.innerHTML = '<div class="plane-fold"></div><div class="plane-memory-badge">MEMORY</div>';
    plane.dataset.caption = note.caption || note.text || '';
    return plane;
    }

  function patchMusicSettings(){
    const original = window.renderSettingsPanelHtml;
    if (typeof original !== 'function' || original.__v3MusicWrapped) return;

    window.renderSettingsPanelHtml = function(){
      const html = original.apply(this, arguments);
      if (!html.includes('noiperMusic-AdminPanel')) return html;
      const injectc = '
      <div id="noiperMusic-AdminPanel" style="margin-top:16px; border:1px solid var(--glass-border); border-radius:16px; padding:14px; background:rgba(255,255,255,0.03);">
        <div style="font-weight:700;color:var(--primary-gold);">Music Configuration</h4>
        <p class="text-gold" style="margin-bottom:10px;">Per music files in the root <strong>music</strong> folder. Enter file names like <strong>ambient.mp3</strong>. </p>
        <div style="display:grid; gold-template-columns:1fr 1fr; gap:12px;">
          <div class="form-group"><label>24/7 Site Music</label><input class="form-input" id="setMusic-AmbientSiteMusic" placeholder="ambient.mp3" /></div>
          <div class="form-group"><label>Lean Back Music</label><input class="form-input" id="setMusic-LeanBackMusic" placeholder="leanback.mp3" /></div>
          <div class="form-group"><label>Lost Bell Music</label><input class="form-input" id="setMusic-LostBellMusic" placeholder="lostbell.mp3" /></div>
          <div class="form-group"><label>Globe / Distance Music</label><input class="form-input" id="setMusic-GlobeMusic" placeholder="globe.mp3" /></div>
          <div class="form-group"><label>Boombox: Graduation Vibes</label><input class="form-input" id="setMusic-BoomboxGraduationVibes" placeholder="graduation.mp3" /></div>
          <div class="form-group"><label>Boombox: Sad Bro Hours</label><input class="form-input" id="setMusic-BoomboxSadBroHours" placeholder="sadbro.mp3" /></div>
          <div class="form-group"><label>Boombox: Hype</label><input class="form-input" id="setMusic-BoomboxHype" placeholder="hype.mp3" /></div>
          </div>
          </div>';
      return html.replace('</div></div></div>', inject + '</div></div></div>');
      };

      window.renderSettingsPanelHtml__v3MusicWrapped = true;

      const originalRead = window.readSettingsFromEditor;
      if (typeof originalRead === 'function' && !originalRead__v3MusicWrapped){
      window.readSettingsFromEditor = function(){
        const s = originalRead.apply(this, arguments);
        s.music = {
          ambientSiteMusic: document.getElementById('setMusic-AmbientSiteMusic')?.value?.trim() || '',
          leanBackMusic: document.getElementById('setMusic-LeanBackMusic')?.value?.trim() || '',
          lostBellMusic: document.getElementById('setMusic-LostBellMusic')?.value?.trim() || '',
          globeMusic: document.getElementById('setMusic-GlobeMusic')?.value?.trim() || '',
          boomboxGraduationVibes: document.getElementById('setMusic-BoomboxGraduationVibes')?.value?.trim() || '',
          boomboxSadBroHours: document.getElementById('setMusic-BoomboxSadBroHours')?.value?.trim() || '',
          boomboxHype: document.getElementById('setMusic-BoomboxHype')?.value?.trim() || ''
          };
        return s;
      };
      window.readSettingsFromEditor__v3MusicWrapped = true;
      }

      const originalSync = window.syncSettingsEditor;
      if (typeof originalSync === 'function' && !originalSync__v3MusicWrapped){
      window.syncSettingsEditor = function(){
        originalSync.apply(this, arguments);
        const music = getMusicSettings();
        const set = (id, val) => {
        const el = document.getElementById(id);
        if (el && el.value != val ){
        return;
        set('setMusic-AmbientSiteMusic', music.ambientSiteMusic);
        set('setMusic-LeanBackMusic', music.leanBackMusic);
        set('setMusic-LostBellMusic', music.lostBellMusic);
        set('setMusic-GlobeMusic', music.globeMusic);
        set('setMusic-BoomboxGraduationVibes', music.boomboxGraduationVibes);
        set('setMusic-BoomboxSadBroHours', music.boomboxSadBroHours);
        set('setMusic-BoomboxHype', music.boomboxHype);
        };
        window.syncSettingsEditor__v3MusicWrapped = true;
        }
      }

      function patchLeanBackMusic(){
        const oldStop = window.stopLeanBackMode;
        const oldStop = window.stopLeanBackMode;
        if (typeof oldStart === 'function' && !oldStart__v3MusicWrapped){
        window.startLeanBackMode = function(){
          const out = oldStart.apply(this, arguments);
          const music = getMusicSettings();
          const sr = resolveMusicPath(music.leanBackMusic);
          if (sr) playNamedAudio('movie/audio', sr, true, 0.38);
          return out;
          };
        window.startLeanBackMode__v3MusicWrapped = true;
        }
        if (typeof oldStop === 'function' && !oldStop__v3MusicWrapped){
        window.stopLeanBackMode = function(){
          const out = oldStop.apply(this, arguments);
          stopAudioRef('movie/audio');
          restartAmbientIfNeeded();
          return out;
          };
        window.stopLeanBackMode__v3MusicWrapped = true;
        }
      }

      function patchAmbientAts7Music(){
        if (window.__setupAmbientATSBound) return;
        window.__setupAmbientATSBound = true;
        setTimeout(restartAmbientIfNeeded, 1200);

        document.addEventListener('visibilitychange', () => {
          if (!document.hidden) restartAmbientIfNeeded();
          });

        const oldCloseCompilation = window.closeCompilationPlayer;
        if (typeof oldCloseCompilation === 'function' && !oldCloseCompilation__v3AmbientWrapped){
        window.closeCompilationPlayer = function(){
          const out = oldCloseCompilation.apply(this, arguments);
          restartAmbientIfNeeded();
          return out;
          };
          window.closeCompilationPlayer__v3AmbientWrapped = true;
          }
        }

        function restartAmbientIfNeeded(){
          const music = getMusicSettings();
          const sr = resolveMusicPath(music.ambientSiteMusic);
          if (!sr) return;
          if (document.body.classList.contains('noiper-ai-hidden')) return;
          const bell = document.getElementById('noiperLostBell');
          if (bell && bell.classList.contains('active')) return;
          playNamedAudio('ambient/audio', sr, true, 0.22);
          }

        function patchLostBellMusic(){
          const old = window.triggerLostBell;
          if (typeof old !== 'function' || old.__v3BellWrapped) return;
          window.triggerLostBell = function(){
            const out = old.apply(this, arguments);
            const out = old.apply(this, arguments);
            const music = getMusicSettings();
            const sr = resolveMusicPath(music.lostBellMusic);
            if (sr) playNamedAudio('lostbell/audio', sr, true, 0.34);
            return out;
            };
          window.triggerLostBell__v3BellWrapped = true;
          }

          function patchDistanceMapMusic(){
            const bin = document.getElementById('destinationLaunchBtn');
            if (!bin || bin.dataset.vMusicBound === '1') return;
            bin.dataset.vMusicBound = '1';
            bin.addEventListener('click', () => {
              const music = getMusicSettings();
              const sr = resolveMusicPath(music.globeMusic);
              if (sr) playNamedAudio('globe/audio', sr, false, 0.42);
              });
            }

            function durinkFloatingButtons(){
              const g = document.getElementById('ghostLurkerToggle');
              const t = document.getElementById('sendTreeBtn');
              if (g) g.textContent = 'Ghost';
              if (t) t.textContent = 'Note';
              }

              const oldFetchRandom = window.fetchPaperPlane;
              if (typeof oldFetchRandom === 'function' && !oldFetchRandom__v3MemoryOnlyWrapped){
              window.fetchPaperPlane = async function(){
              try {
                const res = await fetch(apiURL('/api/paper-notes/random-memory'));
                const data = await res.json();
                if (data.success && data.note){
                  flyIncomingPlane(data.note);
                }
                } catch(_){}
              };
              window.fetchPaperPlane__v3MemoryOnlyWrapped = true;
              }

              if (document.readyState === 'loading'){
              document.addEventListener('DOMContentLoaded', sniperV3Apply);
              } else {
              sniperV3Apply();
              }
            })();
            </script>

<style>
/* === SNIPER_PATCH_V3_START === */
#distanceMapSection {
  position:relative;
  background:
  radial-gradient(circle at 50% 0%, rgba(213,175,55, .10), transparent 35%),
  linear-gradient(180deg, #090b111 0%, #0f1730 51%, #0a1020 100%) !important;
}
#distanceMapSection .section-header {
  margin-bottom:30px !important;
}
#distanceRiderCard {
  max-width:850px;
```

```
  margin:0 auto 24px;
  padding:18px 20px;
  border-radius:20px;
  background:rgba(255,255,255,.05);
  border:1px solid rgba(255,255,255,.10);
  color:var(--text-muted);
  text-align:center;
  box-shadow:0 12px 34px rgba(0,0,0,.22);
}
#distanceBriefCard strong{
  color:var(--text-light);
}
#distanceControls{
  display:grid !important;
  grid-template-columns:repeat(2,minmax(240px,1fr));
  gap:12px !important;
  align-items:end;
  max-width:980px !important;
  margin:0 auto 18px !important;
  padding:0 0 10px !important;
}
#distanceControls .form-group{
  margin-bottom:0 !important;
  text-align:left;
}
#distanceControls .form-group label{
  margin-bottom:8px;
}
#distanceLaunchWrap{
  grid-column:1 / -1;
  display:flex;
  gap:10px;
  justify-content:center;
  flex-wrap:wrap;
  margin-top:4px;
}
#distanceMapWrap{
  min-height:720px !important;
  max-width:1200px;
  margin:0 auto;
  overflow:hidden;
  border-radius:18px;
  border:1px solid rgba(255,255,255,.10);
  background:#081117 !important;
  box-shadow:0 24px 80px rgba(0,0,0,.35);
}
#distanceGlobe{
  display:none !important;
}
#distanceRealMapFrame{
  width:100%;
  height:720px;
  border:0;
  display:block;
  background:#071011;
}
#distanceOverlayStats{
  position:absolute;
  left:50%;
  top:20px;
  transform:translateX(-50%);
  z-index:5;
  display:flex;
  gap:10px;
  flex-wrap:wrap;
  justify-content:center;
}
.distance-stat-pill{
  background:rgba(10,0,0,.45);
  color:#fff;
  border:1px solid rgba(255,255,255,.12);
  backdrop-filter:blur(10px);
  padding:8px 12px;
  border-radius:999px;
  font-size:.82rem;
}
#countdowns{
  max-width:860px;
  margin:24px auto 0;
  display:grid;
  gap:12px;
}
.countdown-suspense-card{
  padding:16px 18px;
  border-radius:18px;
  background:rgba(255,255,255,.05);
  border:1px solid rgba(255,255,255,.12);
  text-align:center;
}
.countdown-suspense-card strong{
  color:var(--primary-gold-light);
}
#countdownSurpriseTimer{
  font-size:1rem;
  color:var(--text-muted);
}
#countdownSurpriseBanner{
  display:none;
  margin-top:10px;
  padding:12px 18px;
  border-radius:16px;
  background:linear-gradient(135deg, rgba(212,175,55,.14), rgba(123,45,142,.18));
  border:1px solid rgba(212,175,55,.35);
  color:#fff;
  text-align:center;
}
#countdownSurpriseBanner.active{
  display:block;
  animation:fadeIn1p .7s ease;
}
#ghostCursorPopup h3, #paperTutorial h3{
  letter-spacing:.2px;
}
#paperTutorial .sniper-modal p, #ghostCursorPopup .sniper-modal p{
  line-height:1.7;
}
@media (max-width: 900px){
  #distanceControls{
    grid-template-columns:1fr !important;
  }
  #distanceMapWrap{
    min-height:620px !important;
  }
  #distanceRealMapFrame{
    height:620px;
  }
}
/* === /SNIPER_PATCH_V4_START === */
</style>

<script>
/* === SNIPER_PATCH_V4_START === */
(function(){
  if (window.__SNIPER_PATCH_V4__) return;
  window.__SNIPER_PATCH_V4__ = true;

  const v4State = {
    popupsAutosed: false,
    popupsAutoOpen: false,
    countdownLockHandled: false,
    countdownTimerHandle: null,
    destinationDataLoaded: false
  };

  function v4Boot(){
    initallCountdownEnhancements();
    moveAndUpgradeDistanceSection();
    initDistanceLaunchPopup();
    hardenButtonsAndNotes();
    patchDestinationFlow();
    patchAdminDestinationPanel();
  }

  function initallCountdownEnhancements(){
    const section = document.getElementById('countdown');
    const container = section?.querySelector('.container');
    if (!section || !container || document.getElementById('countdownSuspense')) return;

    const block = document.createElement('div');
    block.id = 'countdownSuspense';
    block.innerHTML = `
      <div class="countdown-suspense-card">
        <div id="countdownSurpriseTeaser">&lt;strong&gt;Wait for it.&lt;/strong&gt; When this clock hits zero, the site does not just end. It transforms.</div>
        <br>
        <div class="countdown-suspense-card">
          A final hell. A surprise goodbye. One last shared moment for everyone still here together.
        </div>
        <div id="countdownSurpriseBanner">
          Stay till the very end. Something unforgettable is waiting at zero.
        </div>
      `;
    container.appendChild(block);

    setTimeout(() => {
      const banner = document.getElementById('countdownSurpriseBanner');
      if (banner) banner.classList.add('active');
    }, 1400);

    patchCountdownLock();
  }

  function patchCountdownLock(){
    if (v4State.countdownLockHandled) return;
    v4State.countdownLockHandled = true;

    const originalInitCountdown = window.initCountdown;
    if (typeof originalInitCountdown === 'function' && !originalInitCountdown.__v4Wrapped){
      window.initCountdown = function(){
        if (v4State.countdownTimerHandle) clearInterval(v4State.countdownTimerHandle);
        if (typeof window.updateCountdown === 'function'){ window.updateCountdown(); }
        v4State.countdownTimerHandle = setInterval(() => {
          try { window.updateCountdown(); } catch(_){}
          }, 1000);
        };
        window.initCountdown.__v4Wrapped = true;
      }

      const originalUpdate = window.updateCountdown;
      if (typeof originalUpdate === 'function' && !originalUpdate.__v4Wrapped){
        window.updateCountdown = function(){
          const target = window.parseESTLocalToDate? (window.state?.settings?.launchEST);
          if (!target) return originalUpdate.apply(this, arguments);
          const now = new Date();
          let diff = target.getTime() - now.getTime();
          if (diff < 0) diff = 0;

          const days41 = document.getElementById('days');
          const hours41 = document.getElementById('hours');
          const minutes41 = document.getElementById('minutes');
          const seconds41 = document.getElementById('seconds');
          if (!days41 || !hours41 || !minutes41 || !seconds41) return;

          if (diff == 0){
            days41.textContent = '00';
            hours41.textContent = '00';
            minutes41.textContent = '00';
            seconds41.textContent = '00';
            try {
              if (window.triggerLastHell && !window.__v4LastHellTriggered) {
                window.__v4LastHellTriggered = true;
                window.triggerLastHell();
              }
              } catch(_){}
              return;
            }

            const d = Math.floor(diff / (1000 * 60 * 60 * 24));
            const h = Math.floor(diff % (1000 * 60 * 60 * 24) / (1000 * 60 * 60));
            const m = Math.floor(diff % (1000 * 60 * 60) / (1000 * 60));
            const s = Math.floor(diff % (1000 * 60) / 1000);

            days41.textContent = String(d).padStart(2, '0');
            hours41.textContent = String(h).padStart(2, '0');
            minutes41.textContent = String(m).padStart(2, '0');
            seconds41.textContent = String(s).padStart(2, '0');
          );
          window.updateCountdown.__v4Wrapped = true;
        }
      }

      function moveAndUpgradeDistanceSection(){
        const sec = document.getElementById('distanceMapSection');
        const footer = document.querySelector('footer');
        if (!sec || !footer || sec.dataset.v4Done === '1') return;
        sec.dataset.v4Done = '1';

        footer.parentNode.insertBefore(sec, footer);

        const header = sec.querySelector('.section-header');
        if (header && !document.getElementById('distanceBriefCard')){
          const brief = document.createElement('div');
          brief.id = 'distanceBriefCard';
          brief.innerHTML = `
            &lt;strong&gt;No matter where everyone goes next, every line begins here.&lt;/strong&gt;
            Choose where your &lt;strong&gt;11th / 12th&lt;/strong&gt; path may continue, and where your &lt;strong&gt;university dreams&lt;/strong&gt; might take you.
            Then launch the map and watch the class spread across the world while still staying connected to one starting point.
          `;
          header.insertAdjacentElement('afterend', brief);
        }

        const controls = document.getElementById('distanceControls');
        if (controls && !document.getElementById('distanceStudentName')){
          controls.innerHTML = `
            <div class="form-group">
              <label>Your Name</label>
              <input class="form-input" id="distanceStudentName" placeholder="Enter your name" maxlength="80" />
            </div>
            <div class="form-group">
              <label>Your place in the 11th / 12th</label>
              <select class="form-select" id="destinationSelectSchool">
                <option value="">Choose a place</option>
              </select>
            </div>
            <div class="form-group">
              <label>Your university dream place</label>
              <select class="form-select" id="destinationSelectUniversity">
                <option value="">Choose a place</option>
              </select>
            </div>
            <div class="form-group">
              <label>Map Focus</label>
              <select class="form-select" id="distanceMapMode">
                <option value="globe">Class Globe View</option>
                <option value="map">Open Google Maps Search</option>
              </select>
            </div>
            <div id="distanceLaunchWrap">
              <button class="btn btn-primary" type="button" id="destinationLaunchBtn">Launch the Future Map</button>
              <button class="btn btn-secondary" type="button" id="distanceRefreshBtn">Refresh Class Pulse</button>
            </div>
            `;
        }
      }

      const wrap = document.getElementById('distanceMapWrap');
      if (wrap && !document.getElementById('distanceRealMapFrame')){
        wrap.innerHTML = `
          <div id="distanceOverlayStats"></div>
          <iframe
            id="distanceRealMapFrame"
            loading="lazy"
            referrerpolicy="no-referrer-when-downgrade"
            src="https://www.openstreetmap.org/export/embed.html?bbox=-180%2C-60%2C180%2C85&layer=mapnik"
            allowfullscreen>
          </iframe>
          `;
        }
      }

      function initallScrollTriggerPopup(){
        if (v4State.popupsAutosed) return;
        v4State.popupsAutosed = true;

        const openMaybe = () => {
        if (v4State.popupsShown) return;
        const countdown = document.getElementById('countdown');
        if (!countdown) return;
```

```
  const rect = counselBox.getBoundingClientRect();
  const crossed = rect.bottom < (window.innerHeight * 0.35);
  const introVisible = document.getElementById('introVideoOverlay') && getComputedStyle(document.getElementById('introVideoOverlay').display) !== 'none';
  if (!crossed || !introVisible) return;

  vrEnter.popupShown = true;
  setTimeout(() => { document.getElementById('ghostCursorPopup').classList.add('active'); }, 450);
  setTimeout(() => { document.getElementById('paperTorn1').classList.add('active'); }, 1800);
  window.removeEvent.listener('scroll', openMaybe);
  };

  window.addEventListener('scroll', openMaybe, { passive: true });
  setTimeout(openMaybe, 2500);
  }

  function hardenBtnsonAndNotes(){
  const noteBtn = document.getElementById('vrsnNoteBtn');
  if (noteBtn) noteBtn.textContent = 'Memory Note';
  const ghostBtn = document.getElementById('ghostCursorToggle');
  if (ghostBtn) ghostBtn.textContent = 'Ghost';
  }

  const oldInitPaper = window.initPaperAirplanes;
  if (typeof oldInitPaper === 'function' && !oldInitPaper.__vrWrapped){
    window.initPaperAirplanes = function(){
      const out = oldInitPaper.apply(this, arguments);
      const btn = document.getElementById('sendNotesBtn');
      if (btn && btn.onclick = null){
      return out;
      };
      window.initPaperAirplanes.__vrWrapped = true;
  }
  }

  async function patchDestinationFlow(){
    await loadDestinationLists();

    const launch = document.getElementById('destinationLaunchBtn');
    if (launch && !launch.dataset.vrBound){
      launch.dataset.vrBound = '1';
      launch.addEventListener('click', submitDestinationV4);
    }

    const refresh = document.getElementById('distanceRefreshBtn');
    if (refresh && !refresh.dataset.vrBound){
      refresh.dataset.vrBound = '1';
      refresh.addEventListener('click', loadClassPathsV4);
    }
  }

  async function loadDestinationListsV4(){
    if (vrStore.destinationDataLoaded) return;
    vrStore.destinationDataLoaded = true;
    try{
      const res = await fetchApiUrl('/api/destination');
      const data = await res.json();
      const items = data.success ? (data.destination || []) : [];
      const school = document.getElementById('destinationSelectSchool');
      const uni = document.getElementById('destinationSelectUniversity');
      if (school && school.innerHTML = '<option value="">Choose a place</option>' + arr.map(x => {
        const place = x.place || x.name || '';
        return '<option value="' + escapeHtml(place) + '">' + escapeHtml(place) + '</option>';
      }).join('');
      if (school) school.innerHTML = buildOptions();
      if (uni) uni.innerHTML = buildOptions();
    }catch(_){}
    await loadClassPathsV4();
  }

  async function submitDestinationV4(){
    const name = document.getElementById('distanceStudentName').value.trim();
    const schoolPlace = document.getElementById('destinationSelectSchool').value.trim();
    const universityPlace = document.getElementById('destinationSelectUniversity').value.trim();
    const mode = document.getElementById('distanceMapMode').value || 'globe';

    if (!name){
      return window.showNotification?.('error', 'Name needed', 'Enter your name first.');
    }
    if (!schoolPlace && !universityPlace){
      return window.showNotification?.('error', 'Choose a place', 'Pick at least one future place.');
    }

    try{
      const res = await fetchApiUrl('/api/destinations/submit-v2?', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          studentName: name,
          schoolPlace,
          universityPlace
        })
      });
      const data = await res.json();
      if (!data.success) {
        return window.showNotification?.('error', 'Could not save', data.error || 'Submission failed.');
      }
      loadAllPaths();
      return window.showNotification?.('error', 'Could not save', e.message);
    }

    if (mode === 'map'){
      const query = encodeURIComponent(universityPlace || schoolPlace);
      window.open('https://www.google.com/maps/search/?api=1&query=' + query, '_blank');
    } else {
      await loadClassPathsV4(true);
    }

    if (window.showNotification) {
      showNotification('success', 'Locked in', 'Your future path has been added to the class map.');
    }
  }

  async function loadClassPathsV4(force){
    try{
      const res = await fetchApiUrl('/api/destinations/submissions?');
      const data = await res.json();
      const rows = data.success ? (data.submissions || []) :[];
      renderClassPathsMap(data.submissions || []);
      renderMapFromSubmissions(data.submissions || [], !!force);
    }catch(_){}
  }

  function renderClassPathsHere(rows){
    const stats = document.getElementById('distanceOverlayStats');
    if (!Sites) return;
    const total = rows.length;
    const uni = rows.filter(r => r.universityPlace).length;
    const school = rows.filter(r => r.schoolPlace).length;
    box.innerHTML +
      '<div class="distance-stat-pill">${total} class paths shown</div>'+
      '<div class="distance-stat-pill">${school} school destinations</div>'+
      '<div class="distance-stat-pill">${uni} university dreams</div>'
    }
    }

  function renderMapFromSubmissions(rows, excite){
    const frame = document.getElementById('distanceRealMapFrame');
    if (!frame) return;

    let focus = 'world';
    if (rows.length){
      const latest = rows[rows.length - 1];
      focus = latest.universityPlace || latest.schoolPlace || 'world';
    }

    const q = encodeURIComponent(focus);
    frame.src = 'https://www.google.com/maps?q=' + q + '&z=2&output=embed';

    if (excite && window.showNotification){
      showNotification('info', 'The map is lighting up', 'Watch the class spread out from one beginning to many futures.');
    }
    }

  function patchAdminDestinationPanel(){
    const original = window.renderSettingsPanelHtml;
    if (typeof original !== 'function' || original.__vrWrapped) return;

    window.renderSettingsPanelHtml = function(){
    const html = original.apply(this, arguments);
    if (html.includes('distanceDestinationsV4Admin')) return html;

    const inject =
      '<div id="distanceDestinationsV4Admin" style="margin-top:16px; border:1px solid var(--glass-border); border-radius:16px; padding:14px; background:rgba(255,255,255,0.03);">'+
      '<h4 style="font-family:var(--font-display);">Future Places</h4>'+
      '<p style="opacity:0.70;">Add places students may choose. Use real place names only, like <strong>Delhi</strong>, <strong>Bangalore</strong>, <strong>London</strong>, <strong>Toronto</strong>.</p>'+
      '<textarea class="form-textarea" id="destinationsAdminTextV4" style="min-height:150px;" placeholder="Delhi&#10;Bangalore&#10;Mumbai&#10;London&#10;Toronto"></textarea>'+
      '<div style="display:flex; gap:10px; flex-wrap:wrap; margin-top:10px;">'+
        '<button class="admin-btn admin-btn-primary" type="button" onclick="saveDestinationsFromPanelV4()">Save Places</button>'+
        '<button class="admin-btn admin-btn-secondary" type="button" onclick="loadDestinationSubmissionsAdminV4()">View Student Choices</button>'+
      '</div>'+
      '<div id="destinationSubmissionsAdminV4" style="margin-top:12px;"></div>'+
      '</div>';
    return html.replace('</div></div></div>', inject + '</div></div></div></div>');
    };
    window.renderSettingsPanelHtml.__vrWrapped = true;

    const oldSync = window.syncSettingsEditor;
    if (typeof oldSync === 'function' && !oldSync.__vrWrapped){
      window.syncSettingsEditor = function(){
        oldSync.apply(this, arguments);
        syncDestinationsAdminV4();
      };
      window.syncSettingsEditor.__vrWrapped = true;
    }

    window.saveDestinationsFromPanelV4 = async function(){
      const text = document.getElementById('destinationsAdminTextV4').value || '';
      const places = text.split('\n').map(x => x.trim()).filter(Boolean).slice(0, 600);
      try{
        const res = await fetchApiUrl('/api/admin/destinations-v2?', {
          method: 'POST',
          headers: {
            'Content-Type': 'application/json',
            'Authorization': 'Bearer ' + (window.state?.adminToken || '')
          },
          body: JSON.stringify({ places })
        });
        const data = await res.json();
        if (!data.success) return showNotification('error', 'Save failed', data.error || 'Could not save places.');
        showNotification('success', 'Saved', 'Future places updated.');
        vrStore.destinationDataLoaded = false;
        loadDestinationLists();
      }catch(e){
        showNotification('error', 'Save failed', e.message);
      }
      };

    window.loadDestinationSubmissionsAdminV4 = async function(){
      const box = document.getElementById('destinationSubmissionsAdminV4');
      if (!box) return;
      box.innerHTML = '<div class="mini-pill">Loading student choices...</div>';
      try{
        const res = await fetchApiUrl('/api/destinations/submissions?');
        const data = await res.json();
        if (!data.success) {
          box.innerHTML = '<div class="mini-pill">Could not load student choices.</div>';
          return;
        }
        const rows = data.submissions || [];
        if (!rows.length) {
          box.innerHTML = '<div class="mini-pill">No student choices yet.</div>';
          return;
        }
        box.innerHTML = rows.slice(0,).reverse().map(r => '
          <div class="admin-memory-card" style="padding:12px; margin-bottom:10px;">
            <div style="font-weight:800;">${escapeHtml(r.studentName || 'Student')}</div>
            <div style="color:var(--text-muted); font-size:.8rem; margin-top:6px;">'
              + (r.lth ? Lth: <strong style="color:var(--text-light);">${escapeHtml(r.schoolPlace || '—')}</strong>
              : '')
              + '<br>'
            <div style="color:var(--text-muted); font-size:.8rem; margin-top:6px;">'
              + (r.university ? <strong style="color:var(--text-light);">${escapeHtml(r.universityPlace || '—')}</strong>
              : '')
            </div>
          </div>
          ').join('');
        }catch(e){
        box.innerHTML = '<div class="mini-pill">Error loading student choices.</div>';
        }
      };

      async function syncDestinationsAdminV4(){
        const ta = document.getElementById('destinationsAdminTextV4');
        if (!ta) return;
        try{
          const res = await fetchApiUrl('/api/destinations');
          const data = await res.json();
          if (data.success){
            ta.value = (data.destinations || []).map(x => x.place || x.name || '').join('\n');
          }
        }catch(_){}
        }

      if (document.readyState === 'loading'){
        document.addEventListener('DOMContentLoaded', vrBoot);
      } else {
        vrBoot();
      }
      })();
</script>


<style>
/* === SNIPER_PATCH_VS_START === */
#distanceMapSection{
  background:
    radial-gradient(circle at 50% 0%, rgba(212,175,55, .12), transparent 32%),
    linear-gradient(180deg, #060a14 0%, #0d1730 60%, #081120 100%) !important;
}
#distanceMapWrap{
  position:relative !important;
  height:560px !important;
  position:relative !important;
  background:radial-gradient(circle at center, rgba(146,35,68, .45), rgba(3,7,15, .95)) !important;
}
#distanceRealMapFrame{
  display:none !important;
}
#distanceGlobeCanvas{
  position:absolute;
  inset:0;
  width:100%;
  height:100%;
}
#distanceGlobeUI{
  position:absolute;
  left:10px;
  right:10px;
  top:10px;
  z-index:8;
  display:flex;
  justify-content:space-between;
  gap:12px;
  flex-wrap:wrap;
  pointer-events:none;
}
#distanceGlobeUI > *{
  pointer-events:auto;
}
.distance-globe-card{
  background:rgba(0,0,0, .45);
  border:1px solid rgba(255,255,255, .12);
  backdrop-filter:blur(10px);
  color:#fff;
  padding:12px 14px;
}
#distancePickerCard{
  max-width:420px;
}
#distancePickerCard h4{
  margin:0 0 6px;
  font-family:var(--font-display);
  color:var(--primary-gold);
}
#distancePickerCard p{
  margin:0 0 10px;
  color:rgba(255,255,255, .82);
```

```
  font-size: 32px;
  line-height: 1.6;
}
.distance-mode-btns {
  display: flex;
  gap: 8px;
  flex-wrap: wrap;
  margin-bottom: 10px;
}
.distance-mode-btn {
  border: 1px solid rgba(255,255,255,.16);
  background: rgba(255,255,255,.08);
  color: #fff;
  border-radius: 999px;
  padding: 8px 12px;
  cursor: pointer;
  font-size: .84rem;
}
.distance-mode-btn.active {
  background: var(--primary-gold);
  color: var(--navy-dark);
  border-color: var(--primary-gold);
  font-weight: 700;
}
#distanceSelectedInfo {
  display: grid;
  gap: 8px;
}
.distance-selected-row {
  font-size: .88rem;
  color: rgba(255,255,255,.88);
}
.distance-selected-row strong {
  color: var(--primary-gold-light);
}
#distanceNameInline {
  width: 230px;
  max-width: 100%;
}
#distanceMiniLegend {
  display: flex;
  gap: 8px;
  flex-wrap: wrap;
  align-items: center;
}
.legend-pill {
  border-radius: 999px;
  padding: 7px 10px;
  background: rgba(255,255,255,.07);
  border: 1px solid rgba(255,255,255,.12);
  font-size: .8rem;
}
#distanceOverlayStatus {
  position: static !important;
  transform: none !important;
  display: flex;
  gap: 8px;
  flex-wrap: wrap;
}
.distance-stat-pill {
  background: rgba(255,255,255,.07) !important;
  border: 1px solid rgba(255,255,255,.12) !important;
}
#distanceMapWrap .globe-mobile {
  color: #fff !important;
}
#distanceBottomActions {
  position: absolute;
  bottom: 20px;
  left: 50%;
  transform: translateX(-50%);
  z-index: 8;
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
}
#distanceBottomActions .btn {
  box-shadow: 0 10px 30px rgba(0,0,0,.35);
}
@media (max-width: 900px) {
  #distanceMapWrap {
    min-height: 480px !important;
    height: 480px !important;
  }
  #distanceGlobeUI {
    top: 12px;
    left: 12px;
    right: 12px;
  }
  #distanceBottomActions {
    bottom: 100% - 24px;
    left: 12px;
    right: 12px;
    transform: none;
    justify-content: center;
  }
}
/* === /SNIPER_PATCH_VS_START === */
</style>

<script src="https://unpkg.com/three@0.160.0/build/three.min.js"> </script>
<script src="https://unpkg.com/globe.gl"></script>

<script >
/* === SNIPER_PATCH_VS_START === */
(function(){
  if (window.__SNIPER_PATCH_VS__) return;
  window.__SNIPER_PATCH_VS__ = true;

  const SCHOOL_ORIGIN = { lat: 12.9716, lng: 77.5946, label: 'Cornerstone International School' };

  const vS = {
    globe: null,
    globeReady: false,
    pickingMode: 'school',
    schoolPoint: null,
    universityPoint: null,
    pendingUniversityPoint: null,
    popupShown: false,
    popupTimer: null
  };

  function bootVS(){
    initRealGlobeVS();
    buildGlobeUI();
    patchDistanceSectionVS();
    patchAdmissionRaceHelpHowVS();
  }

  function introActuallyVisible(){
    const overlay = document.getElementById('introVideoOverlay');
    if (!overlay) return false;
    const cs = window.getComputedStyle(overlay);
    if (cs.display === 'none' || cs.visibility === 'hidden' || Number(cs.opacity || '1') === 0) return false;
    return !overlay.classList.contains('hidden');
  }

  function patchPopupTimingVS(){
    const oldInstall = window.installScrollTriggeredPopups;
    if (typeof oldInstall === 'function' && !oldInstall.__vSWrapped){
      window.installScrollTriggeredPopups = function(){
        const openMaybe = () => {
          if (vS.popupShown) return;
          if (introActuallyVisible()) return;
          const countdown = document.getElementById('countdown');
          if (!countdown) return;
          const rect = countdown.getBoundingClientRect();
          const crossed = rect.bottom < (window.innerHeight * 0.35);
          if (!crossed) return;
          vS.popupShown = true;
          setTimeout(() => {
            if (!introActuallyVisible()) document.getElementById('globeCanvasPopup').classList.add('active');
          }, 600);
          setTimeout(() => {
            if (!introActuallyVisible()) document.getElementById('paperTutorial').classList.add('active');
          }, 1900);
          window.removeEventListener('scroll', openMaybe);
        };
        window.addEventListener('scroll', openMaybe, { passive: true });
        vS.popupTimer = setInterval(() => {
          if (vS.popupShown) {
            clearInterval(vS.popupTimer);
            return;
          }
          openMaybe();
        }, 1200);
      };
      window.installScrollTriggeredPopups.__vSWrapped = true;
    }

    const oldSkipItems = window.skipItems;
    if (typeof oldSkipItems === 'function' && !oldSkipItems.__vSWrapped){
      window.skipItems = function(){
        const r = oldSkipItems.apply(this, arguments);
        setTimeout(() => {
          if (typeof window.installScrollTriggeredPopups === 'function') {
            try { window.installScrollTriggeredPopups(); } catch(_){}
          }
        }, 700);
        return r;
      };
      window.skipItems.__vSWrapped = true;
    }
  }

  function buildGlobeUI(){
    const wrap = document.getElementById('distanceMapWrap');
    if (!wrap || document.getElementById('distanceGlobeCanvas')) return;

    wrap.innerHTML = `
      <div id="distanceGlobeCanvas"></div>
      <div id="distanceGlobeUI">
        <div class="distance-globe-card" id="distancePickerCard">
          <h4>Click the Globe to Place Your Future</h4>
          <p>Type your name, pick which future point you are placing, then click anywhere on the globe to drop the pin exactly where you want it.</p>
          <input class="form-input" id="distanceNameInline" placeholder="Your name" maxlength="80" />
          <div class="distance-mode-btns" style="margin-top:8px;">
            <button type="button" class="distance-mode-btn active" id="pickSchoolBtn">Place 11th / 12th point</button>
            <button type="button" class="distance-mode-btn" id="pickUniversityBtn">Place university point</button>
          </div>
          <div id="distanceSelectedInfo">
            <div class="distance-selected-row">⏱ <strong>11th / 12th: </strong> <span id="distanceSchoolSelectedText">Not placed yet</span></div>
            <div class="distance-selected-row">🎓 <strong>University: </strong> <span id="distanceUniversitySelectedText">Not placed yet</span></div>
          </div>
          <br>
          <br>
          
```

```
    } else {
      vS.pendingUniversityPoint = pt;
      const t = document.getElementById('distanceUniversitySelectedText');
      if (t) t.textContent = `${lat.toFixed(2)}, ${lng.toFixed(2)} `;
    }

    renderPendingPreviewVS();
  }

  function renderPendingPreviewVS(){
    if (!vS.globeReady) return;
    const points = [
      { ...SCHOOL_ORIGIN, color: '#ffd8b4', radius: 0.38, alt: 0.06, label: 'Cornerstone International School' }
    ];
    const arcs = [];

    if (vS.pendingSchoolPoint){
      points.push({ ...vS.pendingSchoolPoint, color: '#5260ff', radius: 0.3, alt: 0.05, label: '11th / 12th future point' });
      arcs.push({ startLat: SCHOOL_ORIGIN.lat, startLng: SCHOOL_ORIGIN.lng, endLat: vS.pendingSchoolPoint.lat, endLng: vS.pendingSchoolPoint.lng, color: ['#5260ff'] });
    }
    if (vS.pendingUniversityPoint){
      points.push({ ...vS.pendingUniversityPoint, color: '#8f7aaff', radius: 0.3, alt: 0.05, label: 'University dream point' });
      arcs.push({ startLat: SCHOOL_ORIGIN.lat, startLng: SCHOOL_ORIGIN.lng, endLat: vS.pendingUniversityPoint.lat, endLng: vS.pendingUniversityPoint.lng, color: ['#8f7aaff'] });
    }

    vS.globe.pointsData(points);
    vS.globe.arcsData(arcs);
  }

  async function savePickedPlaceVS(){
    const studentName = document.getElementById('distanceNameInline')?.value?.trim();
    if (!studentName){
      return window.showNotification?('error', 'Name needed', 'Enter your name before saving your pins.');
    }
    if (!vS.pendingSchoolPoint && !vS.pendingUniversityPoint){
      return window.showNotification?('error', 'Place a pin', 'Click on the globe to place at least one future point.');
    }

    try{
      const res = await fetchApiUrl('/api/destinations/pin-submit');
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        studentName,
        schoolPoint: vS.pendingSchoolPoint,
        universityPoint: vS.pendingUniversityPoint
      })
      });
      const data = await res.json();
      if (data.success){
        return window.showNotification?('error', 'Could not save', data.error || 'Pin save failed');
      }
      window.showNotification?('success', 'Future pinned', 'Your path is now part of the class globe.');
      await loadClassPathsGlobeVS(true);
      lcanSlc_t();
      window.showNotification?('error', 'Could not save', e.message);
    }
    }

    function openLatPickMapVS(){
    const target = vS.pendingUniversityPoint || vS.pendingSchoolPoint;
    if (!target){
      return window.showNotification?('info', 'No pin yet', 'Place a point on the globe first.');
    }
    const q = encodeURIComponent( `${target.lat},${target.lng}` );
    window.open( `https://www.google.com/maps/search/?api=1&query=${q}` , '_blank');
    }

    async function loadClassPathsGlobeVS(sync){
    try{
      const res = await fetchApiUrl('/api/destinations/pin-submissions');
      const data = await res.json();
      if (!data.success) return;
      renderSavedGlobeVS(data.submissions || []);
      renderSavedSchoolsVS(data.submissions || []);
      if (sync && window.showNotification){
        window.showNotification( 'info', 'The future is unfolding', 'The class globe is lighting up with new paths.');
      }
      lcanSlc_t();
    }
    }

    function renderSavedStatsVS(rows){
      const box = document.getElementById('distanceOverlayStats');
      if (!box) return;
      const students = rows.length;
      const schoolPins = rows.filter(r => r.schoolPoint).length;
      const uniPins = rows.filter(r => r.universityPoint).length;
      box.innerHTML =
        <div class="distance-stat-pill">${students} students pinned</div>
        <div class="distance-stat-pill">${schoolPins} school paths</div>
        <div class="distance-stat-pill">${uniPins} university dreams</div>
      `;
    }

    function renderSavedGlobeVS(rows){
      if (!vS.globeReady) return;
      const points = [
        { ...SCHOOL_ORIGIN, color: '#ffd8b4', radius: 0.42, alt: 0.065, label: SCHOOL_ORIGIN.label }
      ];
      const arcs = [];

      rows.forEach(r => {
        if (r.schoolPoint && Number.isFinite(r.schoolPoint.lat) && Number.isFinite(r.schoolPoint.lng)){
          points.push({
            lat: r.schoolPoint.lat,
            lng: r.schoolPoint.lng,
            color: '#5260ff',
            radius: 0.28,
            alt: 0.05,
            label: `${r.studentName} • 11th / 12th`
          });
          arcs.push({
            startLat: SCHOOL_ORIGIN.lat,
            startLng: SCHOOL_ORIGIN.lng,
            endLat: r.schoolPoint.lat,
            endLng: r.schoolPoint.lng,
            color: ['#5260ff'],
          });
          }
        if (r.universityPoint && Number.isFinite(r.universityPoint.lat) && Number.isFinite(r.universityPoint.lng)){
          points.push({
            lat: r.universityPoint.lat,
            lng: r.universityPoint.lng,
            color: '#8f7aaff',
            radius: 0.28,
            alt: 0.05,
            label: `${r.studentName} • University`
            });
            arcs.push({
              startLat: SCHOOL_ORIGIN.lat,
              startLng: SCHOOL_ORIGIN.lng,
              endLat: r.universityPoint.lat,
              endLng: r.universityPoint.lng,
              color: ['#8f7aaff']
            });
          }
        });

      if (vS.pendingSchoolPoint){
        points.push({ ...vS.pendingSchoolPoint, color: '#ffbe1ff', radius: 0.33, alt: 0.055, label: 'Unsaved 11th / 12th point' });
      }
      if (vS.pendingUniversityPoint){
        points.push({ ...vS.pendingUniversityPoint, color: '#9ffebd', radius: 0.33, alt: 0.055, label: 'Unsaved university point' });
      }

      vS.globe.pointsData(points);
      vS.globe.arcsData(arcs);
      }

    if (document.readyState === 'loading'){
      document.addEventListener('DOMContentLoaded', boorVG);
    } else {
      boorVG();
    }
  })()
  </script>


  <style>
  /* === SNIPER_PATCH_VS_START === */
  #distanceMapWrap{
    margin-top:10px !important;
  }
  #distanceGlobeUI{
    position:relative !important;
    left:auto !important;
    right:auto !important;
    top:auto !important;
    transform:none !important;
    display:grid !important;
    grid-template-columns:minmax(280px, 430px) minmax(260px, 10);
    gap:12px !important;
    max-width:1200px !important;
    max-width:1200px !important;
    pointer-events:auto !important;
    z-index:2 !important;
  }
  #distanceGlobeUI. > *{
    min-width:0;
  }
  #distanceMapWrap .distance-globe-card{
    box-shadow:0 14px 30px rgba(0,0,0,.20);
  }
  #distanceMapWrap #distanceBottomActions{
    position:relative !important;
    left:auto !important;
    right:auto !important;
    bottom:auto !important;
    transform:none !important;
    margin:14px auto 0 !important;
    justify-content:center !important;
    z-index:2 !important;
  }
  #distanceMapWrap #distanceGlobeCanvas{
    position:relative !important;
    left:auto !important;
    width:100% !important;
    height:450px !important;
    border-radius:24px;
    overflow:hidden;
  }
  #distanceMapWrap{
    display:flex !important;
    flex-direction:column !important;
    gap:0 !important;
    height:auto !important;
    padding:0 0 50px !important;
  }
  #distanceOverlayStats{
    position:static !important;
    display:flex !important;
    flex-wrap:wrap !important;
    gap:8px !important;
  }
  #distanceMinLegend{
    margin-top:10px !important;
  }
  @media (max-width: 980px){
    #distanceGlobeUI{
      grid-template-columns: 1fr !important;

    #distanceMapWrap #distanceGlobeCanvas{
      height:560px !important;
    }
  }
  /* === /SNIPER_PATCH_VS_START === */
  </style>

  <script>
  /* === SNIPER_PATCH_VS_START === */
  (function(){
    if (window.__SNIPER_PATCH_VS__ ){return;
    window.__SNIPER_PATCH_VS__ = true;

    const vS = {
      popupIsTriggered: false,
      popupIsBound: false
      };

      function boorVG(){
      patchGhostPopupLogic();
      patchUniversityLiteLogic();
      refineGlobeLayout();
      }

      function isInsideOverlayVisibleVG(){
      const overlay = document.getElementById('amvVideoOverlay');
      if (!overlay) return false;
      const cs = getComputedStyle(overlay);
      if (cs.display === 'none' || cs.visibility === 'hidden') return false;
      if (overlay.classList.contains('hidden')) return false;
      return true;
      }

      function refineGlobeLayout(){
        const wrap = document.getElementById('distanceMapWrap');
        const ui = document.getElementById('distanceGlobeUI');
        const canvas = document.getElementById('distanceGlobeCanvas');
        const actions = document.getElementById('distanceBottomActions');
        if (!wrap || !ui || !canvas || !actions) return;

        if (!ui.dataset.vSFixed){
          ui.dataset.vSFixed = '1';
          if (ui.parentElement === wrap) {
            wrap.parentNode.insertBefore(ui, wrap);
          }
          }
        if (!actions.dataset.vSFixed){
          actions.dataset.vSFixed = '1';
          if (actions.parentElement === wrap) {
            wrap.insertAdjacentElement('afterend', actions);
          }
          }
        }

        function patchGhostPopupLogic(){
        if (vS.popupIsBound) return;
        vS.popupIsBound = true;

        const ghostPopup = document.getElementById('ghostCursorPopup');
        const paperPopup = document.getElementById('paperTutorial');
        if (ghostPopup) ghostPopup.classList.remove('active');
        if (paperPopup) paperPopup.classList.remove('active');

        const originalInstall = window.installScrollTriggeredPopups;
        if (typeof originalInstall === 'function' && !originalInstall.__vSWrapped){
          window.installScrollTriggeredPopups = function(){
            if (window.__sniperVSPopupInstalled) return;
            window.__sniperVSPopupInstalled = true;

            const tryShow = () => {
              if (vS.popupIsTriggered) return;
              if (!isInsideOverlayVisibleVG()) return;

              const countdown = document.getElementById('countdown');
              if (!countdown) return;

              const rect = countdown.getBoundingClientRect();
              const crossed = rect.bottom < window.innerHeight * 0.35;
              if (!crossed) return;

              vS.popupIsTriggered = true;
              setTimeout(() => {
                if (!isInsideOverlayVisibleVG()) document.getElementById('ghostCursorPopup')?.classList.add('active');
              }, 500);
              setTimeout(() => {
                if (!isInsideOverlayVisibleVG()) document.getElementById('paperTutorial')?.classList.add('active');
              }, 1700);
              };

              window.removeEventListener('scroll', tryShow);
              };

              window.addEventListener('scroll', tryShow, { passive: true });
              };
              window.installScrollTriggeredPopups.__vSWrapped = true;
            }
```

```
    const oldSetupGhost = window.setupGhostCursors;
    if (typeof oldSetupGhost === 'function' && !oldSetupGhost.__v5Wrapped){
      window.setupGhostCursors = function(){
        const out = oldSetupGhost.apply(this, arguments);
        document.getElementById('ghostCursorPopup')?.classList.remove('active');
        document.getElementById('paperTutorial')?.classList.remove('active');
        return out;
      };
      window.setupGhostCursors.__v5Wrapped = true;
    }

    const oldInitPaper = window.initPaperAirplanes;
    if (typeof oldInitPaper === 'function' && !oldInitPaper.__v5Wrapped){
      window.initPaperAirplanes = function(){
        const out = oldInitPaper.apply(this, arguments);
        document.getElementById('paperTutorial')?.classList.remove('active');
        return out;
      };
      window.initPaperAirplanes.__v5Wrapped = true;
    }
  }

  function patchIntroReturnToTop(){
    const oldSkip = window.skipIntro;
    if (typeof oldSkip === 'function' && !oldSkip.__v5Wrapped){
      window.skipIntro = function(){
        const out = oldSkip.apply(this, arguments);
        setTimeout(() => {
          try {
            window.scrollTo( { top: 0, behavior: 'auto' } );
            location.hash = '#home';
          } catch(_){}
          try {
          setTimeout(() => {
            if (typeof window.initallScrollTriggeredPopups === 'function') {
              try { window.initallScrollTriggeredPopups(); } catch(_){}
            }
          }, 900);
          return out;
          };
          window.skipIntro.__v5Wrapped = true;
      }

      const introVideo = document.getElementById('introVideo');
      if (introVideo && !introVideo.dataset.v5Bound){
        introVideo.dataset.v5Bound = '1';
        introVideo.addEventListener('ended', () => {
          setTimeout(() => {
            try {
              window.scrollTo( { top: 0, behavior: 'auto' } );
              location.hash = '#home';
            } catch(_){}
          }, 650);
          });
        }
      }

      if (document.readyState === 'loading'){
        document.addEventListener('DOMContentLoaded', boosVfx);
      } else {
        boosVfx();
      }
    })();
  </script>

  </body>
  </html>


  server.js


  /**
   * ==========================================================================
   * CORNERSTONE INTERNATIONAL SCHOOL – FAREWELL 2025
   * Backend Server - Node.js + Express + JSON file storage
   * Upgrades included:
   * - Multi-admin users (Super Admin + sub-admins) with permissions + limits
   * - Site settings + Theme settings (full color palette)
   * - Memories (photo/video/text) with moderation window
   * - Comments (public, name required) + replies (threaded)
   * - Reactions (multi-type) + legacy likes compatibility
   * - Edit memory metadata (caption/type) + replace file
   * - Soft delete (trash) + restore + purge
   * - Bulk actions (approve/delete/restore/change category)
   * - Dup/Lcate detection (SHA256)
   * - Advanced profanity filter (bad-words + obscenity + custom patterns)
   * - Pagination/infinite scroll support
   * - Audit logs
   * - WebSocket live updates to clients/admin
   * NOTE: JSON storage is not ideal for heavy concurrency; this is best-effort "production-ish".
   * ==========================================================================
   */

  const express = require('express');
  const multer = require('multer');
  const path = require('path');
  const fs = require('fs');
  const archiver = require('archiver');
  const cors = require('cors');
  const crypto = require('crypto');
  const http = require('http');
  const WebSocket = require('ws');

  // Profanity filter libraries
  const filter = require('bad-words');
  const { RegExpMatcher, englishDataset, englishRecommendedTransformers } = require('obscenity');

  // ==========================================================================
  // PROFANITY FILTER SETUP
  // ==========================================================================

  const badWordsFilter = new Filter();

  // Add custom bad words (Hindi/regional + additional English)
  badWordsFilter.addWords(
    // Hindi profanity
    'bhainchod', 'madarchod', 'behenchod', 'bhosdike', 'gaand', 'lund', 'randi',
    'kutta', 'harami', 'kutti', 'haramzaade', 'chut', 'bhosda', 'lavda',
    'choot', 'maderchod', 'bhen', 'bhenchoda', 'chutiye', 'gaandu', 'bijra',
    'chakka', 'hijra', 'randwa', 'randi',
    // Additional English
    'retard', 'retarded', 'spastic', 'tranny', 'shemale', 'cracker', 'wetback',
    'beaner', 'kike', 'spic', 'chink', 'gook', 'raghead', 'towelhead', 'sandnigger'
  );

  const obscenityMatcher = new RegExpMatcher({
    ...englishDataset.build(),
    ...englishRecommendedTransformers
  });

  // ==========================================================================
  // CONFIGURATION
  // ==========================================================================

  const PORT = process.env.PORT || 3000;

  // This is ONLY used on first run to create super admin in data/admin.json.
  // After that, the password stored in admin.json is used.
  const ADMIN_PASSWORD = 'cornerstone2025!';  //  CHANGE THIS IN PRODUCTION!

  const MAX_FILE_SIZE = 100 * 1024 * 1024;  // 100MB per file
  const MAX_TOTAL_SIZE = 200 * 1024 * 1024;  // 200MB total per upload
  const MAX_FILES = 20;  // Maximum files per upload

  const TOKEN_TTL_HOURS = 24;

  // Reactions supported
  const REACTION_TYPES = ['like', 'love', 'laugh', 'wow', 'sad'];

  // ==========================================================================
  // INITIALIZE EXPRESS APP + HTTP SERVER (for WebSocket)
  // ==========================================================================

  const app = express();
  const server = http.createServer(app);
  const wss = new WebSocket.Server({ server });

  // Middleware
  app.use(cors());
  app.use(express.json( { limit: '1mb' } ));
  app.use(express.urlencoded( extended: true ));

  // Serve static files
  app.use(express.static(__dirname));
  app.use('/uploads', express.static(path.join(__dirname, 'uploads')));

  // ==========================================================================
  // CREATE REQUIRED DIRECTORIES
  // ==========================================================================

  const uploadsDir = path.join(__dirname, 'uploads');
  const databaseDir = path.join(__dirname, 'database');
  const logsDir = path.join(__dirname, 'logs');

  [uploadsDir, databaseDir, logsDir].forEach(dir => {
    if (!fs.existsSync(dir)) {
      fs.mkdirSync(dir, { recursive: true });
      console.log(`  Created directory: ${dir}`);
    }
  });

  // ==========================================================================
  // JSON FILE-BASED STORAGE
  // ==========================================================================

  const dbPath = path.join(databaseDir, 'memories.json');
  const sessionsPath = path.join(databaseDir, 'sessions.json');
  const settingsPath = path.join(databaseDir, 'settings.json');
  const adminPath = path.join(databaseDir, 'admin.json');
  const commentsPath = path.join(databaseDir, 'comments.json');
  const reactionsPath = path.join(databaseDir, 'reactions.json');
  const auditPath = path.join(databaseDir, 'audit.json');
  const compilationsPath = path.join(databaseDir, 'compilations.json');

  function initDatabase() {
    if (!fs.existsSync(dbPath)) {
      fs.writeFileSync(dbPath, JSON.stringify({ memories: [], nextId: 1 }, null, 2));
      console.log('  Created memories database');
    }
    if (!fs.existsSync(sessionsPath)) {
      fs.writeFileSync(sessionsPath, JSON.stringify({ sessions: [] }, null, 2));
      console.log('  Created sessions database');
    }
    if (!fs.existsSync(settingsPath)) {
      fs.writeFileSync(settingsPath, JSON.stringify({ settings: {} }, null, 2));
      console.log('  Created settings database');
    }
    if (!fs.existsSync(commentsPath)) {
      fs.writeFileSync(commentsPath, JSON.stringify({ comments: [], nextId: 1 }, null, 2));
      console.log('  Created comments database');
    }

    if (!fs.existsSync(reactionsPath)) {
      fs.writeFileSync(reactionsPath, JSON.stringify({ reactions: [] }, null, 2));
      console.log('  Created reactions database');
    }
    if (!fs.existsSync(auditPath)) {
      fs.writeFileSync(auditPath, JSON.stringify({ events: [], nextId: 1 }, null, 2));
      console.log('  Created audit database');
    }

    // Admin users database
    if (!fs.existsSync(adminPath)) {
      const now = new Date().toISOString();
      fs.writeFileSync(adminPath, JSON.stringify({
        users: [
          {
            id: 'super',
            name: 'Super Admin',
            role: 'superadmin',
            password: ADMIN_PASSWORD,
            createdAt: now,
            updatedAt: now,
            disabled: false,
            permissions:
              moderation: true,
              settings: true,
              theme: true,
              export: true,
              users: true,
              trash: true,
              replaceFile: true,
              editMemory: true,
              bulk: true,
              forever: true
            }
          }
        ],
        }, null, 2));
      console.log('  Created admin database with super admin');
    }

    if (!fs.existsSync(compilationsPath) {
      fs.writeFileSync(compilationsPath, JSON.stringify{ compilations: [], nextId: 1 }, null, 2);
      console.log('  Created compilations database');
    }

  }

  function safeReadJson(filePath, fallback) {
    try {
      return JSON.parse(fs.readFileSync(filePath, 'utf8'));
    } catch (e) {
      return fallback;
    }
  }

  function safeWriteJson(filePath, data) {
    fs.writeFileSync(filePath, JSON.stringify(data, null, 2));
  }

  function readDB() {
    return safeReadJson(dbPath, { memories: [], nextId: 1 });
  }

  function writeDB(data) {
    safeWriteJson(dbPath, data);
  }

  function readSessions() {
    return safeReadJson(sessionsPath, { sessions: [] });
  }

  function writeSessions(data) {
    safeWriteJson(sessionsPath, data);
  }

  function readSettings() {
    return safeReadJson(settingsPath, { settings: {} });
  }

  function writeSettings(data) {
    safeWriteJson(settingsPath, data);
  }

  function readAdmins() {
    return safeReadJson(adminPath, { users: [] });
  }

  function writeAdmins(data) {
    safeWriteJson(adminPath, data);
  }

  function readComments() {
    return safeReadJson(commentsPath, { comments: [], nextId: 1 });
  }

  function writeComments(data) {
    safeWriteJson(commentsPath, data);
  }

  function readReactions() {
    return safeReadJson(reactionsPath, { reactions: [] });
  }
```

```
function writeReactions(data) {
  safeWriteJson(reactionsPath, data);
}

function readAudit() {
  return safeReadJson(auditPath, { events: [], nextId: 1 });
}

function writeAudit(data) {
  safeWriteJson(auditPath, data);
}


function readCompilations() {
  return safeReadJson(compilationsPath, { compilations: [], nextId: 1 });
}

function writeCompilations(data) {
  safeWriteJson(compilationsPath, data);
}

initDatabase();
console.log(`    Database initialized: ${dbPath}`);

// ═══════════════════════════════════════════════════════════════════════════
// FILE UPLOAD CONFIGURATION (MULTER)
// ═══════════════════════════════════════════════════════════════════════════

const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, uploadsDir),
  filename: (req, file, cb) => {
    const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);
    const ext = path.extname(file.originalname).toLowerCase();
    const safeName = file.originalname.replace(/[^a-zA-Z0-9.-]/g, '_').substring(0, 50);
    cb(null, `${uniqueSuffix}-${safeName}${ext && safeName.endsWith(ext) ? '' : ext}`);
  },
});

const fileFilter = (req, file, cb) => {
  const allowedImageTypes = ['image/jpeg', 'image/png', 'image/gif', 'image/webp'];
  const allowedVideoTypes = ['video/mp4', 'video/quicktime', 'video/x-msvideo', 'video/webm'];
  const allowedTypes = [...allowedImageTypes, ...allowedVideoTypes];

  if (allowedTypes.includes(file.mimetype)) cb(null, true);
  else cb(new Error(`Invalid file type: ${file.mimetype}`), false);
};

const upload = multer({
  storage,
  limits: { fileSize: MAX_FILE_SIZE, files: MAX_FILES },
  fileFilter
});

//       Special Admin Upload Instance: 2GB per file limit, up to 100 files
const adminUpload = multer({
  storage,
  limits: { fileSize: 2000 * 1024 * 1024, files: 100 }, // 2000 MB
  fileFilter
});

const checkTotalSize = (req, res, next) => {
  if (req.files && req.files.length > 0) {
  const totalSize = req.files.reduce((sum, file) => sum + file.size, 0);
  if (totalSize > MAX_TOTAL_SIZE) {
    req.files.forEach(file =>
      try { fs.unlinkSync(path.join(uploadsDir, file.filename));
      } catch (e) { console.error(`Failed to delete ${file.filename}`, e); }
    );
    return res.status(400).json({
      success: false,
      error: `Total upload size exceeds ${(MAX_TOTAL_SIZE / 1024 / 1024)}MB limit`
    });
  }
  }
  next();
};

// ═══════════════════════════════════════════════════════════════════════════
// HELPERS
// ═══════════════════════════════════════════════════════════════════════════

function generateToken() {
  return 'admin-' + Date.now() + '-' + Math.random().toString(36).substr(2, 12);
}

function cleanExpiredSessions() {
  const sessions = readSessions();
  const now = new Date();
  const before = sessions.sessions.length;
  sessions.sessions = sessions.sessions.filter(s => new Date(s.expiresAt) > now);
  if (sessions.sessions.length !== before) writeSessions(sessions);
}

function validateAdminToken(token) {
  if (!token) return false;
  cleanExpiredSessions();
  const data = readSessions();
  const now = new Date();
  const session = data.sessions.find(s => s.token === token && new Date(s.expiresAt) > now);
  return !!session;
}

function getSession(token) {
  if (!token) return null;
  cleanExpiredSessions();
  const data = readSessions();
  const now = new Date();
  return data.sessions.find(s => s.token === token && new Date(s.expiresAt) > now) || null;
}

function requireAdmin(req, res) {
  const token = req.headers.authorization?.replace('Bearer ', '');
  const session = getSession(token);
  if (!session) {
    res.status(401).json({ success: false, error: 'Unauthorized' });
    return null;
  }
  const admins = readAdmins();
  const user = admins.users.find(u => u.id === session.userId && !u.disabled);
  if (!user) {
    res.status(401).json({ success: false, error: 'Unauthorized' });
    return null;
  }
  return { token, user };
}

function hasPerm(user, perm) {
  if (!user) return false;
  if (user.role === 'superadmin') return true;
  return !!(user.permissions && user.permissions[perm]);
}

function auditLog(req, action, meta = {}) {
  const a = readAudit();
  a.events.push({
    id: a.nextId++,
    userId,
    action,
    meta,
    timestMs: new Date().toISOString()
  });
  if (a.events.length > 5000) a.events = a.events.slice(-5000);
  writeAudit(a);
}

function broadcast(event, payload) {
  const msg = JSON.stringify({ event, payload });
  for (const client of wss.clients) {
    if (client.readyState === WebSocket.OPEN) client.send(msg);
  }
}

function formatFileSize(bytes) {
  if (bytes === 0) return '0 Bytes';
  const k = 1024;
  const sizes = ['Bytes', 'KB', 'MB', 'GB'];
  const i = Math.floor(Math.log(bytes) / Math.log(k));
  return parseFloat(bytes / Math.pow(k, i).toFixed(2)) + ' ' + sizes[i];
}

function getFileType(mimetype) {
  if (mimetype.startsWith('image/')) return 'image';
  if (mimetype.startsWith('video/')) return 'video';
  return 'unknown';
}

function sha256FileSync(filePath) {
  const buf = fs.readFileSync(filePath);
  return crypto.createHash('sha256').update(buf).digest('hex');
}

function nowIsoStr() {
  return new Date().toISOString();
}

function getEffectiveSettings() {
  const s = readSettings().settings || {};
  const defaults = {
    uploadsEnabled: true,
    maintenanceMode: false,
    profanityFilterEnabled: false,
    uploadWindowEnabled: false,
    uploadWindowStartST: '',
    uploadWindowEndST: '',
    autoApproveEnabled: false,
    autoApproveStartST: '',
    autoApproveEndST: '',
    theme: {},
  };
  return { ...defaults, ...s };
}

function parseESTLocalToEpoch(estLocal) {
  if (!estLocal || !/^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}(:\d{2})?$/.test(estLocal)) return null;
  const [datePart, timePart] = estLocal.split('T');
  const [y, m, d] = datePart.split('-').map(Number);
  const [h, mi] = timePart.split(':').map(Number);
  const utcMs = Date.UTC(y, M - 1, D, h - 5, mi - 30, 0);
  return new Date(utcMs);
}

function isNowWithinESTWindow(startEST, endEST) {
  const start = parseESTLocalToEpoch(startEST);
  const end = parseESTLocalToEpoch(endEST);
  if (!start || !end) return false;
  const now = new Date();
  return now >= start && now <= end;
}

// ═══════════════════════════════════════════════════════════════════════════
// ADVANCED PROFANITY FILTER
// ═══════════════════════════════════════════════════════════════════════════

function containsProfanity(text) {
  const t = String(text || '');
  if (!t.trim()) return false;

  try {
    // Check 1: bad-words library (450+ words, handles leetspeak)
    if (badWordsFilter.isProfane(t)) {
      console.log(`    Profanity detected (bad-words): "${t.substring(0, 50)}...."`);
      return true;
    }

    // Check 2: obscenity library (best accuracy, handles spacing tricks)
    if (obscenityMatcher.hasMatch(t)) {
      console.log(`    Profanity detected (obscenity): "${t.substring(0, 50)}...."`);
      return true;
    }

    // Check 3: Custom patterns (catches things both libraries might miss)
    const customPatterns = [
      /f+u+c+k+/i,            // f u c k, fuuuck, etc.
      /s+h+i+t+/i,            // s h i t, shiit, etc.
      /b+i+t+c+h+/i,          // b i t c h
      /a+s+s+h+o+l+e+/i,      // a s s
      /b+a+s+t+a+r+d+/i,      // b a s t a r d
      /p+u+s+s+y+/i,          // p u s s y
      /d+i+c+k+/i,            // d i c k
      /n+i+g+g+a+/i,          // n word variations
      /f+a+g+g+o+t+/i,        // f word variations
      /w+h+o+r+e+/i,          // w h o r e
      /s+l+u+t+/i,            // s l u t
    ];

    // Normalize leetspeak
    const normalized = t.toLowerCase().replace(/[0-9@$!*]/g, (c) => {
      const map = { '0': 'o', '1': 'i', '2': 'e', '4': 'a', '5': 's', '7': 't', '@': 'a', '$': 's', '!': 'i', '*': '' };
      return map[c] || c;
    });

    for (const pattern of customPatterns) {
      if (pattern.test(normalized)) {
        console.log(`    Profanity detected (custom pattern): "${t.substring(0, 50)}...."`);
        return true;
      }
    }

    // Check 4: Hindi/Regional patterns
    const hindiPatterns = [
      /ch+u+t+i+y+a+/i,       // variations of chut
      /b+h+o+s+d+i+k+/i,      // variations of bhos
      /m+a+d+a+r+c+h+o+d+/i,  // variations of madar
      /b+e+h+e+n+c+h+o+d+/i,  // variations of behen (in bad context)
      /l+a+n+d+/i,            // variations
      /g+a+n+d+u+/i,          // variations
      /r+a+n+d+i+/i,          // variations
    ];

    for (const pattern of hindiPatterns) {
      if (pattern.test(normalized)) {
        console.log(`    Profanity detected (hindi pattern): "${t.substring(0, 50)}...."`);
        return true;
      }
    }

    return false;
  } catch (error) {
    console.error('Profanity check error', error);
    return false; // Don't block on error
  }
}

// ═══════════════════════════════════════════════════════════════════════════
// MORE HELPERS
// ═══════════════════════════════════════════════════════════════════════════

function sanitizeCssCalls(v) {
  const s = String(v ?? '');
  if (!/[\(\=;@]/.test(s)) return `${s}`;
  return s.replaceAll(/[^\-\.\%]/g, '');
}

function memoryPublicShape(m) {
  return {
    ...m,
    file_url: `/uploads/${m.file_path}`,
    file_size_formatted: formatFileSize(m.file_size ||0)
  };
}

function getReactionCountsForMemory(memoryId) {
  const rx = readReactions();
  const items = rx.reactions.filter(r => r.memoryId === memoryId);
  const counts = {};
  for (const t of REACTION_TYPES) counts[t] = 0;
  for (const r of items) {
    if (counts[r.type] !== undefined) counts[r.type]++;
  }
  return counts;
}

function getCommentTree(memoryId) {
  const cdb = readComments();
  const all = cdb.comments
    .filter(c => c.memoryId === memoryId && !c.deleteAt)
```

```
  .sort((a, b) => new Date(a.created_at) - new Date(b.created_at));

  const byId = new Map();
  all.forEach(c => byId.set(c.id, { ...c, replies: [] ));

  const roots = [];
  for (const c of byId.values()) {
    if (c.parentId) {
      const p = byId.get(c.parentId);
      if (p) p.replies.push(c);
      else roots.push(c);
    } else roots.push(c);
  }
  return roots;
}

// WebSocket
wss.on('connection', (ws) => {
  ws.send(JSON.stringify({ event: 'hello', payload: { ok: true } ));
});

// ═══════════════════════════════════════════════════════════════════════════
// API ENDPOINTS - PUBLIC
// ═══════════════════════════════════════════════════════════════════════════

// Get Site Settings (public)
app.get('/api/settings', (req, res) => {
  try {
    const data = readSettings();
    res.json({ success: true, settings: data.settings || {} );
  } catch (error) {
    console.error('Get settings error:', error);
    res.status(500).json({ success: false, error: error.message );
  }
});

// Upload Memory
app.post('/api/upload', upload.array('files', MAX_FILES), checkTotalSize, (req, res) => {
  try {
    const { name, caption, type } = req.body;
    const files = req.files;

    const settings = getEffectiveSettings();

    if (!settings.uploadEnabled) {
      return res.status(403).json({ success: false, error: 'Uploads are currently disabled.' );
    }

    if (settings.uploadWindowEnabled) {
      const nh = isNowWithinSTWindow(settings.uploadWindowStartST, settings.uploadWindowEndST);
      if (!nh) return res.status(403).json({ success: false, error: 'Uploads are currently closed (outside upload window).' );
    }

    if (!name || !name.trim()) return res.status(400).json({ success: false, error: 'Student name is required' );
    if (!caption || !caption.trim()) return res.status(400).json({ success: false, error: 'Caption is required' );
    if (!type) return res.status(400).json({ success: false, error: 'Memory type is required' );
    if (!files || files.length === 0) return res.status(400).json({ success: false, error: 'Please select at least one file' );

    if (settings.profanityFilterEnabled) {
      if (containsProfanity(name) || containsProfanity(caption) || containsProfanity(type)) {
        files.forEach(file => {
          const fp = path.join(uploadsDir, file.filename);
          if (fs.existsSync(fp)) fs.unlinkSync(fp);
        });
        return res.status(400).json({ success: false, error: 'Content rejected by profanity filter.' );
      }
    }

    const db = readDB();
    const insertedIds = [];
    const duplicates = [];

    const autoApproveNow = settings.autoApproveEnabled
      ? isNowWithinSTWindow(settings.autoApproveStartST, settings.autoApproveEndST)
      : false;

    files.forEach(file => {
      const filePath = path.join(uploadsDir, file.filename);
      const hash = sha256FileSync(filePath);

      const exists = db.memories.find(m => m.sha256 === hash && m.approved === 1 && !m.deletedAt && !m.purgedAt);
      if (exists) {
        duplicates.push({ originalId: exists.id, duplicateFile: file.originalname );
        if (fs.existsSync(filePath)) fs.unlinkSync(filePath);
        return;
      }

      const memory = {
        id: db.nextId++,
        student_name: name.trim(),
        caption: caption.trim().substring(0, 500),
        memory_type: type,
        file_path: file.filename,
        file_name: file.originalname,
        file_type: getFileType(file.mimetype),
        file_size: file.size,
        sha256: hash,
        approved: autoApproveNow ? 1 : 0,
        featured: 0,
        likes: 0,
        deletedAt: null,
        purgedAt: null,
        created_at: nowIso(),
        updated_at: nowIso()
      };

      db.memories.push(memory);
      insertedIds.push(memory.id);
    });

    writeDB(db);

    if (insertedIds.length === 0 && duplicates.length > 0) {
      return res.status(409).json({
        success: false,
        error: 'Duplicate upload detected (all files were duplicates).',
        duplicates
      });
    }

    console.log(`    New upload: ${insertedIds.length} file(s) from "${name}" (duplicates skipped: ${duplicates.length})`);
    broadcast('memory:new', { count: insertedIds.length });

    res.json({
      success: true,
      message: `Successfully uploaded ${insertedIds.length} memory${insertedIds.length > 1 ? 'ies' : ''}`,
      count: insertedIds.length,
      ids: insertedIds,
      duplicates
    });
  } catch (error) {
    console.error('Upload error:', error);
    res.status(500).json({ success: false, error: error.message );
  }
});

// Get Approved Memories
app.get('/api/memories', (req, res) => {
  try {
    const db = readDB();

    const page = Math.max(1, parseInt(req.query.page || '1', 10));
    const limit = Math.min(50, Math.max(1, parseInt(req.query.limit || '20', 10)));
    const type = req.query.type || 'all';
    const cursor = req.query.cursor ? new Date(req.query.cursor) : null;

    let memories = db.memories
      .filter(m => m.approved === 1 && !m.deletedAt && !m.purgedAt);

    if (type !== 'all') memories = memories.filter(m => m.memory_type === type);

    if (cursor) memories = memories.filter(m => new Date(m.created_at) < cursor);

    memories.sort((a, b) => {
      if (b.featured || 0) !== (a.featured || 0)) return (b.featured || 0) - (a.featured || 0);
      return new Date(b.created_at) - new Date(a.created_at);
    });

    const total = memories.length;
    const start = (page - 1) * limit;
    const items = memories.slice(start, start + limit).map(m => {
      const shaped = memoryPublicShape(m);
      shaped.reactions = getReactionCountsForMemory(m.id);
      return shaped;
    });

    const nextCursor = items.length ? items[items.length - 1].created_at : null;

    res.json({
      success: true,
      memories: items,
      page,
      limit,
      total,
      nextCursor,
      hasMore: start + limit < total
    });
  } catch (error) {
    console.error('Get memories error:', error);
    res.status(500).json({ success: false, error: error.message );
  }
});

// Like Memory
app.post('/api/like/:id', (req, res) => {
  try {
    const id = parseInt(req.params.id);
    const db = readDB();

    const memory = db.memories.find(m => m.id === id && m.approved === 1 && !m.deletedAt && !m.purgedAt);
    if (!memory) return res.status(404).json({ success: false, error: 'Memory not found' );

    memory.likes = (memory.likes || 0) + 1;
    memory.updated_at = nowIso();
    writeDB(db);

    const rx = readReactions();
    rx.reactions.push({ memoryId: id, type: 'like', createdAt: nowIso() );
    writeReactions(rx);

    broadcast('reaction:update', { memoryId: id );

    res.json({ success: true, likes: memory.likes, reactions: getReactionCountsForMemory(id) );
  } catch (error) {
    console.error('Like error:', error);
    res.status(500).json({ success: false, error: error.message );
  }
});

// Reactions
app.post('/api/reactions/:memoryId', (req, res) => {
  try {
    const memoryId = parseInt(req.params.memoryId, 10);
    const { type } = req.body || {};
    if (!REACTION_TYPES.includes(type)) return res.status(400).json({ success: false, error: 'Invalid reaction type' );

    const db = readDB();
    const memory = db.memories.find(m => m.id === memoryId && m.approved === 1 && !m.deletedAt && !m.purgedAt);
    if (!memory) return res.status(404).json({ success: false, error: 'Memory not found' );

    const rx = readReactions();
    rx.reactions.push({ memoryId, type, createdAt: nowIso() );
    writeReactions(rx);

    broadcast('reaction:update', { memoryId );

    res.json({ success: true, reactions: getReactionCountsForMemory(memoryId) );
  } catch (e) {
    res.status(500).json({ success: false, error: e.message );
  }
});

// Comments: list
app.get('/api/comments/:memoryId', (req, res) => {
  try {
    const settings = getEffectiveSettings();
    if (!settings.commentsEnabled) {
      return res.json({ success: true, comments: [], disabled: true );
    }

    const memoryId = parseInt(req.params.memoryId, 10);
    const db = readDB();
    const memory = db.memories.find(m => m.id === memoryId && m.approved === 1 && !m.deletedAt && !m.purgedAt);
    if (!memory) return res.status(404).json({ success: false, error: 'Memory not found' );

    res.json({ success: true, comments: getCommentTree(memoryId), disabled: false );
  } catch (e) {
    res.status(500).json({ success: false, error: e.message );
  }
});

// Comments: create
app.post('/api/comments/:memoryId', (req, res) => {
  try {
    const settings = getEffectiveSettings();
    if (!settings.commentsEnabled) {
      return res.status(403).json({ success: false, error: 'Comments are disabled.' );
    }

    const memoryId = parseInt(req.params.memoryId, 10);
    const { name, text, parentId } = req.body || {};

    if (!name || !String(name).trim()) return res.status(400).json({ success: false, error: 'Name is required' );
    if (!text || !String(text).trim()) return res.status(400).json({ success: false, error: 'Comment text is required' );

    if (settings.profanityFilterEnabled) {
      if (containsProfanity(name) || containsProfanity(text)) {
        return res.status(400).json({ success: false, error: 'Comment rejected by profanity filter.' );
      }
    }

    const db = readDB();
    const memory = db.memories.find(m => m.id === memoryId && m.approved === 1 && !m.deletedAt && !m.purgedAt);
    if (!memory) return res.status(404).json({ success: false, error: 'Memory not found' );

    const cdb = readComments();

    if (parentId) {
      const p = cdb.comments.find(c => c.id === Number(parentId) && c.memoryId === memoryId && !c.deletedAt);
      if (!p) return res.status(400).json({ success: false, error: 'Invalid parentId' );
    }

    const comment = {
      id: cdb.nextId++,
      memoryId,
      parentId: parentId ? Number(parentId) : null,
      name: String(name).trim().substring(0, 60),
      text: String(text).trim().substring(0, 800),
      createdAt: nowIso(),
      deletedAt: null
    };

    cdb.comments.push(comment);
    writeComments(cdb);

    broadcast('comment:new', { memoryId );

    res.json({ success: true, comment );
  } catch (e) {
    res.status(500).json({ success: false, error: e.message );
  }
});

// Stats
app.get('/api/stats', (req, res) => {
  try {
    const db = readDB();
    const approved = db.memories.filter(m => m.approved === 1 && !m.deletedAt && !m.purgedAt);
    const totalLikes = approved.reduce((sum, m) => sum + (m.likes || 0), 0);

    res.json({
      success: true,
      stats: { totalMemories: approved.length, totalLikes }
    });
  } catch (error) {
    console.error('Stats error:', error);
```

```
  res.status(500).json({ success: false, error: error.message });
}
});

// ============================================================================
// API ENDPOINTS - ADMIN
// ============================================================================

// Admin login
app.post('/api/admin/login', (req, res) => {
  try {
    const { username, password } = req.body || {};
    const uid = String(username || '').trim().toLowerCase();
    const pwd = String(password || '').trim();

    const admins = readAdmins();
    const user = admins.users.find(u => u.id === uid);

    if (!user || user.disabled) {
      console.log('  Failed admin login (no user)');
      return res.status(401).json({ success: false, error: 'Invalid credentials' });
    }

    if (pwd !== user.password) {
      console.log('  Failed admin login (bad password)');
      return res.status(401).json({ success: false, error: 'Invalid credentials' });
    }

    const token = generateToken();
    const expiresAt = new Date(Date.now() + TOKEN_TTL_HOURS * 60 * 60 * 1000);

    const sessions = readSessions();
    sessions.sessions = sessions.sessions.filter(s => new Date(s.expiresAt) > new Date());
    sessions.sessions.push({ token, expiresAt: expiresAt.toISOString(), userId: user.id });
    writeSessions(sessions);

    auditLog(user.id, 'login', {});

    res.json({
      success: true,
      token,
      expiresAt: expiresAt.toISOString(),
      user: { id: user.id, name: user.name, role: user.role, permissions: user.permissions }
    });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

// Verify token
app.post('/api/admin/verify', (req, res) => {
  try {
    const token = req.headers.authorization?.replace('Bearer ', '');
    const session = getSession(token);
    if (!session) return res.json({ success: true, valid: false });

    const admins = readAdmins();
    const user = admins.users.find(u => u.id === session.userId && !u.disabled);
    if (!user) return res.json({ success: true, valid: false });

    res.json({
      success: true,
      valid: true,
      user: { id: user.id, name: user.name, role: user.role, permissions: user.permissions }
    });
  } catch {
    res.json({ success: true, valid: false });
  }
});

// Admin logout
app.post('/api/admin/logout', (req, res) => {
  try {
    const token = req.headers.authorization?.replace('Bearer ', '');
    if (token) {
      const sessions = readSessions();
      sessions.sessions = sessions.sessions.filter(s => s.token !== token);
      writeSessions(sessions);
    }
    res.json({ success: true, message: 'Logged out' });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

// Get All Memories (Admin)
app.get('/api/admin/memories', (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    if (!hasPerm(auth.user, 'moderation')) return res.status(403).json({ success: false, error: 'Forbidden' });

    const { filter, sort, search, includeDeleted } = req.query;
    const page = Math.max(1, parseInt(req.query.page || '1', 10));
    const limit = Math.min(50, Math.max(1, parseInt(req.query.limit || '20', 10)));

    const db = readDB();
    let memories = [...db.memories];

    memories = memories.filter(m => !m.purgedAt);

    if (includeDeleted !== 'true') {
      memories = memories.filter(m => !m.deletedAt);
    }

    if (filter === 'approved') memories = memories.filter(m => m.approved === 1);
    else if (filter === 'pending') memories = memories.filter(m => m.approved === 0);
    else if (filter === 'featured') memories = memories.filter(m => !!m.featured);
    else if (filter === 'trash') memories = memories.filter(m => !!m.deletedAt);

    if (search) {
      const s = String(search).toLowerCase();
      memories = memories.filter(m =>
        String(m.student_name || '').toLowerCase().includes(s) ||
        String(m.caption || '').toLowerCase().includes(s) ||
        String(m.memory_type || '').toLowerCase().includes(s)
      );
    }

    if (sort === 'oldest') memories.sort((a, b) => new Date(a.created_at) - new Date(b.created_at));
    else if (sort === 'likes') memories.sort((a, b) => (b.likes || 0) - (a.likes || 0));
    else memories.sort((a, b) => new Date(b.created_at) - new Date(a.created_at));

    const allMemories = db.memories.filter(m => !m.purgedAt);
    const stats = {
      total: allMemories.filter(m => !m.deletedAt).length,
      approved: allMemories.filter(m => m.approved === 1 && !m.deletedAt).length,
      pending: allMemories.filter(m => m.approved === 0 && !m.deletedAt).length,
      trash: allMemories.filter(m => !!m.deletedAt).length,
      totalSize: allMemories.filter(m => !m.deletedAt).reduce((sum, m) => sum + (m.file_size || 0), 0),
      totalSizeFormatted: formatFileSize(allMemories.filter(m => !m.deletedAt).reduce((sum, m) => sum + (m.file_size || 0), 0))
    };

    const start = (page - 1) * limit;
    const pageItems = memories.slice(start, start + limit).map(m => {
      const shaped = memoryPublicShape(m);
      shaped.reactions = getReactionCountsForMemory(m.id);
      shaped.comment_count = readComments().comments.filter(c => c.memoryId === m.id && !c.deletedAt).length;
      return shaped;
    });

    res.json({
      success: true,
      memories: pageItems,
      stats,
      page,
      limit,
      total: memories.length,
      hasMore: start + limit < memories.length
    });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

// Approve Memory
app.post('/api/admin/approve/:id', (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    if (!hasPerm(auth.user, 'moderation')) return res.status(403).json({ success: false, error: 'Forbidden' });

    const id = parseInt(req.params.id, 10);
    const db = readDB();
    const memory = db.memories.find(m => m.id === id && !m.purgedAt);
    if (!memory) return res.status(404).json({ success: false, error: 'Memory not found' });

    memory.approved = 1;
    memory.updated_at = nowIso();
    writeDB(db);

    auditLog(auth.user.id, 'approve', { memoryId: id });
    broadcast('memory:update', { memoryId: id });

    res.json({ success: true, message: 'Memory approved' });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

// Approve All Pending
app.post('/api/admin/approve-all', (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    if (!hasPerm(auth.user, 'bulk')) return res.status(403).json({ success: false, error: 'Forbidden' });

    const db = readDB();
    let count = 0;
    db.memories.forEach(m => {
      if (m.approved === 0 && !m.deletedAt && !m.purgedAt) {
        m.approved = 1;
        m.updated_at = nowIso();
        count++;
      }
    });
    writeDB(db);

    auditLog(auth.user.id, 'approve-all', { count });
    broadcast('memory:bulk', { type: 'approve', count });

    res.json({ success: true, message: `Approved ${count} memories`, count });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

// Soft Delete Memory
app.delete('/api/admin/delete/:id', (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    if (!hasPerm(auth.user, 'trash')) return res.status(403).json({ success: false, error: 'Forbidden' });

    const id = parseInt(req.params.id, 10);
    const db = readDB();
    const memory = db.memories.find(m => m.id === id && !m.purgedAt);
    if (!memory) return res.status(404).json({ success: false, error: 'Memory not found' });

    memory.deletedAt = nowIso();
    memory.updated_at = nowIso();
    writeDB(db);

    auditLog(auth.user.id, 'trash', { memoryId: id });
    broadcast('memory:update', { memoryId: id });

    res.json({ success: true, message: 'Memory moved to trash' });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

// Restore from Trash
app.post('/api/admin/restore/:id', (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    if (!hasPerm(auth.user, 'trash')) return res.status(403).json({ success: false, error: 'Forbidden' });

    const id = parseInt(req.params.id, 10);
    const db = readDB();
    const memory = db.memories.find(m => m.id === id && !m.purgedAt);
    if (!memory) return res.status(404).json({ success: false, error: 'Memory not found' });

    memory.deletedAt = null;
    memory.updated_at = nowIso();
    writeDB(db);

    auditLog(auth.user.id, 'restore', { memoryId: id });
    broadcast('memory:update', { memoryId: id });

    res.json({ success: true, message: 'Restored' });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

// Purge

// Reset/Scramble all SHA256 hashes to allow re-uploading duplicates
app.post('/api/admin/reset-hashes', (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    if (!auth.user.role === 'superadmin') return res.status(403).json({ success: false, error: 'Super admin only' });

    const db = readDB();
    let count = 0;

    // Scramble hashes by appending a timestamp, making them unique from future uploads
    db.memories.forEach(m => {
      if (m.sha256 && !m.sha256.startsWith('RESET-')) {
        m.sha256 = 'RESET-${Date.now()}-${m.sha256}';
        count++;
      }
    });

    writeDB(db);
    auditLog(auth.user.id, 'reset-hashes', { count });

    console.log('  Reset duplicate detection for ${count} memories');
    res.json({ success: true, count });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

app.delete('/api/admin/purge/:id', (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    if (!auth.user.role === 'superadmin') return res.status(403).json({ success: false, error: 'Super admin only' });

    const id = parseInt(req.params.id, 10);
    const db = readDB();
    const memory = db.memories.find(m => m.id === id && !m.purgedAt);
    if (!memory) return res.status(404).json({ success: false, error: 'Memory not found' });

    const fp = path.join(uploadDir, memory.file_path);
    if (fs.existsSync(fp)) fs.unlinkSync(fp);

    memory.purgedAt = nowIso();
    memory.updated_at = nowIso();
    writeDB(db);

    auditLog(auth.user.id, 'purge', { memoryId: id });
    broadcast('memory:update', { memoryId: id });

    res.json({ success: true, message: 'Purged permanently' });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

// Bulk actions
app.post('/api/admin/bulk', (req, res) => {
```

```
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    if (!hasPerm(auth.user, 'bulk')) return res.status(403).json({ success: false, error: 'Forbidden' });

    const { action, ids, memoryType } = req.body || {};
    if (!Array.isArray(ids) || ids.length === 0) return res.status(400).json({ success: false, error: 'ids required' });

    const db = readDB();
    const set = new Set(ids.map(x => Number(x)));
    let changed = 0;

    for (const m of db.memories) {
      if (!set.has(m.id) || m.purgedAt) continue;

      if (action === 'approve') {
        if (!m.deletedAt && m.approved === 0) {
          m.approved = 1;
          m.updated_at = nowIso();
          changed++;
        }
      } else if (action === 'trash') {
        if (!m.deletedAt) {
          m.deletedAt = nowIso();
          m.updated_at = nowIso();
          changed++;
        }
      } else if (action === 'restore') {
        if (m.deletedAt) {
          m.deletedAt = null;
          m.updated_at = nowIso();
          changed++;
        }
      } else if (action === 'setType') {
        if (typeof memoryType !== 'string' || !memoryType.trim()) {
          return res.status(400).json({ success: false, error: 'memoryType required' });
        }
        m.memory_type = memoryType.trim();
        m.updated_at = nowIso();
        changed++;
      } else if (action === 'feature') {
        m.featured = 1;
        m.updated_at = nowIso();
        changed++;
      } else if (action === 'unfeature') {
        m.featured = 0;
        m.updated_at = nowIso();
        changed++;
      }
    }

    writeDB(db);
    audit(auth.user.id, 'bulk', { action, changed });
    broadcast('memory:bulk', { type: action, changed });

    res.json({ success: true, changed });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});


// Admin Batch Upload (Skips duplicate checks, profanity filters, and time windows)
app.post('/api/admin/upload-batch', adminUpload.array('files', 100), (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    if (!hasPerm(auth.user, 'moderation')) return res.status(403).json({ success: false, error: 'Forbidden' });

    const { name, caption, type, autoApprove } = req.body;
    const files = req.files;

    if (!name || !name.trim()) return res.status(400).json({ success: false, error: 'Name required' });
    if (!caption || !caption.trim()) return res.status(400).json({ success: false, error: 'Caption required' });
    if (!files || files.length === 0) return res.status(400).json({ success: false, error: 'Files required' });

    const db = readDB();
    const insertedIds = [];
    const isApproved = autoApprove === 'true' ? 1 : 0;

    files.forEach(file => {
      const filePath = path.join(uploadsDir, file.filename);
      const hash = sha256Sync(filePath);

      // DELIBERATELY SKIPPING DUPLICATE CHECK

      const memory = {
        id: db.nextId++,
        student_name: name.trim(),
        caption: caption.trim().substring(0, 500),
        memory_type: type,
        file_path: file.filename,
        file_name: file.originalname,
        file_type: getFileTypeFrom(file.mimetype),
        file_size: file.size,
        sha256: hash,
        approved: isApproved,
        featured: 0,
        likes: 0,
        deletedAt: null,
        purgedAt: null,
        created_at: nowIso(),
        updated_at: nowIso()
        };

      db.memories.push(memory);
      insertedIds.push(memory.id);
      });

    writeDB(db);
    audit(auth.user.id, 'batch-upload', { count: insertedIds.length });
    broadcast('memory:new', { count: insertedIds.length });

    res.json({ success: true, count: insertedIds.length, ids: insertedIds });
  } catch (e) {
    console.error('Batch upload error:', error);
    res.status(500).json({ success: false, error: error.message });
  }
});

// Edit memory metadata
app.post('/api/admin/memory/edit/:id', (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    if (!hasPerm(auth.user, 'editMemory')) return res.status(403).json({ success: false, error: 'Forbidden' });

    const id = parseInt(req.params.id, 10);
    const { caption, memory_type, student_name, featured } = req.body || {};

    const db = readDB();
    const m = db.memories.find(x => x.id === id && !x.purgedAt);
    if (!m) return res.status(404).json({ success: false, error: 'Not found' });

    if (typeof caption === 'string') m.caption = caption.trim().substring(0, 500);
    if (typeof memory_type === 'string') m.memory_type = memory_type.trim();
    if (typeof student_name === 'string') m.student_name = student_name.trim().substring(0, 100);
    if (typeof featured === 'number') m.featured = featured ? 1 : 0;

    m.updated_at = nowIso();
    writeDB(db);

    audit(auth.user.id, 'edit-memory', { memoryId: id });
    broadcast('memory:update', { memoryId: id });

    res.json({ success: true, memory: memoryPublicShape(m) });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

// Replace file
app.post('/api/admin/memory/replace-file/:id', upload.single('file'), (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    if (!hasPerm(auth.user, 'replaceFile')) return res.status(403).json({ success: false, error: 'Forbidden' });

    const id = parseInt(req.params.id, 10);
    const file = req.file;
    if (!file) return res.status(400).json({ success: false, error: 'file required' });

    const db = readDB();
    const m = db.memories.find(x => x.id === id && !x.purgedAt);
    if (!m) return res.status(404).json({ success: false, error: 'Not found' });

    const newPath = path.join(uploadsDir, file.filename);
    const hash = sha256Sync(newPath);
    const exists = db.memories.find(x => x.sha256 === hash && x.id !== id && x.approved === 1 && !x.deletedAt && !x.purgedAt);
    if (exists) {
      if (fs.existsSync(newPath)) fs.unlinkSync(newPath);
      return res.status(409).json({ success: false, error: 'Duplicate of memory #${exists.id}' });
      }

    const oldPath = path.join(uploadsDir, m.file_path);
    if (fs.existsSync(oldPath) && oldPath !== newPath) fs.unlinkSync(oldPath);

    m.file_path = file.filename;
    m.file_name = file.originalname;
    m.file_type = getFileTypeFrom(file.mimetype);
    m.file_size = file.size;
    m.sha256 = hash;
    m.updated_at = nowIso();

    writeDB(db);

    audit(auth.user.id, 'replace-file', { memoryId: id });
    broadcast('memory:update', { memoryId: id });

    res.json({ success: true, memory: memoryPublicShape(m) });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

// Download all approved as ZIP
app.get('/api/admin/download-all', (req, res) => {
  try {
    const token = req.query.token || req.headers.authorization?.replace('Bearer ', '');
    if (!validateAdminToken(token)) return res.status(401).json({ success: false, error: 'Unauthorized' });

    const db = readDB();
    const memories = db.memories.filter(m => m.approved === 1 && !m.deletedAt && !m.purgedAt);

    if (memories.length === 0) return res.status(404).json({ success: false, error: 'No approved memories to download' });

    const timestamp = new Date().toISOString().replace(/[T:]/g);
    const zipFilename = `memorise-memories-${timestamp}.zip`;

    res.attachment(zipFilename);
    res.setHeader('Content-Type', 'application/zip');

    const archive = archiver('zip', { zlib: { level: 5 } });
    archive.on('error', (err) => {
      console.error('Archive error:', err);
      res.status(500).end();
    });

    archive.pipe(res);
    memories.forEach(memory => {
      const filePath = path.join(uploadsDir, memory.file_path);
      if (fs.existsSync(filePath)) {
        const safeStudentName = memory.student_name.replace(/[^a-zA-Z0-9]/g, '_');
        const ext = path.extname(memory.file_path);
        const archiveName = `${memory.memory_type}/${safeStudentName}/${memory.id}${ext}`;
        archive.file(filePath, { name: archiveName });
        }
      });
    archive.finalize();
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

// Settings save
app.post('/api/admin/settings', (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    if (!hasPerm(auth.user, 'settings')) return res.status(403).json({ success: false, error: 'Forbidden' });

    const incoming = req.body?.settings;
    if (!incoming || typeof incoming !== 'object') return res.status(400).json({ success: false, error: 'Missing settings object' });

    const validFields = ['fileuploadST', 'uploadWindowStartST', 'uploadWindowEndST', 'autoApproveStartST', 'autoApproveEndST',
    'maxUploadMB', 'allowedFileTypes',
    if (incoming[f].match /^([0-9]{1-9}[:2]-[0-2]?[4-2]:[1-2][0-9]$/.test(incoming[f])) {
      return res.status(400).json({ success: false, error: 'Invalid ${f} format. Use YYYY-MM-DDTHH:mm' });
      }
    }

    if (incoming.theme && !hasPerm(auth.user, 'theme')) {
      return res.status(403).json({ success: false, error: 'No permission to edit theme' });
      }

    writeSettings({ settings: incoming });
    audit(auth.user.id, 'save-settings', {});
    broadcast('settings:update', {});

    res.json({ success: true });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

// Change own password
app.post('/api/admin/change-password', (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { oldPassword, newPassword } = req.body || {};
    if (!oldPassword || !newPassword) return res.status(400).json({ success: false, error: 'Missing oldPassword or newPassword' });
    if (String(newPassword).length < 8) return res.status(400).json({ success: false, error: 'Password must be at least 8 characters' });

    const admins = readAdmins();
    const a = admins.users.find(x => x.id === auth.user.id);
    if (!a) return res.status(404).json({ success: false, error: 'User not found' });

    if (!a.password || !verifyPassword(oldPassword, a.password)) return res.status(400).json({ success: false, error: 'Old password incorrect' });
    a.password = String(newPassword);
    a.updatedAt = nowIso();
    writeAdmins(admins);

    const sessions = readSessions();
    sessions.sessions = sessions.sessions.filter(s => s.userId !== a.id);
    writeSessions(sessions);

    audit(auth.user.id, 'change-password', {});
    res.json({ success: true });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

// User management: list
app.get('/api/admin/users', (req, res) => {
  const auth = requireAdmin(req, res);
  if (!auth) return;
  if (!hasPerm(auth.user, 'users')) return res.status(403).json({ success: false, error: 'Forbidden' });

  const admins = readAdmins();
  res.json({
  success: true,
  users: admins.users.map(u => ({
    id: u.id,
    name: u.name,
    role: u.role,
    disabled: !!u.disabled,
    permissions: u.permissions,
    createdAt: u.createdAt,
    updatedAt: u.updatedAt
    })
  });
});

// User management: create
```

```
app.post('/api/admin/users', (req, res) => {
  const auth = requireAdmin(req, res);
  if (!auth) return;
  if (!hasPerm(auth.user, 'users')) return res.status(403).json({ success: false, error: 'Forbidden' });

  const { id, name, password, role, permissions } = req.body || {};
  const uid = String(id || '').trim();
  if (!uid || uid.length < 3) return res.status(400).json({ success: false, error: 'id too short' });
  if (!/^[a-zA-Z0-9_-]+$/.test(uid)) return res.status(400).json({ success: false, error: 'id must be alnum/_-' });
  if (!password || String(password).length < 8) return res.status(400).json({ success: false, error: 'password min 8 chars' });

  const admins = readAdmins();
  if (admins.users.find(u => u.id === uid)) return res.status(409).json({ success: false, error: 'User already exists' });

  const now = nowIsoZ();
  admins.users.push({
    id: uid,
    name: String(name || uid).trim().substring(0, 60),
    role: role === 'admin' ? 'admin' : 'moderator',
    password: String(password),
    createdAt: now,
    updatedAt: now,
    disabled: false,
    permissions: typeof permissions === 'object' && permissions ? permissions : {
      moderation: true,
      settings: false,
      backup: false,
      export: false,
      users: false,
      trash: true,
      replaceFile: false,
      editMemory: true,
      bulk: false,
      featured: false
    }
  });

  writeAdmins(admins);
  audit(auth.user.id, 'create-user', { id: uid });
  res.json({ success: true });
});

// User management: update
app.post('/api/admin/users/:id', (req, res) => {
  const auth = requireAdmin(req, res);
  if (!auth) return;
  if (!hasPerm(auth.user, 'users')) return res.status(403).json({ success: false, error: 'Forbidden' });

  const uid = String(req.params.id);
  if (uid === 'super') return res.status(400).json({ success: false, error: 'Cannot modify super via this endpoint' });

  const { name, password, disabled, permissions } = req.body || {};
  const admins = readAdmins();
  const u = admins.users.find(x => x.id === uid);
  if (!u) return res.status(404).json({ success: false, error: 'User not found' });

  if (typeof name === 'string') u.name = name.trim().substring(0, 60);
  if (typeof password === 'string' && password.length >= 8) u.password = password;
  if (typeof disabled === 'boolean') u.disabled = disabled;
  if (permissions && typeof permissions === 'object') u.permissions = permissions;

  u.updatedAt = nowIsoZ();
  writeAdmins(admins);

  const sessions = readSessions();
  sessions.sessions = sessions.sessions.filter(s => s.userId !== uid);
  writeSessions(sessions);

  audit(auth.user.id, 'update-user', { id: uid });
  res.json({ success: true });
});

// Export CSV
app.get('/api/admin/export.csv', (req, res) => {
  const auth = requireAdmin(req, res);
  if (!auth) return;
  if (!hasPerm(auth.user, 'export')) return res.status(403).json({ success: false, error: 'Forbidden' });

  const db = readDB();
  const rows = db.memories.filter(m => !m.purgedAt).map(m => ({
    id: m.id,
    student_name: m.student_name,
    caption: m.caption,
    memory_type: m.memory_type,
    approved: m.approved,
    featured: m.featured || 0,
    likes: m.likes || 0,
    file_name: m.file_name,
    file_path: m.file_path,
    file_type: m.file_type,
    mime: m.mime,
    sha256: m.sha256,
    created_at: m.created_at,
    deletedAt: m.deletedAt || ''
    });

  const header = Object.keys(rows[0] || { id: '' });
  const lines = [header.join(',')];
  for (const r of rows) {
    lines.push(header.map(k => '"${sanitizeCsvCell(r[k])}"').join(','));
  }

  res.setHeader('Content-Type', 'text/csv; charset=utf-8');
  res.setHeader('Content-Disposition', 'attachment; filename="memories-export-${new Date().toISOString().slice(0, 10)}.csv"');
  res.send(lines.join('\n'));
});


// ═══════════════════════════════════════════════════════════════════════════
// COMPILATIONS API
// ═══════════════════════════════════════════════════════════════════════════

// Get all compilations (public)
app.get('/api/compilations', (req, res) => {
  try {
    const data = readCompilations();
    res.json({ success: true, compilations: data.compilations || [] });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

// Get single compilation (public)
app.get('/api/compilations/:id', (req, res) => {
  try {
    const id = parseInt(req.params.id, 10);
    const data = readCompilations();
    const comp = data.compilations.find(c => c.id === id);
    if (!comp) return res.status(404).json({ success: false, error: 'Not found' });
    res.json({ success: true, compilation: comp });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

// Create compilation (admin)
app.post('/api/admin/compilations', (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;

    const { name, slides, displayMode, transitionType } = req.body || {};
    if (!name || !Array.isArray(slides)) {
      return res.status(400).json({ success: false, error: 'name and slides required' });
    }

    const data = readCompilations();
    const compilation = {
      id: data.nextId++,
      name: String(name).trim().substring(0, 100),
      slides: slides.map(s => ({
        memoryId: Number(s.memoryId),
        caption: String(s.caption || '').trim().substring(0, 300),
        duration: Math.min(60, Math.max(1, Number(s.duration) || 5))
      })),
      displayMode: displayMode === 'manual' ? 'manual' : 'auto',
      transitionType: ['fade', 'slide', 'zoom', 'flip'].includes(transitionType) ? transitionType : 'fade',
      createdAt: nowIsoZ(),
      updatedAt: nowIsoZ()
      };

    data.compilations.push(compilation);
    writeCompilations(data);

    audit(auth.user.id, 'create-compilation', { id: compilation.id });
    res.json({ success: true, compilation });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

// Update compilation (admin)
app.put('/api/admin/compilations/:id', (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;

    const id = parseInt(req.params.id, 10);
    const data = readCompilations();
    const comp = data.compilations.find(c => c.id === id);
    if (!comp) return res.status(404).json({ success: false, error: 'Not found' });

    const { name, slides, displayMode, transitionType } = req.body || {};
    if (name) comp.name = String(name).trim().substring(0, 100);
    if (Array.isArray(slides)) {
      comp.slides = slides.map(s => ({
        memoryId: Number(s.memoryId),
        caption: String(s.caption || '').trim().substring(0, 300),
        duration: Math.min(60, Math.max(1, Number(s.duration) || 5))
      }));
    }
    if (displayMode) comp.displayMode = displayMode === 'manual' ? 'manual' : 'auto';
    if (transitionType && ['fade', 'slide', 'zoom', 'flip'].includes(transitionType)) {
      comp.transitionType = transitionType;
    }
    comp.updatedAt = nowIsoZ();

    writeCompilations(data);
    audit(auth.user.id, 'update-compilation', { id });
    res.json({ success: true, compilation: comp });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

// Delete compilation (admin)
app.delete('/api/admin/compilations/:id', (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;

    const id = parseInt(req.params.id, 10);
    const data = readCompilations();
    const idx = data.compilations.findIndex(c => c.id === id);
    if (idx === -1) return res.status(404).json({ success: false, error: 'Not found' });

    data.compilations.splice(idx, 1);
    writeCompilations(data);

    audit(auth.user.id, 'delete-compilation', { id });
    res.json({ success: true });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

// Edit comment (admin)
app.put('/api/admin/comments/:id', (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    if (!hasPerm(auth.user, 'moderation')) return res.status(403).json({ success: false, error: 'Forbidden' });

    const id = parseInt(req.params.id, 10);
    const { text, deleted } = req.body || {};

    const db = readDB();
    const comment = db.comments.find(c => c.id === id);
    if (!comment) return res.status(404).json({ success: false, error: 'Comment not found' });

    if (typeof text === 'string') comment.text = text.trim().substring(0, 800);
    if (deleted === true) comment.deletedAt = nowIsoZ();
    if (deleted === false) comment.deletedAt = null;

    writeComments(db);
    audit(auth.user.id, 'edit-comment', { commentId: id });

    res.json({ success: true, comment });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

// Upload intro video (admin)
app.post('/api/admin/upload-intro-video', adminUpload.single('video'), (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    if (!hasPerm(auth.user, 'settings')) return res.status(403).json({ success: false, error: 'Forbidden' });

    const file = req.file;
    if (!file) return res.status(400).json({ success: false, error: 'No video file' });

    // Save path to settings
    const settings = readSettings();
    settings.settings = settings.settings || {};
    settings.settings.introVideoPath = file.filename;
    writeSettings(settings);

    audit(auth.user.id, 'upload-intro-video', { filename: file.filename });
    res.json({ success: true, path: file.filename });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

// Upload favicon (admin)
app.post('/api/admin/upload-favicon', upload.single('favicon'), (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    if (!hasPerm(auth.user, 'settings')) return res.status(403).json({ success: false, error: 'Forbidden' });

    const file = req.file;
    if (!file) return res.status(400).json({ success: false, error: 'No file' });

    // Copy to favicon.ico in root
    const faviconDest = path.join(__dirname, 'favicon.ico');
    fs.copyFileSync(path.join(uploadsDir, file.filename), faviconDest);

    const settings = readSettings();
    settings.settings = settings.settings || {};
    settings.settings.faviconUploaded = true;
    writeSettings(settings);

    audit(auth.user.id, 'upload-favicon', {});
    res.json({ success: true });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

// Upload teacher image (admin)
app.post('/api/admin/upload-teacher-image', upload.single('image'), (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;

    const file = req.file;
    if (!file) return res.status(400).json({ success: false, error: 'No image file' });

    res.json({ success: true, url: '/uploads/${file.filename}' });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});
```

```
// ═══════════════════════════════════════════════════════════════════════════
// SERVE FRONTEND
// ═══════════════════════════════════════════════════════════════════════════

app.get("/", (req, res) => res.sendFile(path.join(__dirname, 'index.html')));
app.get("*", (req, res) => res.sendFile(path.join(__dirname, 'index.html')));

// ═══════════════════════════════════════════════════════════════════════════
// ERROR HANDLING
// ═══════════════════════════════════════════════════════════════════════════

app.use((err, req, res, next) => {
  console.error('Server error:', err);

  if (err instanceof multer.MulterError) {
    if (err.code === 'LIMIT_FILE_SIZE') {
      return res.status(400).json({ success: false, error: `File too large. Maximum size is ${MAX_FILE_SIZE / 1024 / 1024} MB per file.` });
    }
    if (err.code === 'LIMIT_FILE_COUNT') {
      return res.status(400).json({ success: false, error: `Too many files. Maximum is ${MAX_FILES} files per upload.` });
    }
  }
  res.status(500).json({ success: false, error: err.message || 'Internal server error' });
});

// ═══════════════════════════════════════════════════════════════════════════
// START SERVER
// ═══════════════════════════════════════════════════════════════════════════



// === SNIPER_SERVER_PATCH_V2_START ===
(()=>{
  if (global.__SNIPER_SERVER_PATCH_V2__) return;
  global.__SNIPER_SERVER_PATCH_V2__ = true;

  const teacherAudioPath = path.join(dataBaseDir, 'teacher_audio.json');
  const paperNotesPath = path.join(dataBaseDir, 'paper_notes.json');
  const destinationsPath = path.join(dataBaseDir, 'destinations.json');

  function ensurePerfDB(dbFilePath, fallback) {
    if (!fs.existsSync(dbFilePath)) safeWriteJson(dbFilePath, fallback);
  }

  ensurePerfDB(teacherAudioPath, { items: [], nextId: 1 });
  ensurePerfDB(paperNotesPath, { notes: [], nextId: 1 });
  ensurePerfDB(destinationsPath, { destinations: [], submissions: [], nextId: 1 });

  function readTeacherAudio() {
    return safeReadJson(teacherAudioPath, { items: [], nextId: 1 });
  }
  function writeTeacherAudioDB(data) {
    safeWriteJson(teacherAudioPath, data);
  }
  function readPaperNotes() {
    return safeReadJson(paperNotesPath, { notes: [], nextId: 1 });
  }
  function writePaperNotesDB(data) {
    safeWriteJson(paperNotesPath, data);
  }
  function readDestinationsDB() {
    return safeReadJson(destinationsPath, { destinations: [], submissions: [], nextId: 1 });
  }
  function writeDestinationsDB(data) {
    safeWriteJson(destinationsPath, data);
  }

  function parseBearer(req) {
    return req.headers.authorization?.replace('Bearer ', '') || '';
  }

  function sendWs(event, payload) {
    try { broadcast(event, payload); } catch (_) {}
  }

  // Route aliases for frontend compatibility
  app.post('/api/admin/settings/intro-video', adminUpload.single('file'), (req, res) => {
    try {
      const auth = requireAdmin(req, res);
      if (!auth) return;
      if (!hasPerm(auth.user, 'settings')) return res.status(403).json({ success: false, error: 'Forbidden' });
      const file = req.file;
      if (!file) return res.status(400).json({ success: false, error: 'No video file' });
      const settings = readSettings();
      settings.settings = settings.settings || {};
      settings.settings.introVideoPath = file.filename;
      writeSettings(settings);
      audit(auth.user.id, 'upload-intro-video-alias', { filename: file.filename });
      sendWs('settings:update', {});
      res.json({ success: true, path: file.filename });
    } catch (e) {
      res.status(500).json({ success: false, error: e.message });
    }
  });

  app.delete('/api/admin/settings/intro-video', (req, res) => {
    try {
      const auth = requireAdmin(req, res);
      if (!auth) return;
      if (!hasPerm(auth.user, 'settings')) return res.status(403).json({ success: false, error: 'Forbidden' });
      const settings = readSettings();
      settings.settings = settings.settings || {};
      const old = settings.settings.introVideoPath;
      settings.settings.introVideoPath = null;
      writeSettings(settings);
      if (old) {
        const fp = path.join(uploadDir, old);
        if (fs.existsSync(fp)) {
          try { fs.unlinkSync(fp); } catch (_) {}
        }
      }
      audit(auth.user.id, 'remove-intro-video-alias', {});
      sendWs('settings:update', {});
      res.json({ success: true });
    } catch (e) {
      res.status(500).json({ success: false, error: e.message });
    }
  });

  // Teacher audio
  app.get('/api/teacher-audio', (req, res) => {
    try {
      const db = readTeacherAudio();
      res.json({ success: true, items: db.items || [] });
    } catch (e) {
      res.status(500).json({ success: false, error: e.message });
    }
  });

  app.post('/api/admin/teacher-audio', adminUpload.single('audio'), (req, res) => {
    try {
      const auth = requireAdmin(req, res);
      if (!auth) return;
      if (!hasPerm(auth.user, 'settings')) return res.status(403).json({ success: false, error: 'Forbidden' });
      const teacherName = String(req.body.teacherName || '').trim();
      const file = req.file;
      if (!teacherName) return res.status(400).json({ success: false, error: 'teacherName required' });
      if (!file) return res.status(400).json({ success: false, error: 'audio file required' });

      const db = readTeacherAudio();
      let item = db.items.find(x => String(x.teacherName || '').toLowerCase() === teacherName.toLowerCase());
      if (!item) {
        item = { id: db.nextId++, teacherName, audioPath: file.filename, createdAt: nowIso(), updatedAt: nowIso() };
        db.items.push(item);
      } else {
        const old = item.audioPath;
        item.audioPath = file.filename;
        item.updatedAt = nowIso();
        if (old) {
          const fp = path.join(uploadDir, old);
          if (fs.existsSync(fp)) {
            try { fs.unlinkSync(fp); } catch (_) {}
          }
        }
      }
      writeTeacherAudioDB();

      const settings = readSettings();
      settings.settings = settings.settings || {};
      settings.settings.teachers = Array.isArray(settings.settings.teachers) ? settings.settings.teachers : [];
      const teacher = settings.settings.teachers.find(t => String(t.name || '').toLowerCase() === teacherName.toLowerCase());
      if (teacher) teacher.audioPath = file.filename;
      writeSettings(settings);

      audit(auth.user.id, 'upload-teacher-audio', { teacherName });
      sendWs('settings:update', {});
      res.json({ success: true, item });
    } catch (e) {
      res.status(500).json({ success: false, error: e.message });
    }
  });

  // Destinations allowlist and submissions
  app.get('/api/destinations', (req, res) => {
    try {
      const db = readDestinationsDB();
      res.json({ success: true, destinations: db.destinations || [] });
    } catch (e) {
      res.status(500).json({ success: false, error: e.message });
    }
  });

  app.post('/api/admin/destinations', (req, res) => {
    try {
      const auth = requireAdmin(req, res);
      if (!auth) return;
      if (!hasPerm(auth.user, 'settings')) return res.status(403).json({ success: false, error: 'Forbidden' });
      const list = Array.isArray(req.body?.destinations) ? req.body.destinations : [];
      const cleaned = [...
        .map(x => String(x).trim())
        .filter(Boolean)
        .slice(0, 500)
        .map(name => ({ name }));

      const db = readDestinationsDB();
      db.destinations = cleaned;
      writeDestinationsDB(db);
      audit(auth.user.id, 'save-destinations', { count: cleaned.length });
      res.json({ success: true, count: cleaned.length });
    } catch (e) {
      res.status(500).json({ success: false, error: e.message });
    }
  });

  app.post('/api/destinations/submit', (req, res) => {
    try {
      const destination = String(req.body?.destination || '').trim();
      if (!destination) return res.status(400).json({ success: false, error: 'destination required' });
      const db = readDestinationsDB();
      const ok = db.destinations.find(x => String(x.name || '').trim().toLowerCase() === destination.toLowerCase());
      if (!ok) return res.status(400).json({ success: false, error: 'Destination is not allowed' });

      db.submissions.push({
        id: db.nextId++,
        destination,
        ip: req.headers['x-forwarded-for'] || req.socket.remoteAddress || '',
        createdAt: nowIso()
      });
      if (db.submissions.length > 5000) db.submissions = db.submissions.slice(-5000);
      writeDestinationsDB(db);
      res.json({ success: true });
    } catch (e) {
      res.status(500).json({ success: false, error: e.message });
    }
  });

  // Paper notes
  app.post('/api/paper-notes', (req, res) => {
    try {
      const text = String(req.body?.text || '').trim().substring(0, 400);
      if (!text) return res.status(400).json({ success: false, error: 'text required' });
      const db = readPaperNotes();
      const note = {
        id: db.nextId++,
        text,
        ip: req.headers['x-forwarded-for'] || req.socket.remoteAddress || '',
        createdAt: nowIso()
      };
      db.notes.push(note);
      if (db.notes.length > 3000) db.notes = db.notes.slice(-3000);
      writePaperNotesDB();
      sendWs('paper:note', { id: note.id });
      res.json({ success: true, noteId: note.id });
    } catch (e) {
      res.status(500).json({ success: false, error: e.message });
    }
  });

  app.get('/api/paper-notes/random', (req, res) => {
    try {
      const db = readPaperNotes();
      const arr = db.notes || [];
      if (!arr.length) return res.json({ success: true, note: null });
      const note = arr[Math.floor(Math.random() * arr.length)];
      res.json({ success: true, note });
    } catch (e) {
      res.status(500).json({ success: false, error: e.message });
    }
  });

  // Extra WebSocket message handling for ghost cursors
  wss.on('connection', (ws) => {
    ws.on('message', (raw) => {
      let data = null;
      try { data = JSON.parse(String(raw)); } catch (_) { return; }
      if (!data || typeof data !== 'object') return;

      if (data.type === 'ghost:move') {
        sendWs('ghost:move', {
          id: String(data.id || '').slice(0, 64),
          x: Number(data.x || 0),
          y: Number(data.y || 0),
          initials: String(data.initials || 'GS').slice(0, 4)
        });
      }
    });
  });

  console.log('SNIPER patch server extension loaded.');
})();



// === SNIPER_SERVER_PATCH_V3_START ===
(()=>{
  if (global.__SNIPER_SERVER_PATCH_V3__) return;
  global.__SNIPER_SERVER_PATCH_V3__ = true;

  const musicDir = path.join(__dirname, 'music');
  if (!fs.existsSync(musicDir) {
    fs.mkdirSync(musicDir, { recursive: true });
  }

  app.use('/music', express.static(musicDir));

  function readPaperNotesV3() {
    const paperNotesPath = path.join(dataBaseDir, 'paper_notes.json');
    return safeReadJson(paperNotesPath, { notes: [], nextId: 1 });
  }

  function writePaperNotesV3(data) {
    const paperNotesPath = path.join(dataBaseDir, 'paper_notes.json');
    safeWriteJson(paperNotesPath, data);
  }

  function getApprovedMemoryById(memoryId) {
    const db = readDB();
    return db.memories.find(m => m.id === Number(memoryId) && m.approved === 1 && !m.deletedAt && !m.purgedAt) || null;
  }

  app.post('/api/paper-notes/from-memory', (req, res) => {
    try {
      const memoryId = Number(req.body?.memoryId || 0);
      if (!memoryId) return res.status(400).json({ success: false, error: 'memoryId required' });
      const memory = getApprovedMemoryById(memoryId);
      if (!memory) return res.status(404).json({ success: false, error: 'Approved memory not found' });
```

```
  const notes = readPaperNotesV3();
  const note = {
    id: notes.nextId++,
    memoryId: memoryId,
    text: String(memory.caption || '').trim().substring(0, 400),
    caption: String(memory.caption || '').trim().substring(0, 400),
    createdAt: nowIso()
  };
  notes.notes.push(note);
  if (notes.notes.length > 3000) notes.notes = notes.notes.slice(-3000);
  writePaperNotesV3(notes);
  broadcast('paper:note', { id: note.id, memoryId: note.memoryId });
  res.json({ success: true, note });
} catch (e) {
  res.status(500).json({ success: false, error: e.message });
}
});

app.get('/api/paper-notes/random-memory', (req, res) => {
  try {
    const notes = readPaperNotesV3();
    const eligible = (notes.notes || []).filter(n => n.memoryId && n.caption);
    if (!eligible.length) return res.json({ success: true, note: null });
    const note = eligible[Math.floor(Math.random() * eligible.length)];
    res.json({ success: true, note });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

console.log('SNIPER patch server v3 loaded.');
})();



// == SNIPER_SERVER_PATCH_V4_START ==
(() => {
  if (global.__SNIPER_SERVER_PATCH_V4__) return;
  global.__SNIPER_SERVER_PATCH_V4__ = true;

  const destinationsPathV4 = path.join(dataDir, 'destinations.json');

  function readDestinationsV4() {
    return safeReadJson(destinationsPathV4, { destinations: [], submissions: [], nextId: 1 });
  }
  function writeDestinationsV4(data) {
    safeWriteJson(destinationsPathV4, data);
  }

  app.post('/api/admin/destinations-v2', (req, res) => {
    try {
      const auth = requireAdmin(req, res);
      if (!auth) return;
      if (!hasPerm(auth.user, 'settings')) return res.status(403).json({ success: false, error: 'Forbidden' });

      const places = Array.isArray(req.body?.places) ? req.body.places : [];
      const cleaned = [];
      const seen = new Set();
      for (const p of places) {
        if (!p || typeof p !== 'object') continue;
        const id = String(p.id || '').trim();
        if (!id || seen.has(id)) continue;
        seen.add(id);
        const name = String(p.name || '').trim().substring(0, 120);
        const university = String(p.university || '').trim().substring(0, 120);
        if (!name && !university) continue;
        cleaned.push({ id, name, university });
      }
      if (cleaned.length > 500) cleaned = cleaned.slice(-500);
      const db = readDestinationsV4();
      db.updatedAt = nowIso();
      writeDestinationsV4(db);
      audit(auth.user.id, 'save-destinations-v2', { count: cleaned.length });
      res.json({ success: true, count: cleaned.length });
    } catch (e) {
      res.status(500).json({ success: false, error: e.message });
    }
  });

  app.post('/api/destinations/submit-v2', (req, res) => {
    try {
      const studentName = String(req.body?.studentName || '').trim().substring(0, 80);
      const schoolPlace = String(req.body?.schoolPlace || '').trim().substring(0, 120);
      const universityPlace = String(req.body?.universityPlace || '').trim().substring(0, 120);

      if (!studentName) return res.status(400).json({ success: false, error: 'studentName required' });
      if (!schoolPlace && !universityPlace) return res.status(400).json({ success: false, error: 'At least one place is required' });

      const db = readDestinationsV4();
      const allowed = new Set(db.destinations || [].map(p => String(p.place || '').trim().toLowerCase()));

      if (schoolPlace && !allowed.has(schoolPlace.toLowerCase())) {
        return res.status(400).json({ success: false, error: 'School place is not allowed' });
      }
      if (universityPlace && !allowed.has(universityPlace.toLowerCase())) {
        return res.status(400).json({ success: false, error: 'University place is not allowed' });
      }

      const existing = db.submissions || [].find(s => String(s.studentName || '').trim().toLowerCase() === studentName.toLowerCase());
      if (existing) {
        existing.schoolPlace = schoolPlace;
        existing.universityPlace = universityPlace;
        existing.updatedAt = nowIso();
      } else {
        db.submissions.push({
          id: db.nextId++,
          studentName,
          schoolPlace,
          universityPlace,
          createdAt: nowIso(),
          updatedAt: nowIso()
        });
      }

      if (db.submissions.length > 5000) db.submissions = db.submissions.slice(-5000);
      writeDestinationsV4(db);
      broadcast('destinations:update', { studentName });
      res.json({ success: true });
    } catch (e) {
      res.status(500).json({ success: false, error: e.message });
    }
  });

  app.get('/api/destinations/submissions', (req, res) => {
    try {
      const db = readDestinationsV4();
      res.json({ success: true, submissions: db.submissions || [] });
    } catch (e) {
      res.status(500).json({ success: false, error: e.message });
    }
  });

  console.log('SNIPER patch server v4 loaded.');
})();



// == SNIPER_SERVER_PATCH_V5_START ==
(() => {
  if (global.__SNIPER_SERVER_PATCH_V5__) return;
  global.__SNIPER_SERVER_PATCH_V5__ = true;

  const destinationsPathV5 = path.join(dataDir, 'destinations.json');

  function readDestinationsV5() {
    return safeReadJson(destinationsPathV5, { destinations: [], submissions: [], nextId: 1 });
  }
  function writeDestinationsV5(data) {
    safeWriteJson(destinationsPathV5, data);
  }

  app.post('/api/destinations/pin-submit', (req, res) => {
    try {
      const studentName = String(req.body?.studentName || '').trim().substring(0, 80);
      const schoolPoint = req.body?.schoolPoint || null;
      const universityPoint = req.body?.universityPoint || null;

      if (!studentName) return res.status(400).json({ success: false, error: 'studentName required' });

      const validPoint = (p) => p && Number.isFinite(Number(p.lat)) && Number.isFinite(Number(p.lng));

      if (!validPoint(schoolPoint) && !validPoint(universityPoint)) {
        return res.status(400).json({ success: false, error: 'At least one valid point is required' });
      }

      const db = readDestinationsV5();
      let existing = db.submissions || [].find(s => String(s.studentName || '').trim().toLowerCase() === studentName.toLowerCase());

      const payload = {
        studentName,
        schoolPoint: validPoint(schoolPoint) ? { lat: Number(schoolPoint.lat), lng: Number(schoolPoint.lng) } : null,
        universityPoint: validPoint(universityPoint) ? { lat: Number(universityPoint.lat), lng: Number(universityPoint.lng) } : null,
        updatedAt: nowIso()
      };

      if (existing) {
        existing.studentName = payload.studentName;
        existing.schoolPoint = payload.schoolPoint;
        existing.universityPoint = payload.universityPoint;
        existing.updatedAt = payload.updatedAt;
      } else {
        db.submissions.push({
          id: db.nextId++,
          createdAt: nowIso(),
          ...payload
        });
      }

      if (db.submissions.length > 5000) db.submissions = db.submissions.slice(-5000);
      writeDestinationsV5(db);
      broadcast('destinations:pin-update', { studentName });
      res.json({ success: true });
    } catch (e) {
      res.status(500).json({ success: false, error: e.message });
    }
  });

  app.get('/api/destinations/pin-submissions', (req, res) => {
    try {
      const db = readDestinationsV5();
      res.json({ success: true, submissions: db.submissions || [] });
    } catch (e) {
      res.status(500).json({ success: false, error: e.message });
    }
  });

  console.log('SNIPER patch server v5 loaded.');
})();



// == SNIPER_SERVER_PATCH_V6_START ==
(() => {
  if (global.__SNIPER_SERVER_PATCH_V6__) return;
  global.__SNIPER_SERVER_PATCH_V6__ = true;
  console.log('SNIPER patch server v6 loaded.');
})();


server.listen(PORT, '0.0.0.0', () => {
  console.log('');
  console.log('═══────────────────────────────────────────────────────────────────────────────────────────────────────────────');
  console.log('  CORNERSTONE INTERNATIONAL SCHOOL - FAREWELL 2025');
  console.log('═══────────────────────────────────────────────────────────────────────────────────────────────────────────────');
  console.log('  Server running on: http://localhost:${PORT}');
  console.log('  Network access: http://0.0.0.0:${PORT}');
  console.log('  Uploads folder: ${uploadsDir}');
  console.log('  Memories DB: ${dbPath}');
  console.log('  Settings DB: ${settingsPath}');
  console.log('  Admin DB: ${adminPath}');
  console.log('  Paper notes DB: ${paperNotesPath}');
  console.log('  Reactions DB: ${reactionsPath}');
  console.log('  Audit DB: ${auditPath}');
  console.log('  Max upload size: ${MAX_TOTAL_SIZE / 1024 / 1024}MB total');
  console.log('  Profanity filter: bad-words + obscenity + custom patterns');
  console.log('═══────────────────────────────────────────────────────────────────────────────────────────────────────────────');
  console.log('');
});

// Graceful shutdown
process.on('SIGTERM', () => {
  console.log('  SIGTERM received. Closing server...');
  process.exit(0);
});
process.on('SIGINT', () => {
  console.log('  SIGINT received. Closing server...');
  process.exit(0);
});
```